Mälardalen University Doctoral Dissertation 466

A SYSTEMATIC FRAMEWORK FOR FAULT MANAGEMENT IN EMBEDDED SYSTEMS FOR MOBILE NETWORKS

PROBABILISTIC MODELING, COST-AWARE MITIGATION, AND BOARD-LEVEL DESIGN

Carlo Vitucci

Mälardalen University

Mälardalen University Press Dissertations
No. 466

A SYSTEMATIC FRAMEWORK FOR FAULT MANAGEMENT IN EMBEDDED SYSTEMS FOR MOBILE NETWORKS

PROBABILISTIC MODELING, COST-AWARE MITIGATION, AND BOARD-LEVEL DESIGN

Carlo Vitucci

2026



Department of Computer Science & Engineering

Copyright © Carlo Vitucci, 2026
ISBN 978-91-7485-757-3
ISSN 1651-4238
Printed by E-Print AB, Stockholm, Sweden

Mälardalen University Press Dissertations
No. 466

A SYSTEMATIC FRAMEWORK FOR FAULT MANAGEMENT IN EMBEDDED SYSTEMS FOR MOBILE NETWORKS
PROBABILISTIC MODELING, COST-AWARE MITIGATION, AND BOARD-LEVEL DESIGN

Carlo Vitucci

Akademisk avhandling

som för avläggande av teknologie doktorsexamen i datavetenskap vid Institutionen för datavetenskap och datateknik kommer att offentligen försvaras tisdagen den 9 juni 2026, 13.15 i Lambda, och digitalt, Mälardalens universitet, Västerås.

Fakultetsopponent: Professor Luis Almeida, Universidade de Porto



Institutionen för datavetenskap och datateknik

Abstract

The evolution of telecommunications networks continues without interruption, transforming society and redefining many of the services we've become familiar with, often improving the quality of life. Connectivity, mobility, and the network's ability to support new services have changed the way we perform daily activities: reading a newspaper, managing a bank account, playing games, or communicating. And technologies like IoT, artificial intelligence, virtualization, and the widespread availability of data are paving the way for services that are even difficult to imagine today.

As services become increasingly central and critical, their availability over time cannot be compromised without a significant impact. Disruption can cause a severe setback for those who rely on those services. Network resilience becomes a key requirement for next-generation mobile networks. But what does it mean to have a resilient network? What makes a system resilient, and how can we improve it? Our research starts with these key questions. While using the classic definition of resilience (the ability of a system to respond rapidly to internal or external stresses that compromise performance), we argue that improving it requires identifying the system functions that contribute to resilience and quantifying their contributions.There is also a connection to reliability. Reliability describes a system's ability to maintain expected performance over time under normal conditions. In contrast, resilience is the ability to withstand abnormal conditions and restore proper operation. Both involve managing faults. Increasing service availability means reducing the probability of faults and the time and costs to resolve them.

In this sense, if resilience is the system's ability to respond to an internal or external event that compromises its performance, then fault management (the process of detecting, isolating, and correcting faults) is the system function through which we address faults and seek to resolve them.Our research, as presented in this thesis, produced several key outcomes. We defined a resilience model for mobile network nodes, explored the concept of failure, and developed a flexible method for calculating failure probability. We investigated its probability and the costs to mitigate its impact. Based on these results, we proposed a framework for fault management and identified an optimal hardware design to improve reliability and resilience. Maintaining a holistic approach was necessary to develop a fault management model for all system levels—hardware, firmware, and software. Each layer works autonomously when possible or with others to achieve fault management's essential goal: recovery from a faulty condition. This leads to board-level design requirements for high-quality service and experience.

ISBN 978-91-7485-757-3
ISSN 1651-4238

Failure is not an option

Apollo 13 Flight Director Gene Kranz

Acknowledgment

I would like to thank Ericsson's management and my colleagues for making my industrial PhD possible. Special thanks go to my line manager, John Grenholm, for his constant support, and to the colleagues in Tech Management for their patience and solidarity. I owe them the professional growth I achieved throughout my academic journey: not only technical competence, but also a clearer perspective on research, which I am sure will be an asset to my future work. The balance between industrial needs and research activities was made possible by the invaluable contributions of Marcus Jägemar, my tireless internal supervisor, to whom I extend my sincere gratitude.

Equally heartfelt thanks to Mälardalen University (MDU). From day one, I knew it would become my scientific home: a place where students, researchers, and professors work in extraordinary harmony to improve research, individual expertise, and the university's role in society. Enormous thanks to my supervisors, Daniel Sundmark and Thomas Nolte, for guiding me on this journey. A warm embrace to everyone — students, colleagues, and faculty — with whom I had the pleasure of sharing a course, a chat during a coffee break, or a lunch.

Thank you to the co-authors who helped increase the quality of my research. The sharing of ideas is the foundation of good science, and my work was no exception. The time spent at a whiteboard, trying to give shape to the seed of an idea, is the best part of any project and the part that most endures in the memory of those who took part. Among many, I would like to thank in particular Professor Thomas Westerbäck of MDU and my colleague and dear friend Giovanni Fiaschi of Ericsson.

Thanks to the scientific communities that welcomed the results of our re-

search. I particularly remember the community of the International Conference on System Reliability and Safety, excellently chaired by Professor Enrico Zio of Politecnico di Milano, which I attended regularly throughout my studies.

Thanks to my friends for the laughter and for the time that is never truly wasted.

Infinite thanks to my family for allowing me to become who I am.

A kiss and a hug to my daughter and my wife.

Carlo Vitucci
Västerås, June, 2026

List of Publications

Primary Papers

The following published papers during the doctoral studies constitute the main input for the thesis. Findings and results of these papers are used throughout the thesis and referenced accordingly whenever used.

[1] **Paper 1:** Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Jakob Danielsson, Alf Larsson, Thomas Nolte. *A Reliability-Oriented Faults Taxonomy and a Recovery-Oriented Methodological Approach for Systems Resilience*. In IEEE 46th Annual Computers, Software, and Applications Conference, COMPSAC, 2022.

[2] **Paper 2:** Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Jakob Danielsson, Alf Larsson, Thomas Nolte. *Fault Management Framework And Multi-Layer Recovery Methodology for Resilient System*. In IEEE 6th International Conference on System Reliability and Safety, ICSRS, 2022.

[3] **Paper 3:** Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Jakob Danielsson, Alf Larsson, Thomas Nolte. *Fault Management Impacts on the Networking Systems Hardware Design*. In IEEE 49th Annual Conference of the IEEE Industrial Electronics Society, IECON, 2023.

[4] **Paper 4:** Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Jakob Danielsson, Alf Larsson, Thomas Nolte. *Run Time Memory Error Recovery Process in*

Networking System. In IEEE 7th International Conference on System Reliability and Safety, ICSRS, 2023.

[5] **Paper 5:** Selma Rahman, Mattias Olausson, Carlo Vitucci, Ioannis Avgouleas. *Ambient Temperature Prediction for Embedded Systems Using Machine Learning*. In Springer Nature 8th International Conference on Engineering of Computer-based Systems, ECBS, 2023.

[6] **Paper 6:** Carlo Vitucci, Thomas Westerbäck, Daniel Sundmark, Håkan Forsberg, Thomas Nolte, Marcus Jägemar. *A Deductive Fault Analysis Method Based on Hypergraphs*. In Springer Nature 12th IFAC Symposium on Fault Detection, Supervision and Safety for Technical Processes, SAFEPROCESS, 2024.

[7] **Paper 7:** Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Thomas Nolte. *A Fault Management Oriented Resilience Model for Networking Systems*. In IEEE 8th International Conference on System Reliability and Safety, ICSRS, 2024.

[8] **Paper 8:** Selma Rahman, Mattias Olausson, Carlo Vitucci, Ioannis Avgouleas. *Machine Learning-Based Ambient Temperature Prediction in Radio Access Network Environments*. In Springer International Journal on Software Tools for Technology Transfer, 2025.

[9] **Paper 9:** Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Thomas Nolte. *Low-Level Anomaly Detection in Embedded Systems Using Machine Learning*. In IEEE 11th International Conference on Computer Technology Application, ICCTA, 2025.

[10] **Paper 10:** Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Thomas Nolte. *Fault Management Cost System Analysis in Radio Access Network Embedded Systems*. In IEEE 9th International Conference on System Reliability and Safety, ICSRS, 2025.

[11] **Paper 11:** Giovanni Fiaschi, Carlo Vitucci, Thomas Westerbäck, Daniel Sundmark, Thomas Nolte. *A Formal Model for Path Set Attribute Calculation in Network Systems*. In IEEE 12th International Symposium on Networks, Computers and Communications, ISNCC, 2025.

Technical Reports and Other Works

The following publications served as preparatory contributions to this dissertation. They examined and investigated the technological context that prompted a reconsideration of the concepts of resilience and fault management within a RAN embedded system.

[12] **Licentiate Thesis:** Carlo Vitucci. *The Role of Fault Management in the Embedded System Design*. Ericsson AB - Mälardalen University, MDU, 2024.

[13] **Paper X1:** Carlo Vitucci, Juri Lelli, Andrea Parri, Mauro Marinoni. *A Linux-based Virtualized Solution Providing Computing Quality of Service to SDN-NFV Telecommunication Applications*. In 7th Franework Programme JUNIPER (FP7-ICT-2011.4.4), 2014.

[14] **Paper X2:** Carlo Vitucci, Alf Larsson. *HAT, Hardware Assisted Trace: Performance Oriented Trace & Debug System*. In 26th International Conference on software & systems engineering and their applications. ICSSEA, 2015.

[15] **Paper X3:** Carlo Vitucci, Alf Larsson. *Flexible 5G Edge Server for Multi Industry Service Network*. International Journal on Advances in Networks and Services, 2017.

[16] **Paper X4:** Carlo Vitucci, Alf Larsson. *SEED, A Server Platform for the Edge of the Network*. in the 16th International Conference on Networks, ICN, 2017.

[17] **Paper X5:** Tommaso Cucinotta, Mauro Marinoni, Alessandra Melani, Andrea Parri, Carlo Vitucci. *Temporal Isolation Among LTE/5G Network Functions by Real-Time Scheduling*. In proceedings of the 7th International Conference on Cloud Computing and Services Science, CLOSER, 2017.

[18] **Paper X6:** Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Alessio Balsini, Carlo Vitucci. *Virtual Network Functions as Real-Time Containers*

in Private Clouds. in IEEE International Conference on Cloud Computing, CLOUD, 2018.

[19] **Paper X7:** Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Carlo Vitucci. *The Importance of Being OS-aware: In Performance Aspects of Cloud Computing Research*. In proceedings of the 8th International Conference on Cloud Computing and Services Science, CLOSER, 2018.

[20] **Paper X8:** Mauro Marinoni, Tommaso Cucinotta, Luca Abeni, Carlo Vitucci. *Allocation and Control of Computing Resources for Real-time Virtual Network Functions*. In 17th International Conference on Networks, ICN, 2018.

[21] **Paper X9:** Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Alessio Balsini, Carlo Vitucci. *Reducing Temporal Interference in Private Clouds Through Real-Time Containers*. In proceedings of the IEEE International Conference on Edge Computing, EDGE, 2019.

[22] **Paper X10:** Carlo Vitucci, Luca Abeni, Tommaso Cucinotta, Mauro Marinoni. *The Strategic Role of Inter-Container Communications in RAN Deployment Scenarios*. In 18th International Conference of Networks, ICN, 2019

[23] **Paper X11:** Gabriele Ara, Luca Abeni, Tommaso Cucinotta, Carlo Vitucci. *On the Use of Kernel Bypass Mechanisms for High-Performance Inter-Container Communications*. In journal Lecture Notes in Computer Science, 2019.

[24] **Paper X12:** Gabriele Ara, Tommaso Cucinotta, Luca Abeni, Carlo Vitucci. *Comparative Evaluation of Kernel Bypass Mechanisms for High-Performance Inter-Container Communications*. In proceedings of the 10th International Conference on Cloud Computing and Services Science, CLOSER, 2020.

[25] **Paper X13:** Carlo Vitucci, Tommaso Cucinotta, Riccardo Mancini, Luca Abeni. *Implementation and Deployment of a Server at the Edge Using OpenStack Components*. In the 19th International Conference on Networks, ICN, 2020.

[26] **Paper X14:** Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Riccardo Mancini, Carlo Vitucci. *Strong Temporal Isolation Among Containers in OpenStack for NFV Services*. In IEEE Transactions on Cloud Computing, 2021.

[27] **Paper X15:** Gabriele Ara, Leonardo Lai, Tommaso Cucinotta, Luca Abeni, Carlo Vitucci. *A Framework for Comparative Evaluation of High-Performance Virtualized Networking Mechanisms*. In journal Communications in Computer and Information Science, 2021.

Patents in Telecommunication Domain

[28] **Patent 1:** Claudio Porfiri, Carlo Vitucci. *A transmitting Entity and Method Performed Thereby for Transmitting One or More Data Packets to a Receiving Entity*. Patent Number: WO2017063672A1, 2015.

[29] **Patent 2:** Claudio Porfiri, Carlo Vitucci. *Method and Network Node for Handling SCTP Packets*. Patent Number: EP3466015B1, 2016.

[30] **Patent 3:** Claudio Porfiri, Carlo Vitucci. *Methods and Arrangements for Determining Indication of Maximum Datagram Size Supported Without Fragmentation in an IP Network*. Patent Number: US20220353735A1, 2018.

[31] **Patent 4:** Claudio Porfiri, Carlo Vitucci. *Methods and Apparatus for Multihoming SCTP Communication Between a Network Entity and a Remote Host*. Patent Number: WO2021121550A1, 2021.

[32] **Patent 5:** Angelo Centonza, Gian Luca Masini, Claudio Porfiri, Carlo Vitucci. *Encapsulating an SCTP Packet in a UDP Packet*. Patent Number: WO2022203555A1, 2022

Acronyms

3GPP 3rd Generation Partnership Project

5G the fifth-generation of wireless communication technologies

6G the sixth-generation of wireless communication technologies

AER Advanced Error Reporting

AI Artificial Intelligence

ASIC Application Specific Integrated Circuit

BBU BaseBand Unit

BER Bit Error Rate

BIOS Basic Input/Output System

BIST Built-In Self Test

CBIT Continuous Built-In Test

CE Corrected Error

CPM Central Processor Module

CPRI Common Public Radio Interface

CPU Central Processing Unit

CRC Cyclic Redundancy Check

CSP Critical Systems Practice

CTMC Continuos Time Markov Chain

CTX ConTeXt switch

CU Centralized Unit

DCO Digital-Controlled Oscillators

DDRx Double Data Rate, generation x

DFT Design For Testability

DP Device Processor

DU Distributed Unit

ECC Error Correcting Code

ECS Error Correcting and Scrubbing

eCPRI enhanced Common Public Radio Interface

EDAC Error Detection and Correction

ETH ETHernet

FEC Forward Error Correction

FMEA Failure Mode and Effects Analysis

FPGA Field Programmable Gate Array

FM Fault Management

FT Fault Tree

FTA Fault Tree Analysis

FW FirmWare

GSM Global System for Mobile

HDD Hard Disk Drive

HW HardWare

I2C Inter-Integrated Circuit

I3C Improved Inter-Integrated Circuit

IBIT Initiated Built-In Test

ID IDentificator

IoT Internet of Things

ISP Internet Service Provider

JTAG Joint Test Action Group

LDPC Low-Density Parity-Check

LPDDR Low-Power-DDR

LTE Long Term Evolution

M-BIST Memory BIST

MB MegaByte

MC Memory Controller

MCA Machine Check Architecture

MCC Matthews Correlation Coefficient

MEMS Micro-Electro-Mechanical Systems

ML Machine Learning

MSI-x Message Signaled Interrupts Extended

MTBF Mean Time Between Failure

MTC Master Test Controller

MTTD Mean Time To Detect

MTTF Mean Time To Failure

MTTR Mean Time To Repair

NFV Network Function Virtualization

NVMe Non-Volatile Memory Express

OBS OBServability

OCXO Oven Controlled Crystal Oscillator

OS Operating System

PBIT Power-up Built-In Test

PCIe Peripheral Component Interconnect Express

PM Power Module

PPR Post-Package Repair

QoE Quality of Experience

QoS Quality of Service

RAID Redundant Array of Independent Disks

RAM Random Access Memory

RAN Radio Access Network

RAS Reliability, Availability, Serviceability

RAT Radio Access Technologies

RBD Reliability Block Diagram

RBER Raw Bit Error Rate

RBS Radio Base Station

RG Reliability Graph

RT Real-Time

RTOS Real-Time Operating System

RR Return Rate

RRH Remore Radio Heads

SEC SECurity

SECDED Single Error Correction / Double Error Detection

SDN Software-Defined Networking

SoC System on Chip

SoS System of Systems

SMS Short Message Service

SPN Stochastic Petri Net

SSD Solid State Disk

SSM Soft Systems Methodology

SW SoftWare

TMR Triple Modular Redundancy

TN Transport Network

TT Training Time

TTF Time To Failure

TTD Time To Detect

UART Universal Asynchronous Receiver/Transmitter

UCE UnCorrected Error

UCR UnCorrected, but Recoverable error

UE User Equipment

UMTS Universal Mobile Telecommunication System

USB Universal Serial Bus

Contents

1 Introduction **1**

2 Research Plan **7**
2.1 Research Goals 7
2.1.1 Research Challenges 8
2.2 Research Contributions 11

3 Related Works **13**

4 Preliminaries **17**
4.1 Concepts 17
4.1.1 Embedded Systems 17
4.1.2 Radio Access Network (RAN) 19
4.1.3 Fault Management 21
4.2 Terminology & Definitions 23
4.2.1 System State 23
4.2.2 HW Error Taxonomy 25
4.2.3 Resilience Parameters 26
4.2.4 Resiliency, Availability and Serviceability: RAS . . . 29

5 System Resilience **33**
5.1 Functional Levels of Resilience 34
5.2 Resilience Quantitative Conceptual Model 40

6 Dependability and Failure Probability 49
6.1 Dependability Evaluation 51
6.2 Hypergraph Theory Recall 53
6.3 A Novel Quantitative Model 54
6.4 Model Application Example 56
6.5 Method Verification in Different Domain 62
6.6 Method Verification for Different Functional Characterization 65

7 Fault Management Cost Analysis 69
7.1 FM Cost System Definition 69
7.2 System Classification 70
7.3 System Modelling 73
7.4 System Complexity 74
7.5 Game Theory FM Cost SoS Analysis 77
7.6 The RR - Fault Coverage Relation 79

8 Fault Management Model 81
8.1 FM Subfunctions Relationship 81
8.2 FM Deployment Model 88
8.2.1 A Framework Deployment Use Case: System Memory 93

9 Fault Prediction 99
9.1 Outlier Analysis Based on System Fingerprint 102
9.1.1 Data Acquisition 103
9.1.2 Data Prepocessing 104
9.1.3 Model Building 107

10 Fault Management Hardware Requirements 111
10.1 Hardware Reference 112
10.2 Functional Requirements 114
10.3 Board Requirements 118
10.3.1 Board Test Domain 118
10.3.2 Temperature and Temperature Sensor 119
10.3.3 Clock and Synchronization 120
10.3.4 Power Distribution 121

10.4 Component Requirements 124
10.4.1 Central Processor Module 124
10.4.2 Memory System 131
10.4.3 Storage System 135
10.4.4 Low-Processing Device 138
10.4.5 SoC Device 141
10.5 Embedded System Connectivity 146

11 Conclusions **151**
11.1 Summary of Research Aims and Achievements 151
11.2 Conceptual Contributions to Resilience and Fault Management 152
11.3 Quantitative Modeling of Failure Probability 153
11.4 Fault Management Cost and System-Level Implications . . . 153
11.5 Fault Prediction and Data-Driven Methods 154
11.6 Hardware-Centered Design Guidelines 154
11.7 Industrial and Scientific Impact 155
11.8 Limitations and Future Work 155
11.9 Concluding Remarks 155

Bibliography **157**

I

Chapter 1

Introduction

The evolution of mobile communications networks (see Figure 1.1) has led to a profound transformation in users' communication habits, which has given rise to the need for a more cost-effective business model for telecommunication operators.

In the 1990s, with the introduction of digital systems, users began to use their phones for more than just phone calls. SMS text messages were highly profitable for telephone operators, generating enormous value per connection. In the 2000s, smartphones revolutionized the concept of the telephone, transforming

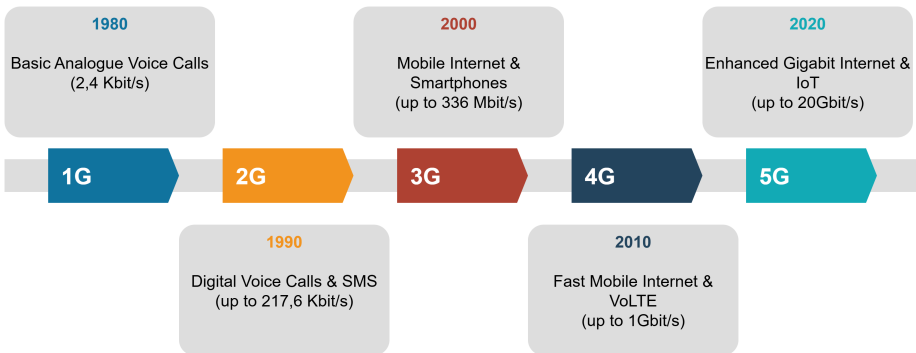


Figure 1.1. Timeline of the Mobile Communication Evolution.

it into a mobile device that allows for Internet access. However, this gradually shifted the value of communication from connectivity to serviceability. 4G amplified both the user experience and the shift in business from connectivity to serviceability. Infrastructure has enabled the provision of increasingly innovative services, which have contributed to the success of major ISPs, such as Google, Amazon, and Netflix, among others. This ongoing shift in the business model from network operators to service providers has created a tension between who controls the infrastructure and who captures the value: operators do not have sufficient returns to justify investments in new and more efficient infrastructure, while new services are increasingly demanding power and transmission speed. 5G is, in fact, the technology that aims to eliminate this tension. New technological solutions are enabling telecommunications operators to enter the large services market, regaining significant market share. In this new scenario, service continuity and the ability to develop new services become strategic elements for telecommunications operators, making the issue of resilience a crucial aspect in network infrastructure design. In a market where the value of connectivity alone is continually eroding, the ability to provide reliable and resilient services becomes crucial to operators' competitiveness. The Radio Access Network (RAN) domain plays a central role as the direct interface between the network and user devices. Any disruption or degradation is immediately perceived, with immediate impacts on the quality of experience, customer trust, and the operator's economic sustainability. The growing demand for heterogeneous services has transformed the RAN into an enabling platform for innovative business models, extending beyond end-user data traffic to include vertical services such as industrial IoT, automotive, and mission-critical applications. This transformation has led to an exponential increase in architectural complexity and the computational capacity required of the RAN: the introduction of distributed architectures, virtualized Distributed and Centralized Units (DU/CU), software-defined networking (SDN), and network function virtualization (NFV), along with the integration of multi-vendor resources orchestrated in complex environments, has multiplied points of vulnerability, requiring embedded solutions capable of dynamic adaptation, self-healing, and business continuity.

If the value of the network is increasingly measured in terms of services and

no longer (or not exclusively) in terms of connectivity, and technological solutions for access nodes increase the number of users they can manage, the cost of service disruptions grows exponentially. As highlighted by Iddrisu et al. [33], an interruption in highly complex environments can generate significant economic losses, reputational damage, and competitive risks, irreparably compromising the operator's market position. The resilience of embedded systems, therefore, no longer represents an optional feature but an enabling condition for the sustainability of next-generation networks. Recent literature confirms the growing importance of this topic. Martian [34] emphasizes the need for flexible RAN solutions that can respond to heterogeneous access scenarios, while Valenti [35] highlights the high management costs associated with distributed architectures, making the adoption of resilient strategies essential. De Simone [36] clearly indicates that the resilience of RAN embedded systems is now a top priority, given the centrality of this domain in ensuring service continuity and perceived reliability.

In light of this evidence, the resilience of embedded systems in the RAN domain emerges as a critical success factor, essential for maintaining competitiveness, economic sustainability, and user trust. RAN development will need to move towards solutions capable of integrating resilience mechanisms natively into embedded systems, combining redundancy, self-adaptation, and predictive capabilities, to support very low-latency applications and mission-critical services, in line with the needs of next-generation mobile networks. This work systematically examines the functional relationship between resilience and fault management in embedded systems. The analysis is mainly in the hardware domain, with a focus on applications in the RAN environment, characterized by stringent performance constraints, high sensitivity to disturbances, and limited availability of system resources. The goal is to highlight how specific hardware design choices, both at the board and subsystem levels, can directly impact overall resilience. In particular, the study analyzes the functional interdependencies that enable us to identify the critical points at which project investment can yield the maximum benefit in terms of operational robustness. The fault management function requires special attention. The work starts from the hypothesis that fault management is a pivotal element of resilience. The direct involvement of

hardware design is the efficiency enabler of the fault management function. As a result, resilience depends not only on fault management strategies but also on how each hardware subsystem and device contributes to the detection, isolation, and recovery of faults.

A systematic focus on achieving objectives through close, ongoing, and formalized collaboration with the industrial partner characterizes the research process developed within the industrial Ph.D. The presence of an industrial partner distinguishes it from purely academic research paths, as it introduces a two-way interaction that plays a crucial role in project development. First, the company is the primary source for identifying, defining, and thoroughly understanding use cases, providing a concrete application context aligned with market and production needs. Second, the industrial partner provides an environment for the empirical verification and experimental validation of results generated in the academic setting, thereby allowing for the assessment of the robustness, relevance, and transferability of the developed solutions.

Research activity is therefore structured as an iterative process of continuous osmosis between theoretical and practical dimensions: the concepts and models developed in academia are translated into concrete implementations on industrial products, and these implementations, in turn, constitute an essential testing ground for refinement of solutions. This circular mechanism of development and validation produces a dual effect: on the one hand, it progressively enhances the scientific quality and reliability of theoretical contributions; on the other, it increases the quality and competitive value of the products designed based on the research contributions.

Ultimately, the emerging framework is one in which the integration between academia and industry is not limited to a mere transfer of knowledge, but takes the form of a shared evolutionary process, capable of generating significant advances in both scientific production and technological and industrial innovation.

Chapter 2 illustrates the research objectives of the thesis, relating them to the original contribution provided by the work itself. Subsequently, Chapter 3 analyzes the main research trends in the scientific community in relation to the

defined objectives, highlighting the most significant results and discussing their connection to this study.

The same word can take on different meanings depending on the context in which it is used. To avoid potential terminological ambiguities, Chapter 4 introduces a series of definitions and taxonomies, ensuring an unambiguous understanding of the terminology used in this work.

As emphasized at the beginning of this chapter, system resilience represents a fundamental interpretative key to understanding the role, evolution, and optimal design of fault management. It is therefore necessary to clarify what it truly means for a system to be resilient, which functions are indispensable, how these functions are interrelated, and what the functional nature of resilience itself entails. Chapter 5 addresses these issues in detail, providing the theoretical foundations linking resilience to fault management and, in particular, its impact on hardware design.

An innovative contribution of this research lies in the definition of a new model for calculating the probability of failure. Chapter 6 describes in detail the developed mathematical model, its application starting from a block diagram, and its subsequent extension to telecommunication networks as a whole.

One of the main obstacles to developing optimal fault management solutions is the required investment. Since the benefits of increased investment in this area typically emerge later in the design process, Chapter 7 analyzes fault management as a complex system. A breakdown of the various direct and indirect cost components is proposed to identify the conditions under which it is worthwhile to invest and how these investments relate to product yield.

Chapter 8 introduces a new fault management model, particularly suited to embedded systems in radio access networks (RAN). The description shows the model hosted within a development framework that highlights the differential role of the various system levels (hardware, firmware, and software).

Chapter 9 explores the methodology to classify fault condition based on system fingerprint statistical deviation using machine learning approach.

One consequence of systematic cost analysis is the need to define precise requirements to guide the optimal development of fault management. To this end, Chapter 10 proposes a set of hardware requirements, intended as guidelines for both the component selection phase and the design of embedded systems in

RANs.

Finally, Chapter 11 presents the research's conclusions and outlines possible future development directions.

II

Chapter 2

Research Plan

In this chapter, we discuss the research goals of the thesis, the research contributions, and their relation.

2.1 Research Goals

In Chapter 1, it has been highlighted that system resilience represents a key factor for the economic sustainability of telecommunication network infrastructure development. It is the central concept driving our research. Resilience is a transversal property that involves heterogeneous technological layers (hardware, firmware, and software) as well as different categories of systems (data center infrastructures, cloud environments, telecommunication networks, etc.). A comprehensive analysis of all these aspects would exceed the scope of this study. Consequently, our research focuses on a more specific domain: RAN embedded systems. In particular, the research focuses its attention on the role that the hardware design of RAN nodes plays in determining the overall resilience of the system, with a specific emphasis on the fault management function as a critical component to ensure operational continuity.

In light of these considerations, we summarise the overall research goal as:

Develop an efficient resilience-oriented fault management model for RAN embedded systems.

2.1.1 Research Challenges

To achieve the overall goal, the research needs to address several key challenges.

RG$_1$ Characterization of System Resilience.

RG$_2$ Definition of a Fault Management Model characterizing Fault Management.

RG$_3$ Analysis of how the Hardware contributes to the proposed Fault Management Model.

RG$_4$ Development of a Novel Mathematical Model for Fault Probability.

RG$_5$ Validation of the Proposed Fault Management framework in a realistic RAN products use case set.

The first sub-goal involves analyzing the concept of resilience in the context of embedded systems for RAN, to identify its defining properties and evaluate the contribution of the FM function to the overall system resilience. The sub-goal includes both a conceptual framework for resilience characterization and a methodological approach to qualify the impact of FM mechanisms.

The second sub-goal aims at defining fault management as a first-class system function. The outcome will be the development of a structured model of FM that can serve as a reference framework for the design and implementation of embedded systems in RAN nodes. This model should enable the systematic integration of FM into the system development lifecycle.

The third sub-goal investigates the role of hardware design in the efficiency of fault management implementation in RAN embedded systems. The analysis considers two levels:

i the architectural level, in terms of system and node structures supporting fault detection, isolation, and recovery; and

ii the component level, by assessing the specific contributions of individual hardware elements to FM strategies.

The fourth sub-goal is to define a new mathematical model for simplifying the calculation of fault probability, thereby overcoming the practical limitations of classical fault tree analysis (FTA) when derived from hardware block diagrams. The analysis evaluates the broadening of its application for characterizing telecommunication networks.

The fifth sub-goal concerns the validation of the proposed FM model through practical application. It checks the model's correctness, effectiveness, and adaptability for specific case studies, as realistic RAN embedded system scenarios.

The dual nature of the research, which encompasses both academic and industrial aspects, influences the structure of our dissertation. It is therefore natural that the initial motivation for our research stemmed from a concrete industrial need: assessing the effectiveness of FM implementation as a key factor in enhancing system resilience. Within this context, the continuous interaction between the academic and industrial domains, combined with the need to meet theoretical and formal perspectives with the resolution of practical problems, has progressively guided the development of the research, addressing its objectives and methodologies. The literature identifies several methodological approaches suitable for conducting high-quality research in academic–industrial contexts: the research method for engineering design [37], for information systems [38], qualitative and quantitative case studies [39] with their respective strengths and limitations [40], and literature review methodologies tailored explicitly to computer science research [41]. In our study, the individual investigations carried out on selected use cases employed different methods, selecting the most appropriate for each instance. A methodology that we call the *sandwich approach* helps in selecting use cases. The method differs from both a top-down strategy, where the FM function is defined a priori and subsequently mapped onto the available hardware, and a bottom-up strategy, where one starts from the hardware to identify a compatible FM function. Instead, the adopted method finds the most effective solution through a constant and cyclic interaction between functional requirements and technology features. Such a process enables the consistent preservation of product quality in future designs by progressively integrating technological advancements into the functional

framework. Specifically, starting from the primary goal of enhancing system resilience, the selection of use cases followed two complementary prospectives: on the one hand, the functional dimension, oriented toward the identification of a framework for FM; on the other, the practical dimension, focused on analyzing how hardware technologies should be employed for the proper implementation of the framework, thereby ensuring its realistic applicability within the domain of interest. The following steps represent the adopted methodology, which is iterative by definition:

1. **Systemize**: consider the sub-functions of FM
2. **Assess**: identify a critical issue specific to the domain under investigation
3. **Innovate**: define a solution, whether systematic or structural
4. **Prove**: Introduce the solution in the industrial products, implement it, and demonstrate its benefits
5. **Contextualize**: evaluate the effects of the solution on system resilience and consider industrial feedback to the research study.

Participation in industrial development projects allows for active collaboration with industrial partners and creates opportunities for integrating research outcomes into real-world application contexts. In fact, it is essential that scientific results not only contribute to academic knowledge but are also helpful and strategically relevant to the industry. The realization of this common point of interest requires the identification and harmonization of both domains' expectations, which, while inherently divergent, may nonetheless converge on common ground. From the industrial perspective, we can summarize the primary objectives as follows:

- **Technological Leadership**: strengthening product competitiveness through the implementation of advanced fault management solutions, aimed at enhancing operational resilience.
- **Customer Satisfaction**: delivering solutions characterized by high performance and reliability, thereby improving user experience and the overall perceived value of the product.

- **Cost Optimization**: reducing expenses associated with maintenance, repair, and reducing the RR, by means of predictive mechanisms and more efficient fault management policies.
- **Supply Chain Management**: providing vendors with precise, comprehensive, and standardized technical specifications and fault management requirements.
- **Sustainability and Long-Term Vision**: developing fault management policies that are not only efficient in the present but also capable of guiding the evolution of next-generation systems.

On the other hand, the academic domain places greater emphasis on requirements such as generalizability, reproducibility, and methodological validation of results. The intersection between these two perspectives lies in the development and testing of proof-of-concept implementations based on selected use cases that serve as mediating instruments. They demonstrate the transferability of research findings to practical scenarios, and they provide the industry with tangible evidence of the effectiveness of the proposed solutions. In this way, the proof of concept should not be seen as a bare intermediate outcome but rather as a checkpoint of mutual validation, capable of aligning scientific rigor with industrial relevance.

2.2 Research Contributions

The development of efficient solutions in the field of FM has required continuous engagement with the evolution of hardware technologies and with the growing interest in innovative approaches, such as the adoption of AI/ML techniques. It was indispensable to adopt a holistic perspective: We did not limit the focus solely to ensuring system resilience, but we concluded that we also needed to problematize and define the very concept of resilience rigorously, as well as to analyze the relationships between resilience and the various functions that, either directly or indirectly, contribute to its enhancement. This approach has endowed the analysis with greater objectivity, transforming the FM into a rigorous framework that can increase system quality and the value of the service

delivered to the end user, rather than a theoretical exercise pursued for its own sake.

The following is the list of the research's contributions.

C_1	A Novel Fault Taxonomy.	Scientific Contribution
C_2	Fault Management as Resilience System Function Analysis.	Scientific Contribution
C_3	Fault Management Cost, Analysis, and Fault Cost Mathematical Model.	Both Scientific & Industrial Contribution
C_4	A Revised Fault Management Architecture.	Industrial Contribution
C_5	A Novel Fault Management Model.	Scientific Contribution
C_6	Fault Detection and Prediction Framework Using Machine Learning.	Both Scientific & Industrial Contribution
C_7	Fault Management HW Design Requirements.	Industrial Contribution
C_8	A Novel Methodology for Fault Analysis (and network characterization).	Both Scientific & Industrial Contribution

Table 2.1 maps contributions to research sub-goals and planned inclusion in specific thesis chapters. Papers are also available in the table.

Contribution	**Paper**	**Subgoal**	**Chapter**
C_1	[1]	RG_2	4
C_2	[7, 10]	RG_1	5
C_3	[10]	RG_1	7
C_4	[1]	RG_2	8
C_5	[2, 4]	RG_2	8
C_6	[9, 5, 8]	RG_5	7
C_7	[3]	RG_3	10
C_8	[6, 11]	RG_4	6

Table 2.1. Contributions Brought by the Proposed Thesis, Mapped to the Previously Introduced Research Sub Goals and Thesis Chapters.

III

Chapter 3

Related Works

The analysis of studies and research conducted in the field of FM constitutes an indispensable starting point for any further investigation of this system's function. We structured the literature review into several significant areas of investigation.

Definition and Terminology

In addition to the well-known work of Parhami [42] on fault taxonomy, more recent contributions deserve attention, particularly those that have adapted this taxonomy to modern telecommunication architectures (e.g., [43]). A common attention on system resilience emerges in the analysis of literature (e.g., [44]). Nevertheless, the majority of studies consider the network node as an atomic entity (e.g., [45]). As a consequence, although there is broad consensus that the Reliability, Availability, and Serviceability (RAS) mindset should serve as the new reference standard for error management (e.g., [46]), the definitions and terminologies adopted remain primarily at the network level, introducing concepts such as Quality of Service (QoS) [47] and Quality of Experience (QoE) [48]. Our research, by contrast, focuses on the node, examining the contribution of individual components to system resilience. This gap has motivated the need to revisit taxonomy both dimensionally (by extending error classification granularity down to the component level) and functionally (by including information

about faults that are either required for or consequent to recovery actions) [1].

Survey

By definition, surveys examine the state of the art of a discipline, offering a critical perspective that enables simultaneous consideration of the various motivations and viewpoints underpinning the proposed solutions. Our analysis concentrated on domains most closely related to embedded RAN systems, the primary domain of our research. Specifically, we examined fault-tolerance techniques for embedded systems (e.g., [49]), fault management in wireless networks (e.g., [50]), and the impact of FM in relation to technological innovations such as AI/ML (e.g., [51]). Existing surveys suffer from the same limitation identified in the analysis of taxonomies: the atomic perspective of the node, which inevitably leads to a network-level approach. There are significant consequences. Although the literature acknowledges the crucial importance of recovery actions, many studies restrict their scope to fault localization [52] or performance monitoring [53]. As a result, FM models remain incomplete, since they do not account for a comprehensive node recovery strategy. Alternative traffic rerouting is often regarded as sufficient; however, the effectiveness of this solution diminishes as the number of users served by the same node increases, a condition that becomes increasingly likely with the evolution of networks.

Hardware Design

The literature consistently recognizes fault tolerance as a key element in error management, to be incorporated at the design stage in order to ensure recovery from fault conditions [54]. The keyword that encapsulates the challenges and future directions of hardware design is RAS: Reliability, Availability, and Serviceability [55]. It is therefore essential to design hardware devices to be aware of FM policies [56]. Fault-tolerance techniques at the network architecture level (e.g., [57]) are of great significance; however, as long as FM remains confined to the network level, attention is focused primarily on traffic path supervision (e.g., [58]) or the adoption of dynamic routing policies (e.g., [59]). Many studies

that analyze the impact of individual hardware components on FM focus particularly on memory devices. The technological evolution of such devices (from DDR2 to DDR5 and beyond) highlights a trend of continuous improvement in fault management support. Recurring themes in the literature on FM for memory devices include the correlation between component aging and error probability(e.g., [60]), the impact of system memory errors on node behavior (e.g., [61]), the development of increasingly sophisticated ECC mechanisms (e.g., [62, 63]), and the introduction of isolation techniques aimed at preventing the propagation of data corruption in memory [64]. More broadly, research tends to concentrate on three main areas: automatic test algorithms (e.g., [65, 66]), mechanisms for verifying fault supervision policies (e.g., [67]), and redundancy analysis (e.g., [68]).

Wireless

Given the limitations of FM solutions that treat the node as an atomic entity, we can recognize that the network-level FM currently represents the most consolidated and mature area of research. Numerous studies investigated the automatic fault detection [69], with particular attention to the evolution of telecommunication networks [70]. Research on wireless networks converges on the crucial importance of FM for service quality, a relationship that becomes even more relevant in the context of 5G and its applications [71, 72, 73]. Equally significant are studies that consider interference management as an integrative tool within FM, particularly from the perspective of reliability in 5G networks [74, 75].

Fault Prediction

Special attention has been given to studies exploring the application of AI/ML in fault prediction, which is widely recognized as a key enabling technology in this area. Several approaches have been proposed, with one of the most common focusing on the optimization of AI/ML algorithms (e.g., [76]). There is also a growing trend toward employing AI/ML as a preventive tool (e.g., [77, 78]). Particularly valuable are studies evaluating the cost–benefit ratio of AI/ML

algorithms in relation to resource consumption [79]. However, in general terms, fault prediction research also appears to be oriented more toward network management than toward node hardware (e.g., [80, 81]).

IV

Chapter 4

Preliminaries

This chapter defines and clarifies the fundamental concepts that form the theoretical and terminological basis of the research. The objective is twofold: first, to ensure semantic precision and consistency in the use of terms, acronyms, and technical terms; second, to prevent potential ambiguities arising from the attribution of different meanings in disciplinary fields or application contexts distant from the focus of this research. The ultimate goal is to provide a shared conceptual framework that facilitates a clear understanding and accurate interpretation of subsequent chapters.

As is often the case in such contexts, it is appropriate to start from the beginning. In our case, this means first clarifying the meaning of the title of this research: *...Fault Management in embedded systems for Mobile Networks...*.

4.1 Concepts

4.1.1 Embedded Systems

Dori et al. [45] define a system as ... *an integrated set of elements, subsystems, and assemblies that accomplish a defined objective. These include products (hardware, software, firmware), processes, people, information, techniques, facilities, services, and other support elements*. This definition highlights the intrinsic complexity of a system, resulting from the close interface between

heterogeneous objects that contribute to the realization of the service or product in question. Considering this definition from a complementary perspective, it follows that the availability of a service, regardless of its complexity, always depends on the correct and well-defined interaction among multiple actors.

However, what characteristics must a system possess to be defined as embedded? One possible definition holds that a system is embedded when it constitutes an integral part of a product with which the end user does not interact directly [82]. Behind this relatively simple formulation, in our view, embedded systems can be characterized by a set of distinctive properties; these properties need not all be present in every instance, but collectively they provide a useful descriptive framework for analysis.

- *Dedicated to a specific function*. An embedded system is designed to perform a particular task, in contrast to general-purpose computers, which are designed to support a variety of different applications. This specificity may require the use of dedicated hardware, such as ASICs, FPGAs, or SoCs, each of which can be considered a subsystem [83, 84].

- *Limited resources*. Embedded systems operate within stringent constraints on energy, space, or computational resources (e.g., processing power, memory, or connectivity). A limitation that requires a highly efficient use of resources to ensure the required performance [85, 86].

- *Reliability and robustness*. An embedded system must guarantee the delivery of its function without interruptions for extended periods and with minimal human intervention [87, 88]. The concept of reliability and robustness is a cornerstone of this thesis and will be the subject of a specific chapter, which will analyze the resulting design implications.

- *Part of larger systems*. Typically, an embedded system represents a subsystem within a more complex product [89].

- *Interface with the environment*. The ability to perceive the external environment through sensors and modify it through actuators is a crucial feature of most embedded systems. This capability has become increasingly important with the widespread adoption of IoT architectures [90, 91, 92].

- *Real-time (RT) or soft real-time (Soft-RT) constraints*. Embedded systems are often subject to stringent time constraints on the execution of their functions. Failure to meet these constraints can compromise service availability. RT systems are defined when timing constraints must be rigorously respected [93], whereas the definition of Soft-RT applies when the system can tolerate a limited number of timing violations, both in frequency and duration [94, 95, 96].

One aspect deserves a deeper look: the importance of the specific function performed. In the list above, such a characterization is in correlation with the need to use highly specialized hardware. Therefore, defining the study domain is crucial, as it determines the design essence of the system under consideration and substantially influences the research conclusions. For our use case the reference domain is the Radio Access Network (RAN).

4.1.2 Radio Access Network (RAN)

Telecommunications specifications [97, 98] define the RAN concept. Although some details may be subject to change from one technology generation to the next, its function remains constant, allowing for a clear identification of its role.

A mobile network is a communication system that allows wireless devices, such as smartphones and tablets (User Equipment, UE), to connect and communicate over large geographic areas without physical cables. It is composed of

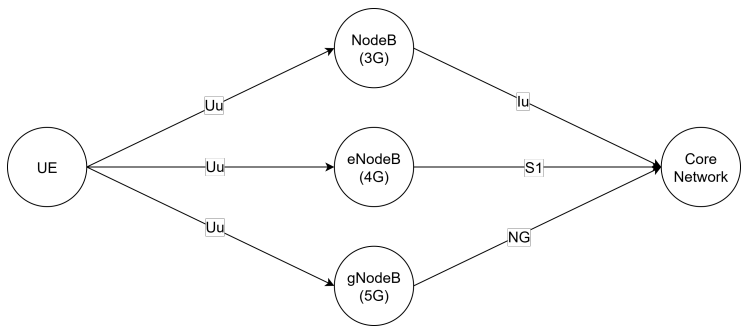


Figure 4.1. The RAN Interface.

different interconnected components that manage the wireless connection from the user to the broader telecommunications infrastructure. Although some details may be subject to change from one technology generation to the next, the function of these components remains constant, allowing for a clear identification of their role.

The RAN represents the functional component of the mobile network that connects the User Equipment (UE) to the Core Network through standardized radio interfaces, thus ensuring access to telecommunications services by managing and optimizing radio resources. It consists of a set of radio nodes (e.g., base stations, gNodeBs, eNodeBs, NodeBs) and a connection network (backhaul or fronthaul) that implements the radio access layer protocols defined by the 3GPP standards. Its main functions include managing transmission and reception on the radio interface (Uu), allocating radio resources, supporting mobility, ensuring quality of service, and interacting with the Core Network via standard interfaces, e.g., S1, NG (see Figure 4.1). In short, the RAN translates service and connection requirements into radio abstractions, coordinating multi-user communications and heterogeneous mobility scenarios. Within the RAN domain, embedded systems are typically characterized by stringent latency constraints, often classified as soft real-time requirements, and by limited computational and memory resources. These requirements and constraints drive the design of hardware and platforms that are highly specialized and function-oriented, rather than relying on general-purpose solutions. As a consequence, RAN embedded systems commonly integrate ASICs, FPGAs, or heterogeneous SoCs, in order to provide deterministic performance, energy efficiency, and scalability. In addition, reliability and fault-tolerance are essential design aspects, given the critical nature of telecommunication services. Finally, security considerations impose strict restrictions on data accessibility, availability, confidentiality, and integrity, shaping both hardware and software architectures in the RAN embedded ecosystem. The reliability of the system begins to emerge as critical after a better characterization of the products. As already mentioned above, the analysis of the reliability deserves a dedicated chapter. For now, having established the definition of an embedded symbol within the RAN domain, the chapter can continue with the fault management concept examination.

4.1.3 Fault Management

Since the 90s, designers and engineers have recognized the necessity of designing systems with dedicated fault management (FM) support. However, attempts to integrate correctness knowledge (the characteristic of recognizing that the system is not working as intended) have suffered performance drawbacks, like including increased resource consumption, higher design complexity, and reduced maintainability [99, 70]. More recently, research in Artificial Intelligence and Machine Learning (AI/ML) has renewed interest in FM design, particularly in approaches based on fault prediction [100, 101, 102, 103, 104, 105, 106, 107]. The challenge for an efficient implementation of FM is therefore the ability to support new AI/ML-based fault prediction techniques while minimizing or eliminating performance impacts. A challenge that becomes a real system requirement capable of influencing the choice of hardware and software components. Research commonly recognizes that new hardware and software requirements are necessary for the development of highly reliable systems [108, 109, 110]. The exponential growth of connectivity among heterogeneous devices [15], coupled with the increasing complexity of interconnected systems, has intensified the challenge of efficient FM.

In this context, FM has become synonymous with system reliability and performance optimization [111, 52]. However, it would be more correct to say that the FM is a crucial component of the system's resilience. Formally,

Fault Management is a system function that aims to detect, localize, isolate, and recover from fault conditions [112].

Figure 4.2 illustrates the interactions among FM components. The Figure also shows that FM is more complex than a basic error-detection mechanism. The ultimate purpose of the FM is to ensure that the system provides its service within a defined performance range while maintaining continuous service availability. Even better, FM has direct and indirect impacts on the Quality of Experience (QoE) parameter, i.e., the measure of customer satisfaction, precisely because of its nature, which involves maintaining a service available for a long time and with precise performance levels. The cost of the FM is a complex analysis and deserves a dedicated chapter. For now, we recognize that a passive

fault supervision system immediately appears inefficient for achieving such a challenging goal. The FM is a function system built on what Nouioua et al. [51] describe as the four pillars of recovery:

- **Fault detection**. The automatic collection of alarms generated by hardware devices or system software components. Detection may range from simple checks for potential faults to complex monitoring mechanisms that gather extensive data and metrics. It may also employ AI/ML techniques to predict faults in advance [113].

- **Fault localization**. The processing and enrichment of alarm data and metrics to increase their diagnostic value. The objective of fault localization is to identify the root cause of a failure. Time representation and temporal relationships among events play a critical role in this process [52].

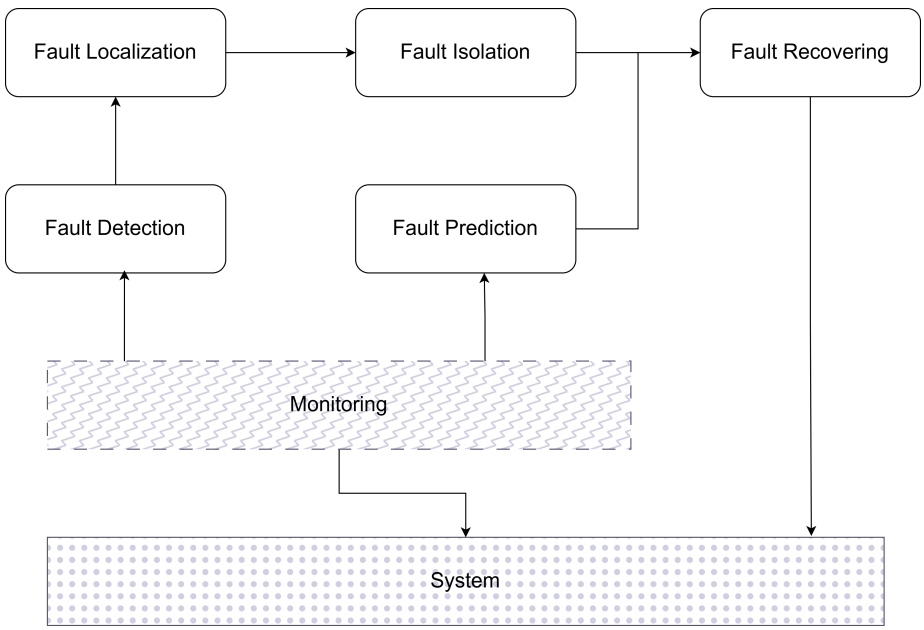


Figure 4.2. The Fault Management Main Component Diagram.

- **Fault isolation**. The reduction of conditions that facilitate fault propagation. It involves resolving dependencies among events, establishing priority-based alarm classifications, and marking faulty states in active alarms. Hardware and Software often support isolation mechanisms and features, such as temporal and memory isolation or poisoning algorithms that mark corrupted memory regions [64].

- **Fault recovery**. The application of corrective or compensatory actions to eliminate the root cause of the fault or to restore the system to an operational state. Recovery strategies may involve resource reallocation, the isolation and removal of faulty components, or the deployment of specific repair procedures [114].

At this point, we possess all the necessary knowledge to understand what fault management entails in RAN embedded systems. It remains to define and clarify some key terms that we will use throughout the following pages.

4.2 Terminology & Definitions

4.2.1 System State

A system in a working condition is an intuitive condition. Instead, saying that a system is in a fault condition is a phrase that lends itself to different interpretations, especially if one's professional background is different. In our research, we use Parhami's [42] definition of a system's different working states, listed as follows;

- **Ideal**: The state where the system is performing according to its specification.

- **Defective**: The state where an incorrect behavior is detected.

- **Faulty**: The state where a behavior outside specification is manifested in the system, causing an error.

- **Erroneous**: The state where the system may start working with behavioral deviation from the ideal state if it cannot execute the needed recovery actions.

- **Malfunctioning**: The state where the system is working with behavioral deviation from the ideal state.

- **Degraded**: The state of a system able to support a malfunction behavior thanks to tolerance provisions.

- **Failed**: The state where a system is not able to meet the specification.

It is worth mentioning how the description of the transition model state emphasizes the system's ability to avoid a failed condition using both recovery actions and tolerance provisions as shown in Figure 4.3. Defective is the state where hardware or software may deviate from the ideal system state. Wear and aging of the hardware may cause a defect and may, in turn, bring the system into a faulty state. A faulty condition may or may not cause an erroneous state if no recovery action is possible. An erroneous state may or may not cause the affected system to malfunction, depending on the system's error tolerance. A malfunction does not necessarily mean a perceivable service-level drawback, but

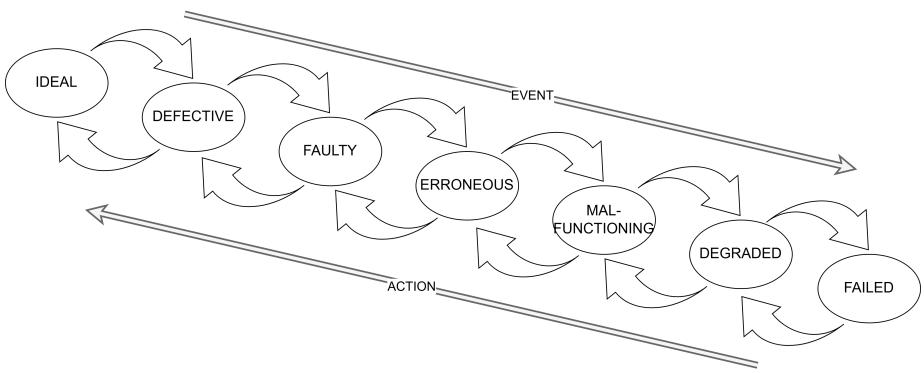


Figure 4.3. System State Transaction Diagram.

if that happens, the degradation of the service has the consequence of a failed system condition.

4.2.2 HW Error Taxonomy

Together with other objectives, our research aims to analyze the contribution of hardware design to fault management. In this context, it is helpful to define a taxonomy of errors within the device domain to interpret the meaning of notifications generated by hardware accurately.

Traditionally, hardware error reporting has been defined in three primary dimensions [50, 115, 43]:

- **Component-Based**: identifies the location of the error (Node, Device, Link, etc.);
- **Time-Based**: describes the temporal duration of the error (Permanent, Transient, Intermittent, etc.);
- **Priority-Based**: evaluates the impact of the error on the system (Fatal, Critical, Local Impact, etc.).

However, from a reliability perspective, error reporting should also provide information relevant to the recovery actions that can be taken. The reason for having recovery information is that the hardware itself may have already initiated a corrective mechanism to restore the erroneous condition. Consequently, it is crucial that notifications also convey the recovery status (a **Reliability-Based** dimension), distinguishing among the following cases:

- **Corrected Error (CE)**: It indicates that the self-recovery mechanism has successfully eliminated the error condition. The system remains fully operational and requires no further intervention. This information is also valuable for statistical characterization of the device, as it may reveal, for instance, whether a component is operating under adverse environmental conditions or is progressively degrading due to aging.
- **Correctable Error (UCR)**: it indicates that the self-recovery mechanism is not able to correct the error, which nevertheless remains isolated and

does not compromise system performance or functionality. In this system "defect" state, the notification receiver could initiate light corrective actions to prevent the condition from escalating into a "faulty" state.

- **Uncorrectable Error (UCE)**: it indicates that the self-recovery mechanism is unable to correct the error, but it recognizes that the error has a tangible impact on system performance and functionality. In the UCE case, the notification receiver must adopt additional recovery strategies to restore proper operation, considering that the system function is already compromised.

The reliability-based taxonomy helps to understand that hardware reports errors exclusively within its own domain of competence. For example, a UCE should be interpreted as "Uncorrectable" solely from the hardware's standpoint: it signifies that the device cannot correct or isolate the error, but the notification itself provides no direct indication of the actual severity of the issue at the system level. To achieve a more comprehensive characterization, a **Behavior-Based** taxonomy is often employed, which classifies errors according to their persistence with respect to recovery attempts:

- An error is defined as ***Soft*** if a recovery action, whether or not performed by the hardware, exists that can eliminate the error condition. In this sense, a "Soft" error may correspond to a UCE from the hardware perspective, yet still be recoverable by other system components.
- An error is defined as ***Hard*** if it persists despite multiple recovery actions. In other words, it is an error classified as UCE by the hardware and one that, at the system level, may be regarded as "Permanent."

4.2.3 Resilience Parameters

In our research, the concept of resilience has progressively assumed a key role in system characterization, ultimately establishing itself as a primary criterion for evaluating a system. This relevance justifies the decision to have a dedicated chapter to an in-depth analysis of system resilience. In this subsection, it is therefore helpful to introduce the main parameters that define and assess the

level of resilience of a system. Their introduction not only clarifies potential misunderstandings in the reading of subsequent chapters but also highlights the intrinsic relationship between resilience and fault management, a connection already implied in the conceptual meaning of the term 'resilience'.

Different application domains, such as avionics [116], automotive [117], nuclear [118], or electronics [119], tend to adopt their own definitions of resilience parameters, selecting those that best address the specific challenges inherent to the domain under consideration. In this work, we adopt the version that is most suitable for emphasizing the relationship between resilience and fault management.

Definition 1. A *failure* is the inability of a device to meet the specified functionality [1].

Definition 2. We define λ as the *failure rate* of the system. During the lifetime phase, it tends to remain constant [120].

Definition 3. We define *reliability*, $R(t)$ (4.1) (where t is the time), as the probability of a system to operate correctly, without a failure condition, during a time interval $[0, t]$, assuming that the system is performing correctly at the time 0 [54].

$$R(t) = e^{(-\lambda t)} \tag{4.1}$$

Definition 4. MTTF, *Mean Time to Failure* (4.2), is the expected time a system works properly before the first failure is observed [54].

MTTF is strictly connected to reliability by definition.

$$MTTF = 1/\lambda \tag{4.2}$$

Definition 5. MTBF, *Mean Time Between Failure* (4.3) is the average time between two consecutive failures of the system [121].

$$MTBF = \frac{\#operationhours}{\#failure_{sametime}} \tag{4.3}$$

Definition 6. RR, *Return Rate* (4.4) is the percentage of installed products that are returned to the hardware repair center because of a suspected failure condition.

RR is a fault avoidance requirement type because it is the most obvious way to reduce costs: high-quality hardware components are achieved through careful device selection in the design test and device screening in the production test. The return rate is also a function of time [121].

$$RR_{annual} = \frac{1}{MTBF_{annual}} = \frac{24 * 365,25}{MTBF_{hourly}}$$

$$RR_{annual} = \frac{8766}{MTBF_{hourly}} \tag{4.4}$$

Definition 7. MTTR, *Mean Time to Repair* (4.5) is the average time required to isolate, repair, and test a fault condition for the system [120].

From a mathematical point of view, the MTTR value is [120]:

$$MTTR = \frac{1}{\mu} \tag{4.5}$$

where μ is the repair rate, which is the expected number of repairs per unit of time.

MTTF, MTBF, and MTTR are correlated via the steady-state availability factor.

$$A(\infty) = \frac{MTTF}{MTTF + MTTR}$$

where $A(\infty)$ is the steady-state availability and specifies the downtime per year.

$$MTBF = \frac{MTTF}{A(\infty)}$$

Definition 8. In this thesis, we also use MTTD, *Mean Time to Detect*, as the average time required to detect the occurrence of a faulty condition and start the recovery action needed by that specific fault.

It is not a *traditional resilience parameter*, but it is helpful to characterize the hardware design's efficiency in term of fault management. Formally, let us consider a set of N incidents indexed by $i = 1, 2, \ldots, N$. For each incident i, we denote by $t_i^{(\mathrm{occ})}$ the time of occurrence of the incident, and by $t_i^{(\mathrm{det})}$ the time at which the incident is detected. The *time to detect* associated with incident i is then defined as [122]

$$TTD_i = t_i^{(\mathrm{det})} - t_i^{(\mathrm{occ})}. \tag{4.6}$$

The *Mean Time to Detect* is the arithmetic mean of all TTD_i values over the set of incidents:

$$\mathrm{MTTD} = \frac{1}{N} \sum_{i=1}^{N} \left(t_i^{(\mathrm{det})} - t_i^{(\mathrm{occ})} \right) = \frac{1}{N} \sum_{i=1}^{N} TTD_i. \tag{4.7}$$

It is important to emphasize that MTTD exclusively measures the detection latency, and therefore, it does not account for the subsequent remediation or recovery phase. Consequently, MTTD must be distinguished from the MTTR. In practice, the combination of MTTD and MTTR measures the total expected time from the occurrence of an incident to its complete resolution.

4.2.4 Resiliency, Availability and Serviceability: RAS

RAS (Reliability, Availability, and Serviceability) is a set of system design features, functions, and mechanisms intended to improve a system's reliability (ability to operate without failure), availability (ability to remain operational and accessible), and serviceability (ease and speed of maintenance and repair) [123].

Reliability is the ability of a system to perform a required function under stated conditions for a stated period. In other words, it is a measure of how a functional unit is not prone to error and generates the correct outputs [55].

Availability is the measure of a system's capability or readiness to provide service. It can also be defined as the measure to which a system or subsystem is able to provide a specified service to customers [55].

Serviceability is defined as the characteristics involved in performing service or maintenance on a system, including diagnostic capability and manual repair capabilities [55].

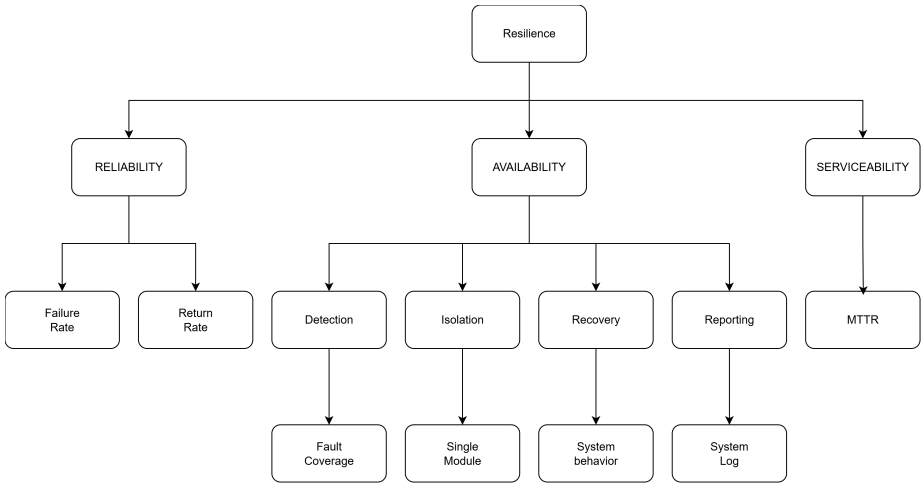


Figure 4.4. RAS Requirements Mapping into the FM Functions.

RAS features are essential considerations in hardware design, as they influence architectural decisions from the earliest stages of a project. The effective implementation of RAS mechanisms often serves both as an enabler and as a distinguishing factor in achieving system-level resilience. Understanding RAS requirements entails analyzing how individual components contribute to the overall resilient behavior of the system. These requirements can be categorized into three principal dimensions:

- Self-Describing: The capability of a component to convey its identity and operational state;
- Fault Information: The ability to accurately detect, isolate, and report error conditions;
- Contextualization and Recovery: The capacity to contextualize faults, identify root causes, and execute appropriate recovery strategies.

Integrating these RAS dimensions during the design phase ensures that resilience emerges as an intrinsic property of the system architecture, rather than

as an afterthought. Figure 4.4 [108] shows how to concretely map the macro areas of RAS requirements in the fault management functions, which concretely allows us to have a better vision of what it actually means to be RAS compliant.

Chapter 8.1 describes the FM functions with higher details degree

V

Chapter 5

System Resilience

As discussed in Chapter 1, there exists a strong relationship between the evolution of embedded systems for RAN and the concept of resilience. It is intuitive to observe that, as the complexity and size of a node increase, in terms of the number of end users served, more stringent requirements for resilience emerge [124].

This observation, in fact, reflects a substantial shift in the approach to embedded system design [125]. It has been widely acknowledged that, in complex systems, designing for a zero-error condition throughout the product's entire life is unattainable [126].

The logical consequence is therefore the need to develop architectures that are not only capable of preventing errors but also of responding effectively when they occur.

The above consideration constitutes a genuine paradigm shift toward an uncertainty-aware resilience paradigm [127]: the transition is from a design model aimed exclusively at error avoidance to one that emphasizes both minimizing error occurrence and optimizing the system's response to failure [128].

The growing attention to the recovering phase has progressively consolidated the notion of resilience as a fundamental property of a system. The system resilience concept is now widely recognized in the literature (e.g. [129, 130, 131, 132, 133]), and can be formally defined as follows.

The resilience of a system is its capacity to respond to an internal or external disturbance that compromises its proper functioning, through actions aimed at restoring the operational level that the system possessed prior to the disturbance.

5.1 Functional Levels of Resilience

We now return to the definition of Fault Management (FM) as introduced in Chapter 4.1.3. Its comparison with the definition of system resilience emerges without ambiguity. The resilience of a system is its ability to react to an internal or external disruptive event to return to a *state of equilibrium* in the shortest possible time. In that case, FM represents the system function to achieve this goal. It is the function responsible for implementing recovery actions after faults; it is therefore a major contributor to system resilience, although not the only one.

To understand which functions contribute to system resilience, we can begin with a logical consideration: an alteration in system behavior is always the result of an internal or external event compromising the correct operation of one or more system components. In order to recover an acceptable functional level as rapidly as possible (a typical action of resilience that directly influences its implementation effectiveness [134]), it is standard to classify the possible actions into two main groups [135]:

(i) **Component recovery mechanisms**. For example, runtime recovery actions characteristic of fault management, as well as adaptation mechanisms tailored to the environmental context, and machine learning techniques to enhance the system's response.

(ii) **Global resilience strategies**, meaning the adoption of resilience-oriented strategies as an integral part of product design, in order to implement more complex and structural recovery mechanisms. For example, system

performance adaptation involves the dynamic redistribution of available resources and the deployment of redundancy mechanisms.

For a better and complete picture, it is important to add a third group:

(iii) **Protection against attacks, intrusions, or alterations** that, through malicious behavior, attempt to disrupt the system and compromise its *equilibrium state*. In other words, security management [136].

Based on these functional macro-groups, our analysis interprets resilience as the outcome of interactions across multiple functional domains. Comprehending resilience thus necessitates an explicitly functional-compositional approach.

L1 – Structural and Design Level

In this level, the functions contribute to the system design. They are explicitly considered to be the enablers of resilience. For that reason, L1 level is the *enabler level*. Design for Testability (DFT), Redundancy Management, and Security Architecture belong to this level.

L2 – Operational and Functional Level

It is the level of the functions that actively operate at runtime to detect, isolate, mitigate, and contain faults. For that reason, the L2 level is the *executive level*. FM, and Health Monitoring belong to this level.

L3 – Adaptive and Reactive Level

It is the level of the functions that modify the system's configuration or behavior to react and respond to a faulty condition. For that reason, the L3 level is the *dynamic executive level*. Reconfiguration Management and Graceful Degradation belong to this level.

L4 – Cognitive and Decision-Making Level

It is the level of the functions that operate at a strategic level, guiding and optimizing the system's response in the presence of faults. For that reason, the

L4 level is the *supervisor level*. Decision and Control System, and Learning & Self-Improvement belong to this level.

Figure 5.1 shows the hierarchical representation of resilience functional levels. Functions at **L1 and L2** provide inputs to the higher **L3 layer**, which in turn interacts with the **supervisor L4 level**.

Contextualization of Functional Contributions to Resilience

The list below provides a structured summary of functional contributions to system resilience, organized across the four hierarchical levels of abstraction (L1–L4) mentioned above. The list delineates for each level the specific functions, their definitions, intended purposes, and representative examples, creating a contextualization of how resilience-oriented mechanisms spread across different layers of system design and operation. The hierarchical framing highlights the progressive transition from structural provisions (L1) to advanced autonomous capabilities (L4), emphasizing the multi-dimensional nature of

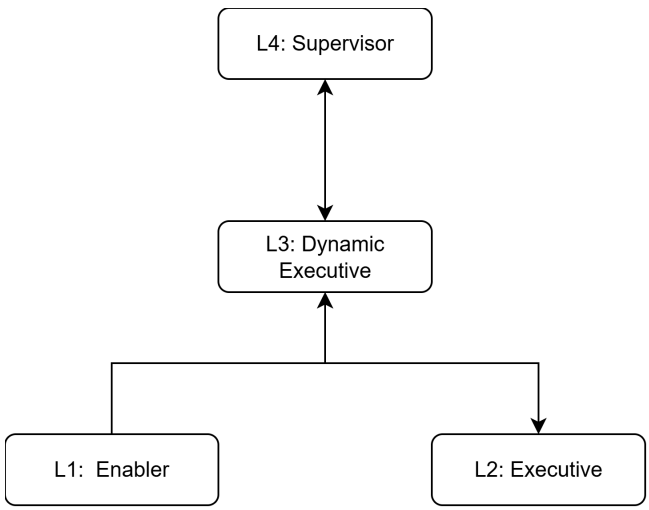


Figure 5.1. Hierarchical Representation of Resilience Functional Levels.

resilience engineering in complex embedded systems.

L1 Level

Function DFT

- **Definition**: Design techniques that increase internal observability and controllability
- **Purpose**: Enable accurate testing and diagnosis of faults
- **Example**: Scan chains, Built-In Self Test (BIST), boundary scan, Hardware device self test

Function Redundancy Management

- **Definition**: Inclusion of redundant resources (HW) for fault tolerance
- **Purpose**: Ensure operational continuity in the presence of faults
- **Example**: Dual-modular redundancy, spare resources, ECC, and syndrome

Function Security Architecture

- **Definition**: Design of structural security mechanisms
- **Purpose**: Prevent threat-induced faults
- **Example**: Hardware encryption, trusted execution environments

L2 Level

Function FM

- **Definition**: Automatic detection, isolation, and correction of faults
- **Purpose**: Contain and eliminate the impact of faults
- **Example**: Fault tree analysis, fault isolation, and recovery

Function Health Monitoring

- **Definition**: Monitoring and prediction of system health status
- **Purpose**: Prevention and predictive maintenance

- **Example**: Wear sensors, system logs, remaining useful life models

L3 Level

Function Graceful Degradation

- **Definition**: Controlled degraded operation after a failure
- **Purpose**: Maintain partial operability
- **Example**: Function reduction in safe mode

Function Reconfiguration Management

- **Definition**: Dynamic reconfiguration to adapt to faults or events
- **Purpose**: Restore functionality by reallocating resources
- **Example**: FPGA runtime reconfiguration, software re-routing

Function Contextual Adaptation

- **Definition**: Adaptation to variable operational and environmental conditions
- **Purpose**: Maintain performance and reliability in dynamic environments
- **Example**: Parameter adjustment based on network load or climate

L4 Level

Function Decision and Control System

- **Definition**: System fault handling decision maker
- **Purpose**: Supervisor control of system operational state
- **Example**: Alarm management, error suppression, recovery coordination

Function Learning & Self-Improvement

- **Definition**: Fault-driven learning mechanisms for improved responses
- **Purpose**: Autonomous system behavior adaptation

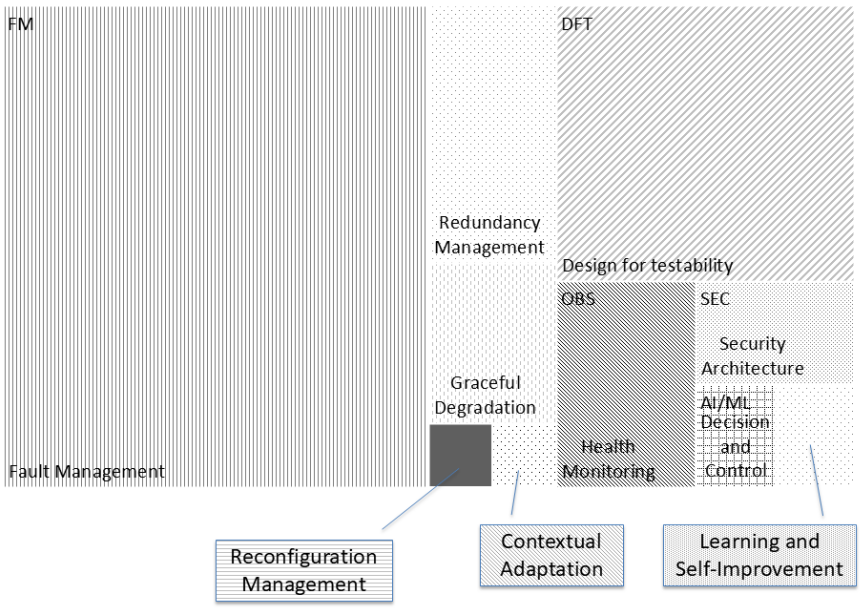


Figure 5.2. Functional Impact on Resilience Efficiency Map.

– **Example**: AI updating recovery strategies based on historical data

Figure 5.2 presents a treemap-based representation of the functional components that together underpin system resilience. Each block's area corresponds to a professionally informed estimate of that function's relative impact on overall resilience effectiveness; these percentages are expert judgments derived from accumulated observation of the of these resilience functions in product implementations, not the result of a formal quantitative measurement or statistically validated analysis. FM is therefore identified as a key contributor within this estimated weighting. The treemap both summarizes the set of functions and emphasizes their weighted contributions, offering a structured, interpretive overview to support the design and enhancement of resilient systems.

5.2 Resilience Quantitative Conceptual Model

Once we have identified the functional groups underlying a system's resilience, we need a quantitative model of resilience to truly understand its effectiveness. Figure 5.3 illustrates the resilience triangle, a model introduced by Bruneau in 2003 [137] that has gained recognition in research addressing resilience. It is the conceptual, quantitative model of the system's resilience and the direct translation of the very definition of resilience. It illustrates the extent of the disturbance event's impact on the system's performance and the reaction over time aimed at recovering full functionality.

if

$$\Delta_F = F^{(\max)} - F^{(\min)}, \quad where\ 0 < \Delta_F < 1$$

represents the impact on the proper functioning of the system produced by the disturbance at time $t^{(\mathrm{occ})}$ and

$$\Delta_T = t^{(\mathrm{rec})} - t^{(\mathrm{occ})}$$

represents the time interval between the time of disturbance $t^{(\mathrm{occ})}$ and that of full functional recovery $t^{(\mathrm{rec})}$, then the area of the triangle

$$\mathcal{A}_{\mathcal{R}} = \frac{1}{2}\Delta_F \Delta_T \tag{5.1}$$

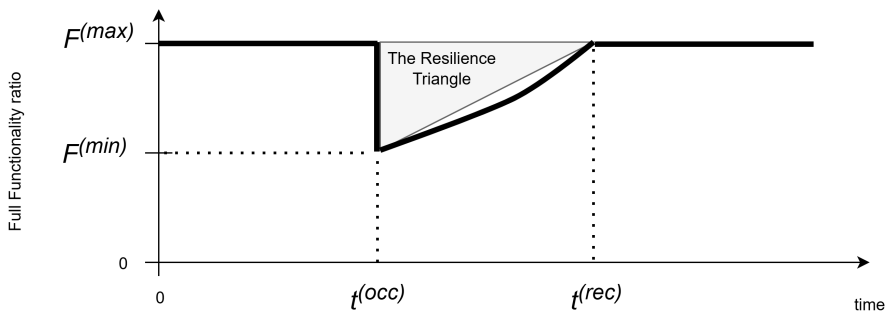


Figure 5.3. Bruneau's Model [137] for the Resilience Triangle.

it is the measure of the impact caused by the disturbance on the system.

$$\mathcal{R} = \Delta_T \left(1 - \frac{\Delta_F}{2}\right) \tag{5.2}$$

The area below the triangle is the quantitative value that allows us to evaluate the resilience of a system. Therefore, by definition, improving the resilience of a system means minimizing the area of the triangle. That is, reduce the Δ_F impact of the disturbance on system performance or reduce the Δ_T recovering interval time. By extension, even in the presence of a system behavior that is not as linear as that represented by the resilience triangle, we can continue to maintain the definition of resilience unchanged as the area below the curve (of which the triangle represents its linear approximation). If $\mathcal{F}(t)$ is the curve describing the functional variation of the system over time as a result of a disturbance ($with\ 0 < \mathcal{F}(t) < 1$), then the measure of resilience is represented by the following equation:

$$\mathcal{R} = \int_{t^{(\mathrm{occ})}}^{t^{(\mathrm{rec})}} \mathcal{F}(t)dt \tag{5.3}$$

The resilience triangle represents an intuitive model, useful for an initial classification of a system's resilience, but it has significant application limitations. On the other hand, the limitations of Bruneau's model are well known. Several research groups have addressed these limits in their respective application domains [138, 139, 140], but, to the best of our knowledge, they lack a detailed analysis in the RAN embedded system domain, the context covered by this thesis. In particular, Bruneau's model does not lend itself to adequately describing the system's ability to isolate an error condition, nor to representing the contribution provided by the filtering mechanisms that detect or prevent defect or fault conditions before they evolve into failure. The reader can intuitively associate the ascent ramp of the triangle with the error recovery actions. However, the role of detecting and supervising the system's health status is much less evident. Similarly, the model does not clarify how L3 level functions should be considered, for example, what impact the management of degraded functions or the redistribution of residual resources has on the resilience triangle. These limits appear critical for the present work, which focuses on mapping resilience

functions to understand how they contribute to increasing the overall resilience of a system, and necessitate a conceptual revision of the original definition.

The shortcomings identified in Bruneau's model become particularly evident when applied to the realistic case of an embedded RAN system. In the FM context, this implies recognizing the distinction between the moment a disturbance occurs and the moment the system can detect it, identify the root cause, and activate the corresponding recovery action. Furthermore, the ability to fully restore operational functionality is a key dimension of resilience in itself, rather than an absolute necessity. In cases where a fault condition is irreversible, it may be more advantageous to allow the system to operate under controlled conditions, rather than suspending service altogether. In the baseband domain, for example, keeping a node active, albeit at reduced capacity, is a preferable strategy, since the maximum number of users served by a single node is only reached in exceptionally high and infrequent traffic conditions.

Figure 5.4 illustrates our proposal for the quantitative resilience model, which overcomes the inherent limitations of Bruneau's model and allows a more faithful representation of the resilience properties of a complex system. We can still apply Equation 5.3 to calculate the resilience. The *dots triangle*, the *cross hatch triangle* and the *dashed rectangle* are the approximation of the area over the curve, and we can follow similar computation of Equation 5.2

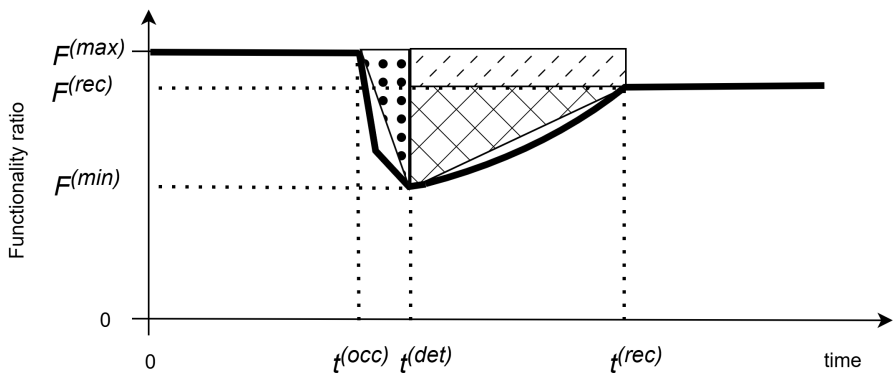


Figure 5.4. The New Quantitative Model for the Resilience Triangle.

if $\Delta_{F_{gap}} = F^{(\max)} - F^{(\min)}$ is the disturbance drawback to the system function, and $\Delta_{F_{rec}} = F^{(\mathrm{rec})} - F^{(\min)}$ is the level of functionality recovered after the recovery action, and $\Delta_{F_{lost}} = F^{(\max)} - F^{(\mathrm{rec})}$ is the functionality lost due to a degraded function condition then the area of the three polygons is:

$$Triangle_{dots} = \frac{\Delta_{F_{gap}}(t^{(\mathrm{det})} - t^{(\mathrm{occ})})}{2}$$

$$Triangle_{cross} = \frac{\Delta_{F_{rec}}(t^{(\mathrm{rec})} - t^{(\mathrm{det})})}{2}$$

$$Rect_{dashed} = \Delta_{F_{lost}}(t^{(\mathrm{rec})} - t^{(\mathrm{det})})$$

Figure 5.5 shows the resilience parameters, as introduced in chapter 4, into the shape of the new resilience triangle model. The figure allows for an immediate visualization between the times of occurrence, detection, and recovery, and the temporal parameters of resilience. Indeed, returning to the definitions of 4.5 and 4.7, we can rewrite the previous equations:

$$Triangle_{dots} = \frac{\Delta_{F_{gap}} MTTD}{2}$$

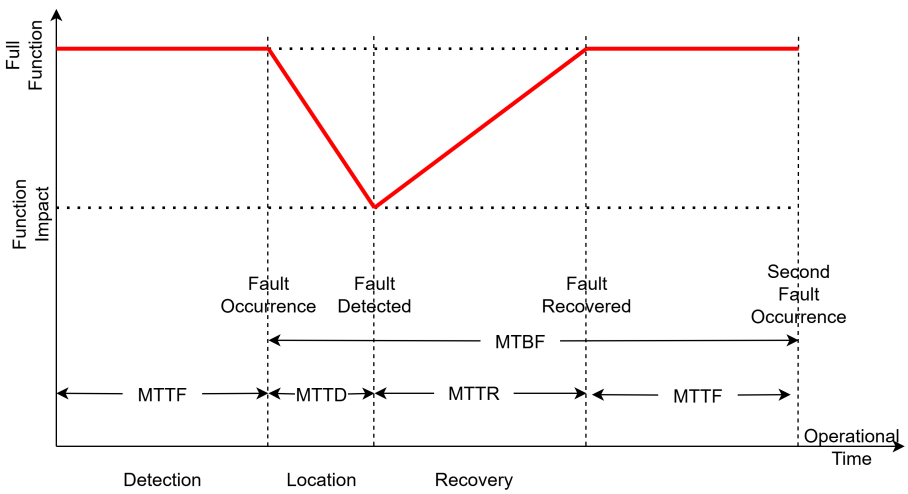


Figure 5.5. The Resilience Parameters Map into the New Resilience Triangle Model.

$$Triangle_{cross} = \frac{\Delta_{F_{rec}} MTTR}{2}$$

$$Rect_{dashed} = \Delta_{F_{lost}} MTTR$$

the areas of the three geometric objects in figure 5.4 are the components of the area of the *triangle* 5.1 to minimize. Since increasing the resilience of a system means reducing the area of the *triangle*, we are now able to identify areas to work in:

- Reduce MTTD time.
- Reduce MTTR time.
- Reduce the impact of disturbance on system function f(min).
- Reduce the permanent loss necessary for managing the degraded function condition.

The analysis of each area enables a deeper understanding of the implications associated with enhancing system resilience. The reduction of the dots triangle represents system robustness, and it is an important factor for the system's resistance. Improving the system robustness means strengthening both the capacity and efficiency of the fault detection mechanism illustrated in Figure 4.2. The primary objective is to minimize the temporal gap between the occurrence of a fault and its subsequent detection. Nevertheless, this alone is insufficient: it is equally crucial to act on fault localization and fault isolation to minimize as much as possible the disruptive impacts on the overall system functionality. Indeed, the functional cost associated with such disturbances also influences the computation of the cross-triangle area; hence, the ability to mitigate such impacts yields significant benefits.

The reduction of the cross triangle, in reference to Figure 4.2, represents fault recovery, that is, the process through which the system responds to an error condition and restores its functionality. Minimizing the area of this triangle not only reduces system recovery time but also enables the system to continue functioning even in the event of a permanent fault, albeit with reduced performance due to a resource availability restriction. The capability to identify,

isolate, and remove a fault resource is the enabler for the introduction of the dashed rectangle, which reduction denotes the system's capability to operate under degraded functionality. While not formally part of fault supervision, this element belongs to resilience level L3, as described in Chapter 5.1. The correlation with FM becomes even more evident when considering the sub-functions of the five macro-areas illustrated in Figure 4.2. Specifically, we define [54]:

- ***Fault Masking*** Fault masking refers to the system's ability to maintain full functionality despite the occurrence of a fault. It is often based on the adoption of redundancy mechanisms (data, hardware, or execution-based). In summary, fault masking ensures that faults do not compromise system operations, enhancing both reliability and overall performance.

- ***Fault Reporting*** Fault reporting refers to the system's ability to identify and communicate a fault condition within its components in a timely and accurate manner. This process involves signaling the occurrence of a fault, which subsequently enables corrective measures such as isolation and recovery. Well-structured and transparent fault reporting mechanisms support rapid diagnostic activities and efficient remediation efforts.

- ***Fault Coverage*** Fault coverage quantifies the extent to which a system's design and testing processes are capable of detecting and addressing faults. It measures the completeness of detection mechanisms and the system's ability to identify potential failure points. A high degree of fault coverage reflects a rigorous approach to FM, significantly reducing the likelihood of undetected critical faults.

- ***Fault Tolerance*** Fault tolerance refers to a system's ability to maintain acceptable performance levels despite the presence of faults or failures. It relies on the incorporation of redundancy, error detection, and recovery mechanisms to ensure continuous operation under adverse conditions. By adopting fault-tolerant architectures and techniques, systems can maintain reliability even in the face of disruptive events.

- ***Fault Avoidance*** Fault avoidance strategies aim to proactively identify and mitigate potential faults before they evolve into critical failures. This

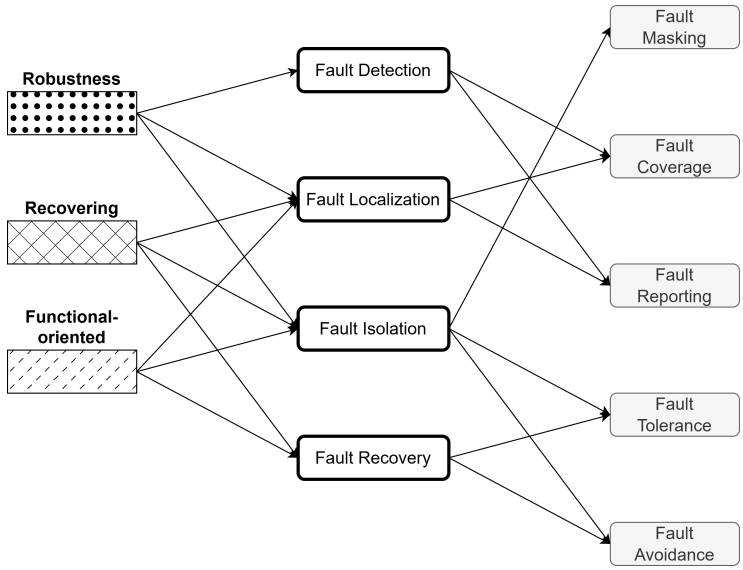


Figure 5.6. Map of FM Micro Functions into the Resilience Triangle.

anticipatory approach involves rigorous design, validation, and verification processes to minimize the probability of faults occurring during operation. As a complement to other FM techniques, fault avoidance enhances system robustness and reliability.

Figure 5.6 provides a detailed representation of the specific functional improvements within FM that directly contribute to enhanced system resilience. The figure also presents an initial mapping between the five foundational pillars of FM (see Figure 4.2) and their respective sub-functions. Chapter 8 expands on this analysis, offering a more comprehensive examination of the interrelations among FM functions.

Figure 5.7 further illustrates the relationship between the three geometric objects of the resilience triangle and the macro-functional contributions to resilience, as defined in Chapter 5.1. It shows that a strong interdependency exists between DFT and FM. This dependency becomes immediately evident when considering that the reduction of disruptive event impacts on system

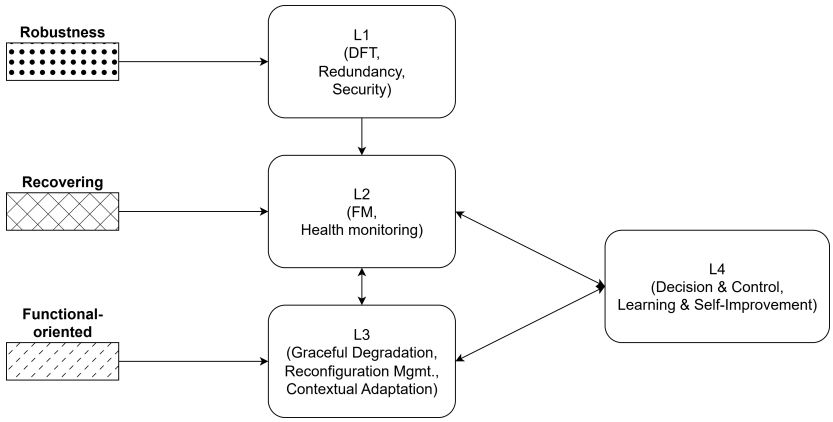


Figure 5.7. Map of Resilience Functions into the Resilience Triangle.

functionality (how quickly the system reacts to a fault condition thanks to the DFT self-awareness requirement) has a direct impact on the system's recovery capacity, namely the parameter $\Delta_{F_{rec}}$.

VI

Chapter 6

Dependability and Failure Probability

The core principle advanced in this thesis is that optimal FM design should be regarded as a means for augmenting overall system resilience. From this perspective, FM can be defined as the set of policies, characteristics, and implementation practices that aim to increase a system's tolerance to internal and external threats, with the objective of restoring service availability as quickly as possible. FM, when viewed through this principle, allows us to explicitly visualize its relationship to the concept of dependability. In systems engineering, dependability denotes the overall property that expresses the degree to which a system can be trusted to deliver its intended service.

For the sake of clarity, and by restricting the taxonomy to dependability alone (thereby excluding the contributions of Security), Figure 6.1 (derived from the work of Laprie et al. [141] on the dependability taxonomy) facilitates the understanding of the logical and functional relationship between dependability and resilience. If we consider their formal definitions only, dependability can be described as the system characteristic that guarantees the correctness, continuity, and security of the service over time (prevention, protection, and reliable design), while Resilience refers to the system's ability to preserve service stability during and after unexpected shocks (impact absorption, behavioral response, and recovery). A key interpretative link between these two concepts is service

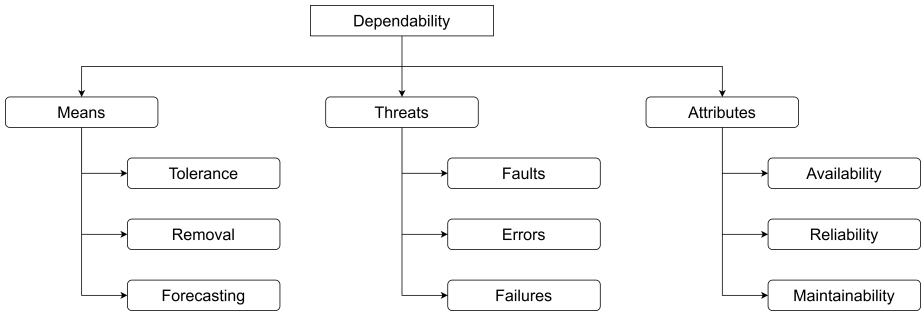


Figure 6.1. Dependability Taxonomy.

"Availability," which, as discussed in Chapter 1, was identified as the most significant emerging challenge for the design of embedded RAN systems. Our research shows that the ultimate objective of FM is system reliability, an attribute of dependability, and that FM directly targets the reduction of resilience-related parameters, such as MTTR and MTTD, which are themselves measures of the system dependability. System resilience constitutes a key contributor to overall dependability, alongside security. In other words, dependability is directly influenced by the resilience of the system. Therefore, the optimization of FM is a critical enabler for attaining acceptable levels of dependability. The primary intersection between dependability and fault management lies in the enhancement of reliability, although fault management also contributes significantly to other dependability attributes such as availability and safety. Reliability is defined as the probability that a system will perform its intended function within a specified period of time or operate in a given environment without failure. In this context, "Quality" refers to a system's ability to deliver a given service at a high level of performance. With respect to system quality, a more operational definition of Reliability is "the extent to which the system maintains that performance level over time under different circumstances" [142]. For instance, the number of end users that can be served at a given power consumption represents a measure of the quality of the baseband product, whereas the system's ability to sustain the same number of end users over years of operation, across the expected operational lifetime, and under the environmental and operational conditions specified at design time, constitutes a measure of its Reliability.

The reliability function for a hardware device is expressed by the following equation [143]:

$$R(t) = \Pr(T > t) = \int_t^{\infty} f_T(x)\, dx = 1 - F_T(t)$$

where:

- T is the random variable Time To Failure (TTF) of the device;
- $f_T(t)$ is the probability density function of the TTF;
- $F_T(t)$ is the cumulative distribution function of the TTF;
- $R(t)$ represents the probability that the device is operational beyond time t.

The evaluation of system reliability necessarily relies on the characterization of the failure process (as clearly stated in Equation 4.2), typically expressed through the failure rate and the mean time to failure. Reliability $R(t)$ represents the probability that a component remains operational up to time t, whereas the failure probability represents the probability that the same component has entered a failure state within the same time horizon. These two quantities therefore describe the same underlying stochastic phenomenon from complementary viewpoints. As a consequence, reliability and failure probability are intrinsically linked and cannot be treated as independent measures, but rather as mathematically dual representations of system behavior over time.

$$R(t) = 1 - P_f(t)$$

6.1 Dependability Evaluation

The qualitative evaluation of dependability identifies, assesses, and classifies failure modes (i.e., the evolutionary dynamics of the states of system components that lead to a failure condition). Quantitative evaluation, by contrast, aims to measure reliability in probabilistic terms [120]. Several quantitative analysis models exist:

- **Combinatorial models**: statistical analysis of the dependencies among the components of a system. These models include, among others, the Reliability Block Diagram (RBD) [144], the Fault Tree (FT) [145], and the Reliability Graph (RG)[146, 147].

- **State-space-based models**: modeling of statistical, temporal, and spatial dependencies through Markovian processes. These models include, among others, the Continuous Time Markov Chain (CTMC) [148] and the Stochastic Petri Net (SPN) [149].

- **Hybrid models**: combinatorial models with temporal dependence. These models include, among others, the Dynamic RBD [150] and the Dynamic FT [151].

- **Hierarchical models**: by adopting different formalisms and strategies across different layers of the system, they employ both combinatorial and state-space-based models, thereby mitigating their respective limitations [152].

The analysis of a system's reliability depends on the operational relationships among its constituent elements. A system service is enabled by the interaction of its subsystems; therefore, the effectiveness of the service cannot be assessed without an accurate understanding of their interactions, nor can reliability be computed or improved without understanding the contribution of each subsystem to service stability. It should be noted that a system may exhibit multiple functional relationships among subsystems as a direct consequence of the complexity of the service provided. In practice, a service such as radio access relies on the simultaneous availability of multiple micro-services, each characterized by its own chain of relationships among subsystems and for which a specific reliability can be computed, in addition to the reliability of the micro-services relationship chain itself. The RBD is nothing more than a representation of the chain of relationships among subsystems: it is a graphical model of the elements of a system and allows the system reliability to be derived starting from the reliability of individual elements [153].

6.2 Hypergraph Theory Recall

Graph theory [154] considers both undirected and directed graphs. For an abstract mathematical representation, a directed graph

$$\mathcal{G} = (V_g, E_g) \tag{6.1}$$

is defined by a set V_g of vertices and a set E_g of edges, where E_g is a subset of $V_g \times V_g$, the set of ordered pairs of vertices. C. Berge introduced the concept of a hypergraph in the 1960s [155] as a generalization of graphs where edges join any number of vertices. A (finite) *hypergraph* is the pair $\mathcal{H} = (V_h, E_h)$ where V_h is a finite set of elements called *vertices*, E_h is a finite set of elements called *edges*, and $e \subseteq V_h$ for each element $e \in E_h$. Hypergraphs can be represented using incidence matrices. The *incidence matrix* $B = (b_{ij})_{m \times n}$ of a hypergraph $\mathcal{H} = (V_h, E_h)$ with $V_h = \{v_1, \ldots, v_m\}$ and $E_h = \{e_1, \ldots, e_n\}$ is defined by

$$b_{ij} = \begin{cases} 1 & \text{if } v_i \in e_j, \\ 0 & \text{otherwise.} \end{cases}$$

We introduced the concept of Vertex-Weighted Hypergraph [11], a generalization of the hypergraph, and the concept of r-incidence matrices, which generalizes the concept of incidence matrices for hypergraphs. A *vertex-weighted hypergraph* is a tuple $\mathcal{H} = (V_h, E_h, w)$ where (V_h, E_h) is a hypergraph and $w : V_h \to \mathbb{R}$ is a weight function which assign a real number $w(v)$ to each vertex $v \in V_h$. If a hypergraph represents a set of sets of edges in a graph $\mathcal{G}$, and $\mathcal{G}$ has an associated edge property vector $\mathbf{w}$, then a vertex-weighted hypergraph can be easily constructed by associating $\mathbf{w}$ to the vertices of the hypergraph, since they correspond to the edges of $\mathcal{G}$. For any real number $r \in \mathbb{R}$, let the *r-incidence matrix* $B_r = (b_{ij})_{m \times n}$ of a vertex-weighted hypergraph $\mathcal{H} = (V_h, E_h, w)$ with $V_h = \{v_1, \ldots, v_m\}$ and $E_h = \{e_1, \ldots, e_n\}$ be defined by

$$b_{ij} = \begin{cases} w(v_i) & \text{if } v_i \in e_j, \\ r & \text{otherwise.} \end{cases}$$

Note that the 0-incidence matrix B_0 of a vertex-weighted hypergraph $\mathcal{H} = (V_h, E_h, w)$ with $w(v) = 1$ for all $v \in V$ equals the common incidence matrix B of the hypergraph (V_h, E_h). The notation $B_r = R(r, w, B)$ denotes the

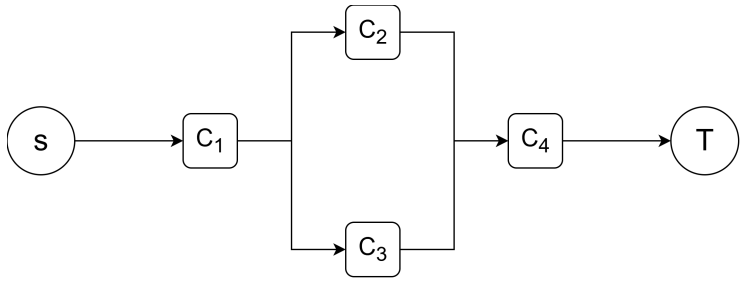


Figure 6.2. RBD With n = 4 Components. The Set $V = \{C_1, C_2, C_3, C_4\}$ Identifies the Components. $\mathcal{E} = \{E_1, E_2\}$ Represents the $\{s, t\}$ Connection Relation, Where $E_1 = (C_1, C_2, C_4)$ and $E_2 = (C_1, C_3, C4)$.

r-incidence matrix of a vertex-weighted hypergraph $\mathcal{H} = (V_h, E_h, w)$, where B represents the incidence matrix of the underlying hypergraph (V_h, E_h) without vertex weights.

As mentioned before, the RBD provides a graphical representation of the logical structure of a system, namely its structure function. If the system is coherent (i.e., monotone and with no irrelevant components), and if $V = \{1, 2, \ldots, n\}$ denotes the set of components in the RBD, then $\mathcal{H} = (V, \mathcal{E})$ defines the relational hypergraph of RBD, where $\mathcal{E} = \{E_1, \ldots, E_m\},\ E_j \subseteq V$, and each E_j corresponds to a set of components that are connected to each other in the RBD according to the connection logic. Figure 6.2 depicts an illustrative example of RBD. The caption describes the related hypergraph.

6.3 A Novel Quantitative Model

In the current session, we are particularly interested in combinatorial models. Among combinatorial models, FTA is one of the most widely adopted methods for systematic risk assessment; however, its application to complex and large-scale systems introduces well-documented challenges: unmanageable tree size, difficulties in detecting event duplication across multiple diagram paths and non-obvious common cause failure, and dependency on highly skilled analysts

for the correct interpretation of advanced logic gates such as inhibit, house, or conditional events [6]. The method proposed in this section is specifically designed to address the challenges of FTA, offering a structured, more accessible alternative that preserves the analytical rigor of classical FTA while mitigating its principal drawbacks. Without loss of generality, we consider the various subsystems to be characterized by independent failure events with exponential distributions, as already described by equation 4.1.

The basic rule for evaluating probability is that two independent components arranged in series exhibit a failure rate equal to the sum of the failure rates of the cascaded blocks. Conversely, two independent components arranged in parallel exhibit an overall failure rate equal to the product of their respective failure rates, as can be readily deduced from the reliability calculation rules of the RBD model [153]. The proposed quantitative mathematical model is based on these two simple operations. It assumes that, starting from an arbitrary graphical representation of a system, a system characterization must necessarily include:

- an operation that represents the contribution to the characterisation due to the serialisation of the device blocks,
- an operation that represents the contribution to the characterisation due to the parallelisation of the device blocks,
- a transformation T of the system hypergraph that expresses the complexity of the relationships among the different subsystems in the graph, namely the fact that a non-trivial RBD is neither purely serial nor purely parallel (even though the transformation itself is capable of representing both a simple serial relationship and a parallel one).

Equation 6.2 summarises these three essential elements of the mathematical model, although not necessarily in the order prescribed:

$$\Psi(P) = OP_s \circ OP_p \circ T \tag{6.2}$$

Applying the proposed mathematical model requires determining which operations are meaningful for characterizing the graphical representation under consideration, as well as which transformation should be adopted as a measure of the functional correlation among the subsystems involved.

6.4 Model Application Example

As a first use case, we consider applying the new quantitative model to evaluate failure probability from an RBD. The algorithm [6] is based on the following steps:

i **Goal definition**: define the boundaries of interest. Similar to the FTA process [145], the first step is to define the boundaries of interest, that is, to select the system functionality for which to calculate the failure probability.

ii **Independent Failure chain identification**: Analysis of the system architecture. A failure path is an independent system component subset that, in a bistable behavior condition, can compromise the functionality under investigation if it fails. A failure chain is a set of components along a failure path that cause a failure condition for that path.

iii **Resolution tuning**: Handling system complexity. It is always possible to view complex, large systems as the sum of multiple subsystems or events. Any subsystem can then be considered an independent system, with its own fault paths and chains. The method works hierarchically, building a larger hypergraph from the subsystem hypergraph based on the deduced relationships between subsystems along the fault path.

iv **Hypergraph 1-incidence matrix identification**: The components identified in the second step represent the vertices, and the failure chain represents the edges of the hypergraph $\mathcal{H}$ for the system under investigation. By using vertices as rows and edges as columns, it is possible to build the 1-incidence matrix of $\mathcal{H}$.

v **Hypergraph coherency and validity check**. Hypergraph reduction. During the building of the system hypergraph, it is possible that it contains duplicate failure paths and should be removed.

vi **Failure Probability calculation and evaluation**: quantification of the fault probability. Use equation 6.2 for failure probability calculation and $B_r = R(r, w, B)$ for fault tolerance classification.

In this section, the proposed model is applied to the quantitative evaluation of failure probability through a representative case study: an electric train motor control system. Figure 6.3 provides a schematic representation of the system. It comprises two electric motors (m_1, m_2), both transmitting torque to the train through a common shaft. System start-up and shutdown are controlled by a simple switch (s). The power supply consists of two batteries (b_1, b_2) and a power selector (ps) that automatically switches to the operational battery when the other is failed or drained. Each motor is connected via two connectors ($c_{ij} = c$).

The proposed methodology for failure probability calculation is applied as follows. For the sake of analytical tractability, it is assumed that both motors are characterized by a failure probability m, batteries by a failure probability b, the switch by a failure probability s, the power selector by a failure probability p, and connectors by a failure probability c.

Following the procedure outlined in the previous sections, the first step is to define the boundaries of interest. In the present case, the objective is to compute the probability of loss of control of the system, understood as the inability to either start or stop the motor.

The analysis proceeds by adjusting the level of resolution through the identification of subsystems and their functional interactions. From a functional

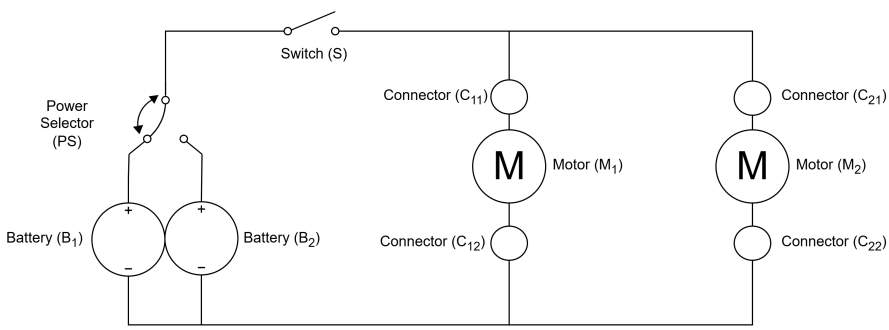


Figure 6.3. Electrical Train Schematic.

standpoint, the schematic can be decomposed into three subsystems connected in series: the battery subsystem, the switch subsystem, and the motor subsystem. Figure 6.4 provides a visual representation of this decomposition.

The failure path graph can then be constructed by noting that, due to the serial functional interaction among subsystems, system failure occurs whenever at least one subsystem is in a failed state. Consequently, all possible failure paths correspond to combinations in which one of the subsystems fails. If V_h = (Battery Subsystem (SB), Switch Subsystem (SS), Motor Subsystem (MS)) and E_h = (Failure Path 1, Failure Path 2, Failure Path 3), the identification of these failure paths enables constructing the incidence matrix of the system as a diagonal matrix, with rows corresponding to the three identified subsystems and columns representing the set of failure paths. This matrix constitutes the initial step in the computation of the hypergraph T in equation 6.2.

$$B(\mathcal{G}(V_h, E_h)) = \begin{array}{c|ccc} & fc1 & fc2 & fc3 \\ \hline SB & 1 & 0 & 0 \\ SS & 0 & 1 & 0 \\ SM & 0 & 0 & 1 \end{array}$$

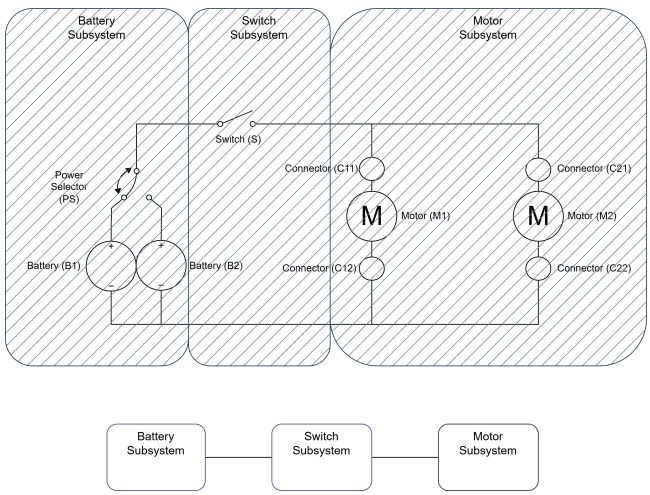


Figure 6.4. Electrical Train Subsystems.

Denoting by S_b, S_s, and S_m the failure probabilities of the battery, switch, and motor subsystems, respectively, $w = (S_b, S_s, S_m)$, the 1-incidence matrix B_1 associated with B can be derived accordingly.

$$B_1 = R(1, w, B) = \begin{array}{c|ccc} & fc1 & fc2 & fc3 \\ \hline SB & S_b & 1 & 1 \\ SS & 1 & S_s & 1 \\ SM & 1 & 1 & S_m \end{array} \tag{6.3}$$

B_1 is the T transformation. Within the framework of equation 6.2, serial operations in the computation of failure probability are represented by sum, whereas parallel operations are represented by multiplication. The operational selection is coming from the classical serial and parallel failure probability rules in the Reliability Theory [142]. The expression assumes independent component failures and is valid as an approximation only when the individual failure rates (or interval failure probabilities) are sufficiently small so that higher-order joint-failure terms can be neglected. The application of the equation 6.2 leads to the result that:

$$FailureProbability = \sum_j \prod_i T_{ij} = S_b + S_s + S_m$$

The method now involves applying the same flow described so far for each of the subsystems considered. We will do this starting from the battery subsystem.

It is worth to remember that the power switch (PS) has the task of automatically switching to the working battery in the event of a failure. Therefore, failure paths are represented by the following combinations:

- both batteries are faulty;
- b_1 is faulty and ps is stuck on it
- b_2 is faulty and ps is stuck on it

If V_h = (Battery 1, Battery 2, PowerSwitch) and E_h = (Failure Path S_{b_1}, Failure Path S_{b_2}, Failure Path S_{b_3}), the identification of these failure paths enables constructing the incidence matrix of the battery subsystem cutset B_{S_b}

$$B_{S_b}(\mathcal{G}_{\mathbb{S}}(V_h, E_h)) = \begin{array}{c|ccc} & fp1.1 & fp1.2 & fp1.3 \\ \hline b_1 & 1 & 1 & 0 \\ b_2 & 1 & 0 & 1 \\ ps & 0 & 1 & 1 \end{array}$$

The 1-incidence matrix B_1 associated with R_{S_b} can be derived accordingly.

$$(B_1)_{S_b} = R(1, w, R_{S_b}) = \begin{array}{c|ccc} & fp1.1 & fp1.2 & fp1.3 \\ \hline b_1 & b & b & 1 \\ b_2 & b & 1 & b \\ ps & 1 & p & p \end{array}$$

The analysis of S_b enables a new definition of B_1 with greater detail than that provided by equation 6.3. Note that functional interactions with other subsystems are preserved by adding "1", which ensures consistency in the size of the final matrix.

$$B_1 = R(1, w, B) = \begin{array}{c|ccccc} & fp1.1 & fp1.2 & fp1.3 & fp2 & fp3 \\ \hline b_1 & b & b & 1 & 1 & 1 \\ b_2 & b & 1 & b & 1 & 1 \\ ps & 1 & p & p & 1 & 1 \\ S_s & 1 & 1 & 1 & S_s & 1 \\ S_m & 1 & 1 & 1 & 1 & S_m \end{array} \quad (6.4)$$

The process for the switch subsystem is trivial, since there is a single component: there is only one failure path, and it is s. The motor subsystem has redundancy, since there are two motors in parallel. Actually, any of the connectors might be faulty, so we can identify the failure path by the following possible combination: c_{11} or c_{12} or m_1 is faulty while c_{21} or c_{22} or m_2 is faulty. If V_h = (c_{11}, c_{12}, c_{21}, c_{22}, m_1, m_2) and E_h = (Failure Path S_{m_1}, Failure Path S_{m_2}, ... Failure Path S_{m_k}), the identification of these failure paths enables constructing the incidence matrix of the motor subsystem cutset B_{S_m}. Note that we are interested in the minimal cutset. The failure path is modeled as a Boolean expression of the form $(c_{11} \vee m_1 \vee c_{12}) \wedge (c_{21} \vee m_2 \vee c_{22})$. The minimal cutset

matrix is obtained by considering all elementary combinations generated by the conjunction of the two disjunctions, that is, the Cartesian product of the sets $\{c_{11}, m_1, c_{12}\}$ and $\{c_{21}, m_2, c_{22}\}$. Since each disjunction contains three elements, the total number of distinct combinations is $3 \times 3 = 9$. No duplicates are generated, as each combination is uniquely defined by the selection of one element from each term of the conjunction.

$B_{0S_m}(\mathcal{G}_{\mathbb{S}}(V_h, E_h)) =$

	3.1	3.2	3.3	3.4	3.5	3.6	3.7	3.8	3.9
c_{11}	1	1	1	0	0	0	0	0	0
c_{12}	0	0	0	1	1	1	0	0	0
c_{21}	0	1	0	0	1	0	0	1	0
c_{22}	0	0	1	0	0	1	0	0	1
m_1	0	0	0	0	0	0	1	1	1
m_2	1	0	0	1	0	0	1	0	0

The 1-incidence matrix B_1 associated with R_{S_m} can be derived accordingly.

$B_{1S_m} = R(1, w, R_{S_m}) =$

	3.1	3.2	3.3	3.4	3.5	3.6	3.7	3.8	3.9
c_{11}	c	c	c	1	1	1	1	1	1
c_{12}	1	1	1	c	c	c	1	1	1
c_{21}	1	c	1	1	c	1	1	c	1
c_{22}	1	1	c	1	1	c	1	1	c
m_1	1	1	1	1	1	1	m	m	m
m_2	m	1	1	m	1	1	m	1	1

Based on the results obtained from the subsystem anlysis, equation 6.4 becomes:

$B_1 =$ (6.5)

	1.1	1.2	1.3	2	3.1	3.2	3.3	3.4	3.5	3.6	3.7	3.8	3.9
b_1	b	b	1	1	1	1	1	1	1	1	1	1	1
b_2	b	1	b	1	1	1	1	1	1	1	1	1	1
ps	1	p	p	1	1	1	1	1	1	1	1	1	1
S_s	1	1	1	s	1	1	1	1	1	1	1	1	1
c_{11}	1	1	1	1	c	c	c	1	1	1	1	1	1
c_{12}	1	1	1	1	1	1	1	c	c	c	1	1	1
c_{21}	1	1	1	1	1	c	1	1	c	1	1	c	1
c_{22}	1	1	1	1	1	1	c	1	1	c	1	1	c
m_1	1	1	1	1	1	1	1	1	1	1	m	m	m
m_2	1	1	1	1	m	1	1	m	1	1	m	1	1

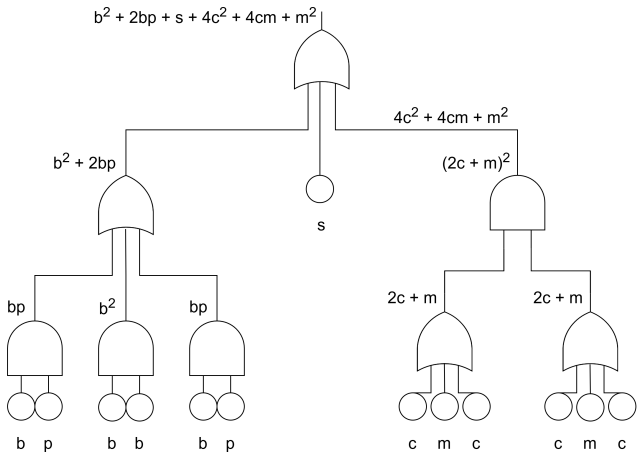


Figure 6.5. FTA for the Train Motor Control System.

The train motor control system failure probability is then:

$$FailureProbability = \sum_j \prod_i T_{ij} = S_b + S_s + S_m = b^2 + 2bp + s + 4c^2 + 4cm + m^2 \tag{6.6}$$

It is possible to validate the method by using the FTA. Fig. 6.5 shows the FT based on the logic gate rules ([145]). Shortly, a logic gate OR requests the sum of the probability of the input events, while a logic AND requests the product. As aspected the failure probability calculated using FTA matches equation 6.6.

6.5 Method Verification in Different Domain

The abstraction formalized in Equation 6.2 derives its conceptual robustness from a construction grounded in an abstract combinatorial structure, the relational hypergraph, rather than from assumptions tied to a specific application domain. This design choice naturally prompts the question of whether the abstraction preserves its validity when transferred to domains other than the one in which it was originally developed, namely, hardware RBDs. Since the

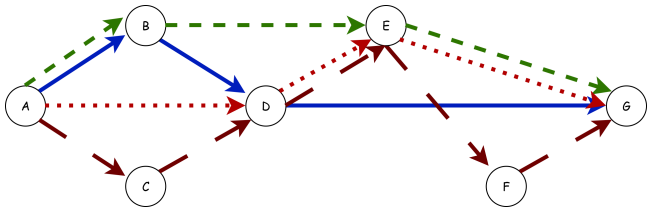


Figure 6.6. Network Graph Example.

primary domain addressed in this research is telecommunications, the applicability of the proposed mathematical model to networking scenarios has been examined critically. This analysis explicitly accounts for both the expressive capabilities of the abstraction and the structural limitations that arise when it is confronted with domain-specific constraints, modeling conventions, and operational requirements [11].

The graphical representation of a network adopted in this thesis builds on the general notion of a collection of devices or users interconnected to exchange and transfer data. A classical and widely accepted formalization is provided by Hoffman [156], who defines a network as a connected linear graph composed of points, referred to as nodes, and directed line segments connecting pairs of nodes, referred to as links. Given a network with a set of nodes $n_1, \ldots, n_k$, a directed path from a source node s to a destination node d is defined as a sequence of network links $p = \{(n_0, n_1), \ldots, (n_{i-1}, n_i), (n_i, n_{i+1}), \ldots, (n_{n-1}, n_n)\}$ such that $n_0, n_1, \ldots, n_n$ are distinct nodes, $s = n_0$, $d = n_n$, and $(n_{i-1}, n_i) \in$ the set of the network links for $i = 1, 2, \ldots, n$.

By defining $V_g = (v_1, \ldots, v_k) = (n_1, \ldots, n_k)$ as the set of vertices corresponding to the network nodes, and $E_g = (e_1, \ldots, e_h) = (link_1, \ldots, link_h)$ as the set of edges corresponding to the network links, the network can be described in a natural and intuitive manner through Equation 6.1, that is, as a directed graph. In detail (see Fiaschi et al. for the formal approach description [11]), if the Pathset $P = p_1, p_2, \ldots, p_k$ is defined as the set of paths characterizing the network, a hypergraph-based representation can be employed to encode the incidence matrices associated with the corresponding graph. Within this framework, the hypergraph provides a compact and expressive abstraction that is structurally equivalent to the routing table associated with the defined Pathset.

Figure 6.6 depicts a simple network example on which we apply the mathematical model to calculate the failure probability, while

$$B(\mathcal{G}(V_h, E_h)) = \begin{array}{c|cccc} & \begin{array}{c} p_1 \\ blue \\ solid \end{array} & \begin{array}{c} p_2 \\ green \\ dashed \end{array} & \begin{array}{c} p_3 \\ brown \\ loosely \end{array} & \begin{array}{c} p_4 \\ red \\ dotted \end{array} \\ \hline A \to B & 1 & 1 & 0 & 0 \\ A \to C & 0 & 0 & 1 & 0 \\ A \to D & 0 & 0 & 0 & 1 \\ B \to D & 1 & 0 & 0 & 0 \\ B \to E & 0 & 1 & 0 & 0 \\ C \to D & 0 & 0 & 1 & 0 \\ D \to E & 0 & 0 & 1 & 1 \\ D \to G & 1 & 0 & 0 & 0 \\ E \to G & 0 & 1 & 0 & 1 \\ E \to F & 0 & 0 & 1 & 0 \\ F \to G & 0 & 0 & 1 & 0 \end{array}$$

is the *incidence matrix*. It is equivalent to the network routing table. The identification of the significant T transformation starts from B. The next step needs to introduce the meaning of the minimal cutset.

Let $P = (p_1, p_2, \ldots, p_m)$ be an S–D pathset in $\mathcal{G} = (V_g, E_g)$, and denote by $E(P) = \{e_1, \ldots, e_k\} \subseteq E_g$ the set of edges induced by P. Let define the reachability relation $\rightsquigarrow_X \subseteq V \times V$, for any $X \subseteq E(P)$, by

$$u \rightsquigarrow_X v \iff \exists \text{ a path from } u \text{ to } v \text{ in } X.$$

A set of edges $C \subseteq E(P)$ is called an S–D cutset relative to P if $\neg\big(S \rightsquigarrow_{E(P)\setminus C} D\big)$ and the cutset C is minimal (with respect to set inclusion) if $\forall C' \subsetneq C, S \rightsquigarrow_{E(P)\setminus C'} D$. Equivalently,

$$C \text{ is a minimal cutset} \iff \begin{cases} \neg\big(S \rightsquigarrow_{E(P)\setminus C} D\big), \\ \forall e \in C,\ S \rightsquigarrow_{E(P)\setminus(C\setminus\{e\})} D. \end{cases}$$

The notation $\mathbf{R}_{\mathrm{Cut}}(P)$ indicates the incidence matrix of the minimal cutsets of P.

$\mathbf{R}_{\mathrm{Cut}}(P) =$

	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9	c_{10}	c_{11}	c_{12}	c_{13}	c_{14}	c_{15}	c_{16}	c_{17}	c_{18}	c_{19}	c_{20}	c_{21}	c_{22}	c_{23}	c_{24}	c_{25}	c_{26}	c_{27}	c_{28}	c_{29}
$A \to B$	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$A \to C$	1	1	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0
$A \to D$	1	0	1	0	0	1	0	1	0	1	1	0	1	1	0	0	0	0	0	1	1	0	1	1	0	0	0	0	0
$B \to D$	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0
$B \to E$	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	1	1	1	1	1	0	0	0	0
$C \to D$	0	0	1	1	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0
$D \to E$	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0
$D \to G$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1
$E \to G$	0	1	0	1	0	0	1	0	1	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	1	1	1	1	1
$E \to F$	0	0	0	0	0	1	1	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0
$F \to G$	0	0	0	0	0	0	0	1	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1

Given the matrix $\mathcal{G} = (V_g, E_g)$, as defined in equation 6.1, we associate a weight vector $\mathbf{w} = (w_1, \ldots, w_n)$, where each component w_i denotes the probability of failure of the edge e_i.

Introducing this weight vector enables the computation of the transformation T by explicitly determining the 1-incidence matrix associated to $\mathbf{R}_{\mathrm{Cut}}(P)$.

$B_1 = R(1, w, \mathbf{R}_{\mathrm{Cut}}(P)) =$

	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9	c_{10}	c_{11}	c_{12}	c_{13}	c_{14}	c_{15}	c_{16}	c_{17}	c_{18}	c_{19}	c_{20}	c_{21}	c_{22}	c_{23}	c_{24}	c_{25}	c_{26}	c_{27}	c_{28}	c_{29}
$A \to B$	w_1	w_1	w_1	w_1	w_1	w_1	w_1	w_1	w_1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
$A \to C$	w_2	w_2	1	1	1	1	1	1	1	w_2	1	1	1	1	w_2	1	1	1	1	w_2	1	1	1	1	w_2	1	1	1	1
$A \to D$	w_3	1	w_3	1	1	w_3	1	w_3	1	w_3	w_3	1	w_3	w_3	1	1	1	1	1	w_3	w_3	1	1	w_3	1	1	1	1	1
$B \to D$	1	1	1	1	1	1	1	1	1	w_4	w_4	w_4	w_4	w_4	w_4	w_4	w_4	w_4	w_4	1	1	1	1	1	1	1	1	1	1
$B \to E$	1	1	1	1	1	1	1	1	1	w_5	w_5	w_5	w_5	w_5	1	1	1	1	1	w_5	w_5	w_5	w_5	w_5	1	1	1	1	1
$C \to D$	1	1	w_6	w_6	1	1	1	1	1	1	w_6	1	1	1	1	w_6	1	1	1	1	w_6	1	1	1	1	w_6	1	1	1
$D \to E$	1	1	1	1	w_7	1	1	1	1	1	1	w_7	1	1	1	1	w_7	1	1	1	1	w_7	1	1	1	1	w_7	1	1
$D \to G$	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	w_8	w_8	w_8	w_8	w_8	w_8	w_8	w_8	w_8	w_8
$E \to G$	1	w_9	1	w_9	1	1	w_9	1	w_9	1	1	1	1	1	w_9	w_9	w_9	w_9	w_9	1	1	1	1	1	w_9	w_9	w_9	w_9	w_9
$E \to F$	1	1	1	1	1	w_{10}	w_{10}	1	1	1	1	1	w_{10}	1	1	1	1	w_{10}	1	1	1	1	w_{10}	1	1	1	1	w_{10}	1
$F \to G$	1	1	1	1	1	1	1	w_{11}	w_{11}	1	1	1	1	w_{11}	1	1	1	1	w_{11}	1	1	1	1	w_{11}	1	1	1	1	w_{11}

Finally, remembering that the serial and parallel functional operators for calculating the failure probability in equation 6.2 are sum and multiplication, respectively, equation 6.7 is the complete expression for calculating the failure probability of the pathset.

$$FailureProbability = \sum_j \prod_i T_{ij} = \sum_j \prod_i B_{1ij} \tag{6.7}$$

6.6 Method Verification for Different Functional Characterization

The migration of the method into the network domain has enabled us to assess the flexibility of the proposed mathematical framework. We have shown that

Example	OP_s	OP_p	T
Delay	$+$	max	$R(0, w, \mathbf{R}_{\mathrm{Id}}(P))$
Administrative cost	$+$	$+$	$R(0, w, \mathbf{R}_{\mathrm{Uni}}(P))$
Combined capacity	min	$+$	$R(0, w, \mathbf{R}_{\mathrm{Cut}}(P))$
Unavailability	$1 - ((1 - (.)) \times (1 - (.)))$	$\times$	$R(1, w, \mathbf{R}_{\mathrm{Cut}}(P))$
Fault probability	$+$	$\times$	$R(1, w, \mathbf{R}_{\mathrm{Cut}}(P))$

Table 6.1. Operations and Transformations.

the transformation T depends on the specific characterization under analysis; however, our investigation indicates that the mathematical model remains valid even when alternative functional characterizations are considered [11]. In the network deployment, the optimal routing problem is well known to be central to overall system efficiency. The identification of an optimal pathset constitutes a holistic decision that simultaneously accounts for implementation costs, network capacity, tolerated transmission latency, network reliability, and the probability of congestion. Although algorithms specialized for individual characterizations exist([157, 158, 159, 160]), our mathematical model has demonstrated sufficient flexibility to provide a transversal and representative description across multiple characterizations. As discussed in detail in our related paper [11], by appropriately adapting the operational operators and the transformation, the proposed framework is capable of expressing latency, administrative cost, capacity, congestion, and failure probability at the pathset level, thereby establishing itself as a practically relevant analytical tool for network design and deployment decision-making.

We define $\mathbf{R}_{\mathrm{Id}}(P)$ as the incidence matrix of the pathset's hypergraph in $\mathcal{G}$. A very simple transformation of a path set P is the union of all the edges pertaining to its member paths. The union of a path set is a single set of edges, and it can be represented by a hypergraph with a single edge. We define $\mathbf{R}_{\mathrm{Uni}}(P)$ as its incidence matrix. According to the previous notation, Table 6.1 shows

Delay(P)	$\max_{i\|p_i\in P}\sum_{j\|e_j\in p_i} w_j$,
Cost(P)	$\sum_{i\|e_i\in\bigcup P} w_i$,
Capacity(P)	$\min_j \sum_i T_{ij}$, where $\mathbf{T} = R(0, w, \mathbf{R}_{\mathrm{Cut}}(P))$,
UnavailProb(P)	$1 - \prod_j (1 - \prod_i T_{ij})$, where $\mathbf{T} = R(1, w, \mathbf{R}_{\mathrm{Cut}}(P))$,
FaultProb(P)	$\sum_j \prod_i T_{ij}$, where $\mathbf{T} = R(1, w, \mathbf{R}_{\mathrm{Cut}}(P))$.

Table 6.2. Property Examples Instantiations.

a summary of network characterization calculation, and Table 6.2 shows their instantiations in a more direct notation.

VII

Chapter 7

Fault Management Cost Analysis

7.1 FM Cost System Definition

Through the analysis of system resilience, the previous chapters have underscored the critical role of FM for the system and why the efficient implementation of FM impacts the service availability. It is helpful to evaluate the business case supporting the FM investments. In fact, there is a common understanding that the cost of extracting a defect from an installed product could be up to 500 times higher than the cost of implementing FM at the very early stage [161, 162], during the conceptual definition of the product. However, as far as we can see, there are still very few cost analyses in embedded systems attempting to establish the relationship between FM and maintenance costs (e.g., [163]), and very few that provide general design guidelines (e.g., [164]). This analysis is vital for those investing in system solutions, as it helps determine whether a higher cost during the development phase (associated with FM) is justified to prevent significantly higher costs during the maintenance phase. Thus, it ultimately makes a sustainable contribution to the business case assessment.

The current chapter examines the costs associated with FM, intending to develop a systematic approach to guide decisions regarding its implementation. While FM remains the key target of the present analysis, considering it

Dimension	Classification
System Type	Engineered, Socio-Technical
Openness	Open System
Complexity	Complex System

Table 7.1. The FM Cost System Classification Table.

as an isolated or self-referential system would risk overlooking its interactions within a broader contextual framework. Such a framework is indispensable for a comprehensive evaluation of implementation costs, the consequent effects on corporate reputation from the customers' perspective, and the intricate balance between initial design expenditures and prospective long-term savings. Adopting a holistic system perspective [45], we arrive at the conclusion that the FM cost system considers the entire ecosystem of processes, decisions, and economic as well as operational outcomes that revolve around a product's FM. It includes both the design phase and the full life cycle. It integrates all relationships among costs, benefits, risks, and temporal dynamics associated with the implementation or omission of Fault Management [10]. In other words, the system must incorporate FM alongside the economic, operational, and temporal dynamics that inform well-founded design-phase decisions. The primary implication of such a definition is that the system can be explicitly delineated—clearly identifying the elements that constitute it and those that fall outside its scope.

7.2 System Classification

It is possible to analyze the FM cost system to identify its classification. Based on the INCOSE definitions [165], Table 7.1 summarizes the classification conclusion that follows in the text.

- **Engineered System**. FM Cost System is a composite of people, products, services, information, and processes that provides a capability that satisfies a stated customer need or objective.
  - The system is not naturally occurring—it is deliberately conceived

to analyze and optimize the cost-related aspects of FM.

  - It integrates both technical subsystems (hardware/software testing, field interventions) and human-driven processes (decision-making, customer interactions).
  - It is aligned with a business objective: minimizing cost while maintaining or improving quality and reputation, thus fulfilling a customer-centric need.

- **Socio-Technical System**. FM Cost System has closely coupled social and technical parts.
  - Social components include decision-making dynamics, strategic policies, and customer experience considerations.
  - Technical components involve FM technologies, operational cost structures, and intervention mechanisms.
  - The performance and outcome of the system depend on both technical efficiency and human judgment, typical of socio-technical systems.

- **Open System**. The FM Cost System has flows of information, energy, and/or matter between the system and its environment.
  - The system continuously interacts with its external environment:
    * Feedback loops from customers (reputation, satisfaction)
    * Economic evaluations and market expectations
    * Technological change and organizational learning
  - These interactions are essential to evaluate implementation impact and to inform future design decisions.

- **Complex System**. There are non-trivial relationships between cause and effect, potentially with feedback loops and entangled causal chains.
  - Multiple interdependent components (costs, reputation, field resources, design decisions) contribute to emergent outcomes.

- Feedback and time delays (e.g., how initial design savings may increase future costs or affect customer satisfaction) are key behaviors.
- Decision-making must account for these entangled effects, requiring holistic system analysis.
- Predicting system behavior is non-trivial due to its non-linear dynamics and multiple interacting variables.

The FM cost system is a **HYBRID** construct that integrates decisional and engineering dimensions. As a decisional model, the system incorporates strategic considerations, such as balancing short-term cost reductions against long-term operational risks. As an engineering model, it engages with the technical layers of system design, fault detection, and mitigation processes, which require a granular understanding of component behaviors and system interactions. The hybrid nature allows the model to support decision-making across organizational layers, ensuring operational realities effectively inform strategic intentions. Regarding behaviour, **DYNAMIC** and **ADAPTIVE** terms best characterize the FM cost system. For instance, in less mature implementations, the system may behave in a reactive manner—responding to faults as they occur. In contrast, more advanced implementations exhibit proactive and even predictive characteristics, leveraging real-time data and feedback loops to anticipate and prevent failures before they materialize. The adaptive behavior is continuous within the FM domain, reflecting the uninterrupted nature of system monitoring, feedback integration, and performance adjustment. The FM cost system state transitions are **NON-LINEAR**, meaning that inputs and interventions do not produce proportional outcomes. For example, a modest increase in design expenditure might yield disproportionately significant reductions in failure rates or customer complaints. The stochastic nature of the system compounds the non-linearity. Fault occurrences, user interactions, and environmental influences introduce inherent randomness, making purely deterministic models inadequate. The above classification underscores the importance of employing modeling approaches that are sensitive to the interplay between technical and human dimensions, capable of capturing evolving behaviors, and robust enough to address uncertainty. It positions the FM cost system not as a static, linear, or isolated structure but as an open, evolving, and interdependent system that must be understood through

Characteristic	SSM	CSP
Methodological flexibility	Use of a single methodology	Combined use of multiple methodologies
Origin	Soft Systems Thinking	Critical Systems Thinking
Structure	Learning cycle	Four EPIC phases
Approach	Participatory and interpretive	Critical, reflective, and multimethodological
Objective	Accommodation among worldviews	Continuous improvement in complex contexts
Evaluation	Not central but implicit	Fundamental in the “Check” phase

Table 7.2. The Systems Modeling Methods Summary.

dynamic, multi-perspective lenses to manage and optimize effectively.

7.3 System Modelling

Selecting a suitable modeling approach for the FM cost system requires alignment with the system’s hybrid, socio-technical, open, and complex nature. Table 7.2 summarizes the main differences between Checkland’s Soft Systems Methodology (SSM) [166] and Jackson’s Critical Systems Practice (CSP) [167], and it helps to analyze the FM cost system.

CSP is the more appropriate framework for analyzing and modeling this system. The FM cost system integrates both human and technical subsystems, includes strategic decision-making layers, and exhibits dynamic, non-linear, and adaptive behavior. It encompasses multiple stakeholders and systemic feedback loops that affect organizational learning, customer satisfaction, and long-term performance. These characteristics exceed the scope of a single worldview or modeling paradigm. As such, a method that supports methodological pluralism and explicit boundary critique is required. CSP, grounded in Critical Systems Thinking, provides a structured yet flexible approach to engaging complex, multi-dimensional systems. It allows practitioners to explore the system through

multiple systemic perspectives—mechanistic, organismic, cultural/political, societal/environmental, and relational. A characterization that it is essential for the FM cost system, whose behavior and outcomes cannot be fully captured by technical analysis alone. For instance, economic evaluations of early design decisions demand both quantitative cost models and qualitative insights into stakeholder priorities and policy constraints.

The FM cost system demands a modeling approach capable of navigating structural, behavioral, and normative complexity. CSP's critical, multimethodological foundation enables comprehensive exploration and intervention across social, technical, and strategic domains. Therefore, CSP represents the more suitable modeling methodology for supporting the analysis, design, and continuous evolution of the FM cost system.

7.4 System Complexity

As stated in the previous sections, the FM cost system is a system of independent systems (SoS) [168], each with its autonomy and often distinct aims: the technical subsystem, the social subsystem, the economic subsystem, and the control or decision-making subsystem. In [10] we have identified the following FM cost subsystems:

- Decisional
- Technical
- Economic
- Social

The FM cost system should be regarded as a complex adaptive system, as its constituent subsystems interact in a non-linear and context-dependent manner, precluding the reliable construction of predictive models based on simple cause-and-effect relationships. According to the Cynefin framework [169], situations in which the link between action and outcome can only be discerned retrospectively fall within the complex domain, where an iterative Probe–Sense–Respond approach is required. In such contexts, decision-making in FM cannot rely solely

on static or linear parameters; instead, it must incorporate continuous learning loops and the management of feedback arising from system malfunctions.

Further evidence of the system's complex nature arises from the tight coupling between its technical and economic dimensions. The value perceived by the end user—and the associated reputational and market costs—does not derive exclusively from technical indicators such as failure frequency or repair efficiency, but rather emerges from dynamic socio-technical factors including service continuity, perceived reliability, and user experience. This phenomenon of emergence, a defining feature of complex systems [168], implies that the overall behavior of the FM cost system cannot be reduced to the simple aggregation of its subsystems' behaviors. Accordingly, effective FM cost management requires adaptive governance strategies capable of sensing, learning, and responding to evolving interdependencies across technical, economic, and experiential domains.

Building on this conceptual understanding, the following section formalizes the complexity of the FM cost system in terms of measurable dimensions and associated metrics.

Complexity Dimensions

The FM cost system exhibits substantial complexity across multiple structural, relational, and functional dimensions. To capture and quantify this complexity, the analysis identifies a set of key complexity metrics that reflect the system's organizational architecture, the nature and density of interdependencies among its components, and the dynamic behaviors emerging from their interactions. These metrics provide a systematic framework to evaluate the complexity of the FM cost system and support evidence-based decision-making in the presence of non-linear and emergent phenomena.

- **Variety**: The system integrates at least six major classes, spanning technical, economic, and reputational aspects.

- **Interconnections and Dependencies**: Multiple cross-linked subsystems interact through tight coupling and feedback loops.

- **Hierarchy and Layering**: The architecture operates across multiple decision-making levels and stakeholder roles.
- **Entropy and Uncertainty**: Non-linear decisions and emergent behaviours drive unpredictability.
- **Computational Complexity**: The system requires stochastic, dynamic, and non-linear modeling approaches.
- **Functional and Organizational Complexity**: Strategic and technical goals interact with institutional culture and subjectivity.
- **Dynamic Complexity**: Delayed temporal feedback and adaptation dominate system evolution.

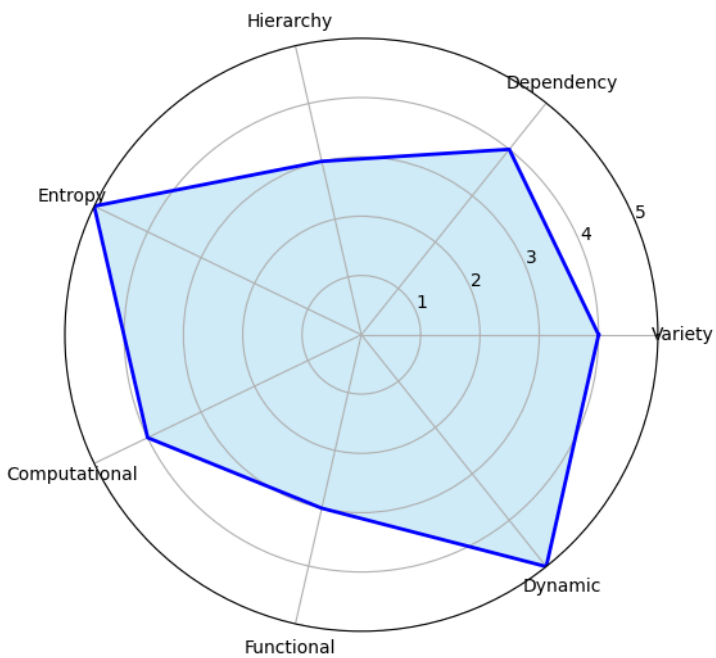


Figure 7.1. FM Cost System Complexity Metrics.

Figure 7.1 shows the complexity metrics perception over a 1-5 range of values.

The FM Cost System scores high across all identified complexity dimensions.

7.5 Game Theory FM Cost SoS Analysis

From the point of view of modeling and decision support, the independence of the subsystems and the multiplicity of actors allows the adoption of the Game Theory method. The FM Cost SoS has heterogeneous subsystems that operate semi-independently and point to individual goals while contributing to a larger systemic purpose. The scenario has dynamics similar to the Prisoner's dilemma or the tragedy of the commons [170], in which the pursuit of one's local interest can compromise the global optimum of the system.

Let the FM Cost SoS consist of N rational agents, each representing a distinct subsystem: S_1 (Technical), S_2 (Economic/Financial), S_3 (Decision-Making/Management), and S_4 (Customer Feedback/Reputational). Each agent selects a strategy $s_i \in \{C, D\}$, where C denotes cooperation (early FM investment) and D denotes defection (defer or avoid FM investment). Each agent evaluates its utility function $U_i(s_1, \ldots, s_N)$ based on local outcomes and systemic consequences.

The strategy profile $\mathbf{s} = (s_1, \ldots, s_N)$ defines the joint action. Each agent maximizes its utility U_i while forming expectations about the strategies of other agents.

Each subsystem evaluates cost-benefit trade-offs differently. For example, the technical subsystem associates cooperation with higher design complexity but reduced future failures. The financial subsystem emphasizes immediate capital expenditure. The decision-making subsystem balances stakeholder expectations, while the reputational subsystem reflects customer satisfaction and brand trust, which are affected by system performance and fault rates.

Let $U_i(C, \ldots, C) = R_i$ denote the long-term return from full cooperation. Let $U_i(D, \ldots, D) = M_i$ denote the myopic return from mutual defection. Typically, $R_i > M_i$ for all i, reflecting the global optimum. However, if

any agent $j \neq i$ defects, agent i may incur a utility loss due to asymmetric cooperation, denoted as $L_i = U_i(C, \ldots, C, D, \ldots, C) < R_i$.

It is the classical dilemma, such as the Prisoner's Dilemma, in which self-interested behavior leads to collectively suboptimal outcomes.

A strategy profile $\mathbf{s}^*$ constitutes a Nash Equilibrium if no agent gains by unilaterally deviating:

$$U_i(\mathbf{s}^*) \geq U_i(s_i', \mathbf{s}^*_{-i}), \quad \forall i, \forall s_i' \in \{C, D\}$$

In the FM SoS, mutual defection $(D, \ldots, D)$ often forms a Nash Equilibrium due to the local dominance of D without enforceable coordination.

Aspect	**Current Suboptimal State**	**Recommended Approach / Benefit**
Equilibrium	$(D, \ldots, D)$	Shift toward globally optimal $(C, \ldots, C)$ through coordination mechanisms
Agent Behavior	Local optimization under uncertainty; low incentive to invest in early FM	Align local utility functions with systemic goals
Consequences	High long-term costs, increased downtime, reputational degradation	Reduced lifecycle costs, enhanced system resilience, preserved reputation
Intervention	None or ad-hoc	Implement FM requirements, common policies, and long-term KPIs from the early design phase
Key Takeaway	Short-term decisions harm overall system performance	Early coordination and FM feature selection improve system robustness and cooperation

Table 7.3. Summary of Systemic Coordination and FM Interventions.

Although all agents recognize the superior systemic outcome under full cooperation, each fears the vulnerability associated with unilateral cooperation.

Without systemic mechanisms for coordination (FM requirements and policy, long-term KPI), agents face strong disincentives to invest early in FM. Thus, local optimization under uncertainty reinforces the equilibrium at $(D, \ldots, D)$, resulting in heightened long-term costs, increased downtime, and reputational degradation. To shift the system toward the globally optimal equilibrium $(C, \ldots, C)$, the SoS must implement coordination mechanisms that realign local utility functions with systemic goals. The necessity to prevent subsystems from making short-term decisions highlights the importance of FM requirements and common policy from the early phase of the embedded system design process. Choosing a specific FM feature for a device at this stage can substantially improve system resilience. Table 7.3 shows how, through such interventions, the organization can promote a cooperative equilibrium that enhances system resilience, reduces lifecycle costs, and preserves long-term reputational capital.

7.6 The RR - Fault Coverage Relation

In Chapter 5.2, we examined the relation between the FM functions and resilience parameters. It is possible to invert that mapping and use it to derive FM requirements from desired resilience targets. That is, by specifying the wanted value of resilience parameters, it is possible to deduce which FM actions must be present. Because of the complexity of the FM cost as a system, the acceptable resilience-parameter values themselves will be part of the decision dynamics of the design process.

For instance, equation 4.4 defines a commonly used resilience parameter, the return rate (RR). In practice, product owner aims to minimize RR, since it often represents the most direct proxy of a product's maintenance cost: the higher the RR, the greater the maintenance cost. In one of our paper [10], we showed a formal relationship between RR and the FM — in particular via the concept of fault coverage. In details, the return rate $\mathcal{P}_{\text{RR}}$ is

$$\mathcal{P}_{\text{RR}} = 1 - e^{-D_{\text{avg}} \cdot A_0 \cdot (N+1) \cdot (1-F)}$$

where: D_{avg} represents the average number of defects per unit area of silicon

(e.g., defects per cm^2, considered, as a first approximation, comparable between each component and the whole board). It is a statistical measure that describes how many defects occur within a given area during the semiconductor manufacturing process. E.g., in a $7nm$ technology is approximately D = 0.09; A_0 is a factor representing the physical area of a single die (chip) that is considered for defect assessment; N is the number of devices housed on the board, and F is the fault coverage, as the probability of detecting defects.

It shows two important outcomes:

- Low fault coverage will increase the $\mathcal{P}_{\text{RR}}$, and it allows a correlation between the indirect cost of downtime and reputation and the cost of the FM implementation.
- There is a limit where increasing the fault coverage doesn't provide significant benefits. For example, in a $7nm$ domain, five components hosted on board, and A_0 of a cm^2, a yield of 99% will require an F of $\approx 99\%$, which is unrealistic and expensive. But by setting the fault coverage to 97%, we can achieve a Yield of $\approx 98.4\%$, which is still an acceptable result.

VIII

Chapter 8

Fault Management Model

8.1 FM Subfunctions Relationship

We resume the definition of Fault Management (FM) as presented in Chapter 4.1.3. FM represents the system function responsible for supervising, locating, isolating, and recovering from a fault condition, to restore the system to its nominal operating state as rapidly as possible. The present section provides an in-depth analysis of the functions involved in fault handling, organizing them into the four macro-areas mentioned above. As illustrated in Figure 4.2, this study highlights the increasing importance, within FM, of the fault prediction function. Chapter 5.2 introduces an initial level of correlation between the FM subfunctions and the five fundamental pillars — detection, prediction, localization, isolation, and recovery. Figure 8.1 employs a pseudo-sunburst diagram to provide a more detailed representation of the relationships between FM subfunctions and these five pillars. This analysis is particularly relevant as it demonstrates how the improvement of an apparently secondary FM function can have a significant impact on the overall system resilience, which constitutes the primary objective of this research. The arrows depicted in the Figure 8.1 represent the logical correlations among the FM pillars. The processes of prediction and detection share a common foundation based on continuous system monitoring. Prediction follows a parallel path, aimed at implementing preventive actions before the fault condition manifests itself; for instance, through the recognition

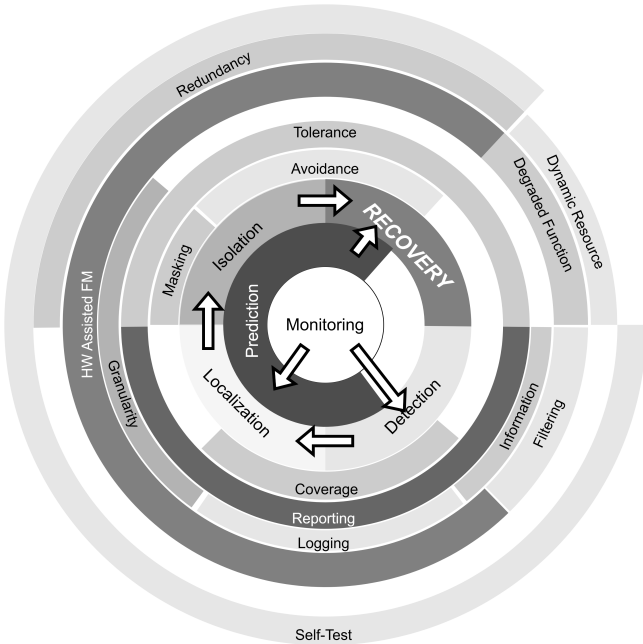


Figure 8.1. Sunburst-Like Chart for the FM Functions.

of defects or deviations from the system's nominal operational pattern. Except for this specific feature, we may state that, within the logical sequence, the phase following system monitoring is detection, namely the system's capability to identify a fault condition. Subsequently, localization focuses on pinpointing the defect with the highest possible precision. At the same time, isolation, building upon the acquired fault information, enacts a series of countermeasures to contain its propagation within the system, potentially without interrupting service continuity. Finally, once the nature of the fault has been clarified, together with the associated information and the adopted isolation policies, the system applies recovery policies to eliminate the faulty condition. The graph presented in Figure 8.1 is referred to as a pseudo-sunburst diagram, because the outer circles do not necessarily represent a hierarchical relationship, although they maintain, in the graphical representation, the description of the relationships

among the various subfunctions. Although several entries in Figure 8.1 are largely self-explanatory, it is helpful to recall their precise meaning.

- *Granularity* By granularity, we refer to the system's capability to report a fault in a sufficiently "atomic" manner, i.e., with a level of spatial and temporal detail that allows the faulty part to be identified and isolated. The finer this indication, the more localized and isolable the fault condition becomes. High granularity in fault indication strongly influences both detection and recovery strategies. For example, consider a case of bit corruption in memory. If the FM function generically reports the fault as a failure of the entire device, the isolation action must necessarily involve the entire device, causing a service interruption. Conversely, suppose the system can indicate that the fault pertains to a single array or even a specific memory section. In that case, the FM system may perform localized isolation actions, allowing the service to continue with limited degradation. It is important to note that hardware design with enhanced fault granularity also helps limit the monitoring and detection overhead that would otherwise occur without direct hardware support [171]. The definition of granularity is inherently system-dependent. It requires awareness of the fault model, i.e., the type of fault and the independence among the components' parts. Finally, granularity plays a key role in architectural design: defining modules or components with clear boundaries, ensuring modularity, and guaranteeing that each "indivisible part" is effectively isolable. Granularity thus becomes a means to implement economically sustainable redundancy solutions. Returning to the previous example, device-level redundancy is considerably more expensive than redundancy implemented at the level of a single or multiple memory arrays.

- *Logging* The subsystem responsible for recording defect, fault, and recovery conditions provides a detailed history of the product's status, as detected by the FM. The primary objective of such a system is to ensure visibility of transition states, as illustrated in Figure 4.3, at any time and regardless of the board's current state. Consequently, the log must reside in non-volatile system memory, guaranteeing information retention even in the event of central processor failures. Another direct implication of

state independence is that logging must necessarily be a distributed system. The *spread in the system* characterization of the logging implies the presence of multiple storage mechanisms, accessible regardless of which board component may be in a fault condition. It is crucial not to confuse the physical multiplicity of the log with the existence of different logs within the system. For example, observability logs are indispensable tools for characterizing the system under analysis and are essential for any form of prediction. In this context, it is appropriate to consider multiple logging systems providing complementary and detailed information. Prediction represents a key example: comparative analysis of different system logs can reveal significant patterns that not only allow identification of the error type but can also help determine the root cause, i.e., the primary source of the malfunction.

- *Information* Error information represents the data that signals and describes the presence of an error in a system. It constitutes the functional basis of error detection, as it enables the identification of error conditions and the preparation of subsequent corrective actions. Such information may take several forms, control codes, diagnostic messages, or warning indicators, and aims to provide meaningful guidance for anomaly recognition. Although closely related to granularity, error information fulfills a distinct function. While granularity determines the level of detail in error representation, error information defines the content of what is communicated about the error itself. The two functions are therefore complementary, as the effectiveness of detection and correction processes depends on both the information content provided and the degree of detail with which detection presents the information.

- *HW-assisted FM* This feature represents the accelerator of every efficient fault management solution. The expression “HW-assisted fault management features” denotes the set of hardware-level mechanisms that support fault management, including detection functions such as error correction codes and machine checks, recording and reporting infrastructures that expose error records and timestamps to software levels, and hardware mitigation capabilities such as on-the-fly correction, containment of

faulty components, and self-diagnosis (BIST). These features reduce the load on diagnostic and recovery software by providing high-granularity telemetry and out-of-band management channels. However, they require well-defined interfaces, reporting conventions, and integration with filtering and correlation policies to prevent false positives or inappropriate interventions. The term "HW-assisted Fault Management features" is helpful because it encapsulates the central concept: hardware is not only prone to failure but also provides telemetry, protection, and response capabilities that reduce the load on software fault management functions.

- *Filtering* The fault filtering function is responsible for analyzing the raw stream of events generated by a system in order to reduce informational noise and limit the number of alarms in subsequent FM phases processing. It does more than suppress duplicates or redundant notifications [172]; it also evaluates the temporal and statistical nature of occurrences to distinguish transient phenomena from persistent conditions [173]. In this way, it can differentiate primary events, indicative of the actual cause of the failure, from secondary events that merely represent consequences. Filtering, therefore, assumes a role of preliminary interpretation: it determines whether recovery action is actually required, identifies abnormal trends reflecting suboptimal configurations or progressive degradation, and provides the correlation and diagnosis modules with a more consistent and reliable data set.

- *Degraded Function* In embedded systems for RAN, errors are inevitable. As discussed in the cost analysis chapter 7, performance or budget constraints may not always guarantee complete recovery from all errors. However, a fault often affects only a portion of the system, while other functionalities can continue operating. In such cases, isolating the fault condition and adapting the available services allows the system to keep only the critical ones active. This principle is known in the literature as graceful degradation, or functional degradation [174]. The degraded function of FM aims to manage the functional degradation. When a non-recoverable error condition is detected, the degraded function determines which services to suspend and which to maintain, guiding the system

toward a controlled degraded state. To make these decisions, the degraded function relies on two key types of information: the dependencies between services and sub-services, represented by a graph $G_s(V_s, E_s)$ (e.g., if the data transmission service depends on the synchronization service, suspending synchronization also interrupts data transmission); the usage of hardware resources by services, represented by a graph $G_r(V_s, E_r)$ (e.g., if two services share the same CPU, the degraded function must decide which service to prioritize to optimize the use of available resources). In practice, the degraded function acts as a *conductor*: it understands the relationships between services and resources and adapts the system to ensure continuity of core functionalities.

- *Redundancy* In the context of embedded systems, redundancy refers to the intentional introduction of additional functional capabilities not aimed at improving system performance, but rather at enhancing its ability to recover from failures. In other words, redundancy consists of integrating backup resources designed solely to enable recovery from a fault [141]. It represents one of the most established reliability mechanisms, whose conceptual origins date back to the 50s [175].

  Traditionally, redundancy is associated with the availability of spare resources, but the literature distinguishes several types. The first taxonomic level separates *Spatial Redundancy* from *Temporal Redundancy* [176].

  a Spatial redundancy involves duplication or addition of components, functions, or data;

  b Temporal redundancy relies on repeating computation or data transmission operations over time and comparing results to verify consistency.

  A second level of classification concerns the nature of the redundant resource:

  i *Hardware Redundancy* involves the physical duplication of components (e.g., dual power supplies);

ii *Software Redundancy* involves code mechanisms for error detection or correction (e.g., watchdogs, rollback points, or alternative algorithms with equivalent functionality);

iii *Data Redundancy* counts on the addition of redundant information or metadata (e.g., Error Detection and Correction Codes or RAID configurations);

iv *Functional Redundancy* performs the same function across distinct systems, as in the well-known Triple Modular Redundancy (TMR) [177] scheme, which determines the correct value via a majority voting mechanism.

Each redundancy type presents specific advantages and disadvantages, and the choice of the most appropriate strategy depends strictly on the constraints imposed by the FM cost function. Factors such as cost, size, and energy consumption vary significantly depending on the adopted redundancy strategy. In the context of this research, which focuses on the hardware domain, attention is primarily directed toward spatial redundancy techniques, at the hardware, data, and functional levels, with particular consideration of the constraints mentioned above.

- *Dynamic Resource Handling* The term *Dynamic Resource Handling* refers to the system's capability to adapt the allocation of hardware resources (computing, memory, and networking) in response to dynamic boundary condition changes, such as service variations, traffic peaks, or error conditions, to maximize system performance [178]. As mentioned for the degraded function, dynamic resource handling capability is fundamental to enabling the system to reach a viable operational state even under failure conditions affecting one or more components. Once FM design establishes possible recovery policies, dynamic resource handling allows the system to implement corresponding reallocations or reductions of available resources. For example, if the monitor detects memory arrays in permanent failure, the degraded function, depending on the designed redundancy implementation, may decide to isolate a single array or an entire memory device. Consequently, the system employs dynamic resource

handling to remove damaged memory portions (swap-outs) and reallocate remaining resources among running services according to their priority.

- *Self-Test* Self-Test is a special case of HW-assisted FM that deserves its own dedicated bullet in the list. Each component's ability to independently verify its operational status is an increasingly common requirement in embedded systems. The Self-Test concept, in the context of the RAS Self-Describing concept 4.2.4, denotes the component's ability to assess its operational state, identify and locate any fault conditions, and execute recovery mechanisms. It is a wide task that implies that the type of self-test must adapt the goal to the actual operational state and diagnostic objective. It is possible to identify three main operational conditions: startup, regular operation, and fault presence—each requiring a different testing approach, as the test's objective depends on the device's working mode [179]:

  - *PBIT* (Power-up Built-In Test) – performed at system startup; it may include structural and functional tests without particular constraints on performance or service availability. Its main goal is to maximize fault coverage before operational startup.
  - *CBIT* (Continuous Built-In Test) – executed continuously during normal operation; it must not compromise system performance or availability. Its purpose is to prevent silent faults by ensuring constant monitoring of the component's status.
  - *IBIT* (Initiated Built-In Test) – activated on demand; it runs with the component isolated from the system. Isolation enables meticulous and potentially more intrusive diagnostics, aimed at detailed fault localization or recovery of defective parts.

8.2 FM Deployment Model

Based on the analyses presented in the previous chapters, we can assert that improving the resilience of a system essentially means containing a fault condition both spatially—that is, limiting its impact on the system's function—and

temporally—that is, minimizing the time required to restore the system to its operational state prior to the fault (or to an acceptable degraded mode). Furthermore, through the detailed analysis of FM (Fault Management) functions, we have concluded that efficient spatial and temporal containment of a fault condition necessarily entails the coordinated employment of FM functionalities across multiple technological domains. Hardware, firmware, and software must be designed and enhanced to ensure the effective and functional implementation of fault management. From these fundamental principles, it becomes possible to develop a framework for implementing FM as a system-level function. The key implementation assumption is that a system-oriented approach to fault recovery represents the core and primary objective of the entire FM function (see Figure 4.4). In other words, fault recovery constitutes its ultimate goal (see Figure 8.1).

To enhance the operational resilience of the system, the FM function must be conceived as a collaborative service involving all technological layers (hardware, firmware, and software)—not as fragmented entities, but as autonomous actors, each responsible for developing and maintaining the four macro-functions: detection, localization, isolation, and recovery. Each layer, within its own domain of responsibility, performs diagnosis and mitigation actions appropriate to its capabilities and, most importantly, communicates in a structured manner three essential elements:

a: information about the detected fault,

b: the recovery action undertaken locally, and

c: the outcome or result of that action.

The immediate and structured transmission of this information ensures that recovery is always executed, whenever possible, at the point closest to the detection. This minimizes the temporal dimension of the fault event—reducing both the effective intervention time and the total degradation period—without requiring the entire FM process to be centralized or any single layer to execute all actions. In other words, optimal recovery is local and coordinated: local because the most effective response often originates where the fault is detected,

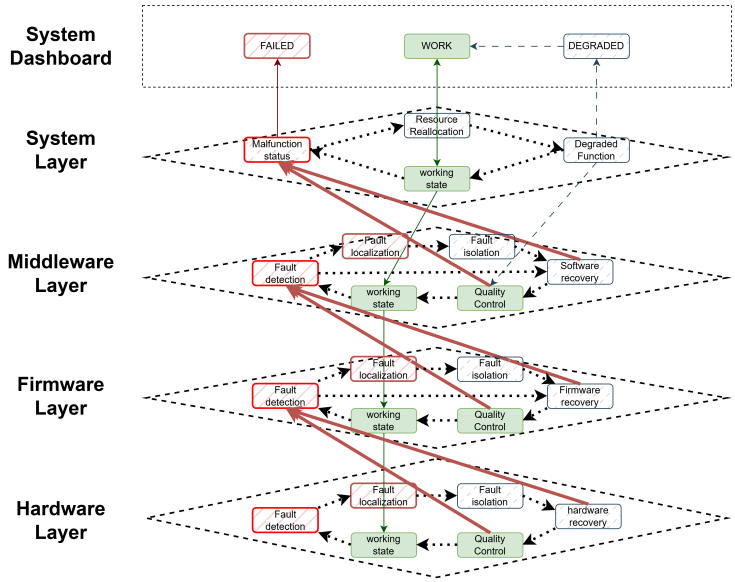


Figure 8.2. FM Deployment Framework.

and coordinated because the rest of the system is informed of the action and its outcome, allowing it to adjust operational and escalation policies accordingly.

This collaborative architecture entails three key operational properties:

i reduced reaction latency, since recovery actions initiated near the detection point lower the operational MTTR (Mean Time To Repair);

ii traceability of executed actions, as each layer records and communicates decisions and outcomes; and

iii controlled escalation capability, whereby higher layers intervene only when local mitigation is insufficient, using normalized information and the history of prior actions. These properties make the model not only pragmatic but also verifiable and reproducible during system testing, without introducing redundant functions or creating technological silos.

Figure 8.2 shows the multiple-layer FM deployment framework. The proposed FM implementation framework considers the architecture structured into

multiple technology layers, each with a distinctive role in fault detection, localization, isolation, and recovery. The main layers are:

- Hardware: includes the physical components of the system.
- Firmware: represents hardware proximity software, often identified as BIOS, uboot, boot loader, or configuration firmware, depending on the type of system.
- Middleware: constitutes the support platform for applications, such as an operating system or a service framework.
- System layer: includes the software responsible for FM functions.

Hardware does not merely detect fault conditions, but must be able to locate the fault in a detailed and granular manner, implementing timely corrective actions to limit cascading effects on other component modules or board systems. In addition to simply reporting the fault, it must carry out local recovery actions within its competence. Even in the event of successful recovery, the layer must support or implement a process of functional quality evaluation to identify any degradation of performance that may indicate component aging phenomena or adverse environmental conditions. If, at the end of those actions, the hardware fails to re-establish a stable working condition, it propagates the error and recovery information to the upper layer. Each higher layer, in a way consistent with its degree of abstraction, is called upon to carry out the four pillars of FM: detection, localization, isolation, and recovery. The detection of the upper layers consists of the ability to correctly receive and interpret, in a timely and unambiguous way, the information coming from the lower layer. The subsequent localization, isolation, and recovery phases instead operate on a progressively larger scale: as the layer increases, recovery actions may involve progressively larger portions of the system. For example, while hardware can act on a single cache line, firmware can perform a complete reset of the device, thus acting on the entire cache. The fault indication is propagated to the higher layer only if it is not possible to re-establish the working condition at the current layer.

A few key considerations emerge from this multi-layer structure:

Resource	Fault Impact	Recovery Action Effect	Alternative Action Effect
$\mathcal{R}_0$	$\mathcal{F}_0$	$\mathcal{A}_0$	$\mathcal{A}_{01}$
$\mathcal{R}_1$	$\mathcal{F}_1$	$\mathcal{A}_1$	$\mathcal{A}_{11}$
$\ldots$	$\ldots$	$\ldots$	$\ldots$
$\mathcal{R}_{\text{n-1}}$	$\mathcal{F}_{\text{n-1}}$	$\mathcal{A}_{\text{n-1}}$	$\mathcal{A}_{\text{(n-1)1}}$
$\mathcal{R}_{\text{n}}$	$\mathcal{F}_{\text{n}}$	$\mathcal{A}_{\text{n}}$	$\mathcal{A}_{\text{n1}}$

Table 8.1. Systematic Failure and Recovery Action Analysis.

- *Common purpose*: each layer has as its primary objective the restoration of the working condition. No layer lower than the FM functions can change the overall operating state of the system.
- *Two-way propagation*: Recovery actions may also require downward communications, for example, updating hardware registers following a state change.
- *Conductor role for the FM*: Fault Management acts as a conductor, coordinating recovery actions based on the error and recovery information received. It uses dynamic resource handling and degraded function condition mechanisms to keep the system in an operational state — possibly degraded — or to declare it in fault condition when the service is compromised.

In parallel, each layer must be aware of the impact of fault conditions and possible recovery actions on the entire system. In addition to the dependency graphs of the services $G_s(V_s, E_s)$ and the resources $G_r(V_s, E_r)$, the framework must also consider systemic impact relationships.

For instance, if a sensor fails to provide a temperature measurement, restarting the entire board would be an excessively invasive recovery action, having a greater impact on the service than the fault itself. There is a need for a comparative analysis between fault, fault effect, and recovery action effect, as described in Table 8.1. It is very similar to the process of evaluating fault impacts of the Failure Mode and Effects Analysis (FMEA) [180] method, but more meaningful for the domain of embedded systems. It is beneficial to identify the most appro-

priate recovery strategies during the design phase. For example, considering the faulty sensor case again, there is the possibility of selectively resetting only the faulty device to avoid resorting to recovery actions that are much more intrusive than the fault itself.

8.2.1 A Framework Deployment Use Case: System Memory

System memory is a crucial component. Through a more or less sophisticated cache system, every application needs to store data in memory. Network traffic packets are also stored and managed by the system memory. Their constant technological evolution evidences the importance of memory devices. While performance improvement has always been and continues to be a strong driver of technological progress, in recent generations—such as with DDR5—resilience has taken on an increasingly important role, to the point that the standard has considered specific RAS features as mandatory in memory devices (Chapter 10 explores in detail the RAS requirements in hardware component design). One of the technological transitions that has demanded a higher value for system reliability has been the shift to nanometer technology, which has significantly increased the probability of bit flips.

The bit-flip phenomenon is not new [181]. It consists of a corruption of the charge in the memory transistor, resulting in the inversion of the associated bit value. Until not long ago, the primary cause of this problem was cosmic rays [61]. The corruption caused by cosmic rays has attracted the attention of hardware designers because it is both unpredictable and irreducible. While shielding memory is possible, it is not always a practical or cost-effective solution. Despite careful verification of specifications, implementation, and memory device quality—aimed at increasing fault avoidance [54], there exist structural differences in memory cell access timing that may cause bit-flips [182]. Similarly, despite high-quality design, RAM Design Flaws may still occur, consisting of signal reflections along data lines between RAM and the motherboard or CPU, potentially resulting in data corruption [183]. With the transition to nanometer technology, the reduced size of memory capacitors causes their stored charge to dissipate more easily [184]. This well-known event has necessitated more frequent memory refresh cycles. This sequence of causes of memory corruption, alternative to cosmic rays, concludes with a security vulnerability issue:

Row-Hammer, which induces bit inversions in adjacent cells [185], and the observation that memory devices exhibit fault probabilities that depend on the natural aging process [60]. All the aforementioned considerations lead to the logical conclusion that an effective reliability process for memory devices cannot be limited to the design phase alone but requires a runtime detection and recovery method. In this section, we demonstrate how the fault management model proposed earlier applies to the case of memory error management.

According to the proposed FM model 8.2, the hardware layer is the first level where faults are detected, located, and recovered. As already noted, starting from the DDR5 generation, memory devices are required to support error detection and correction techniques. The Error Correction Code (ECC) is a data redundancy mechanism that not only allows for the tracing of bit-flips but also enables the precise localization of where they occur and their correction. There exist algorithms capable of detecting and correcting a single bit-flip (Single Error Correction/Double Error Detection - SECDED), such as the Hamming code (72,64) [61]; two bit-flips (Double Error Correction/Triple Error Detection - DECTED), such as the Nordstrom-Robinson code [186]; or three bit-flips (Triple Error Correction/Forth Error Detection - TECFED), such as the extended binary Golay code [187]. Naturally, more complex codes correspond to more expensive solutions, but SECDED-type algorithms are available on all DDR5 memory devices, as explicitly required by device specifications [188]. It is possible, therefore, to assume that the employed architectural solutions include at least an SECDED-type ECC mechanism.

ECC

Figure 8.3 shows the reference system: one or more memory devices (here referred to as the memory subsystem) connected to a control unit, generally a multicore SoC with a complex hierarchical cache structure, interconnection system, and a functional unit called the Memory Controller (MC), dedicated to accessing and controlling the memory subsystem. Chapter 10 provides a detailed description of what a memory subsystem entails. ECC algorithms can be broadly described in a few steps:

- MC generates an ECC code, called a syndrome, for each write request as

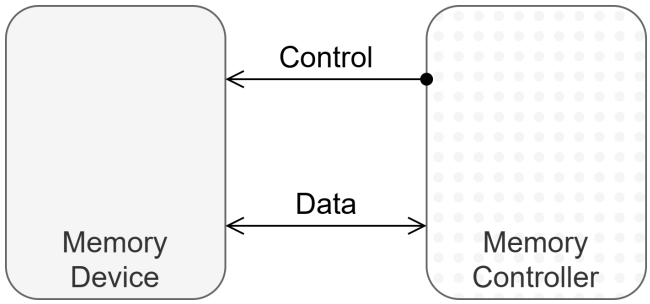


Figure 8.3. The Memory System Model.

a function of the data to be written.

- MC writes both the data and the syndrome to memory.
- During the read phase, MC reads the data, recalculates the syndrome, and compares it with the stored one.
- By comparing the read and recalculated syndromes, MC can determine whether a bit-flip has occurred and where.
- The bit-flips identified as erroneous are delivered corrected to the software. The data itself is not corrected in memory unless an additional mechanism (independent of the ECC algorithm) performs this correction.

There are several architectures for implementing ECC:

- **Side-band ECC:** MC sends the syndrome in side-band, that is, along with the data, in a single memory write. With a wider memory bus (for example, 72 bits for 64-bit data), the number of Read/Write operations remains constant, yielding positive effects on performance (no change in data access latency).
- **Inline ECC:** There are two partitions into the device memory: one for data and one for the syndrome. MC performs a double access in each Read/Write cycle—one to read/write data and the other to read/write the

syndrome. Inline ECC incurs a latency cost but protects against RAM Design Flaws when devices have a fixed and narrow bus width, as in the case of Low-Power-DDR (LPDDR) memories.

- **on-die ECC:** Starting with DDR5, memory devices integrate ECC implementation directly within the chip. In this case, the memory device itself generates the syndrome for each write, reads the syndrome for each read, corrects the data directly in memory, and sends the corrected data to the MC. Note that in this case, the memory device controls internal bit-flips but cannot identify issues such as RAM Design Flaws, for which it is common for the MC to continue performing some ECC mechanism.

- **link-ECC:** MC calculates the error syndrome during data write operations and sends it together with the data. The memory device recalculates the syndrome based on the received data, compares it with the received syndrome, and corrects potential single-bit errors. For read operations, the roles of the controller and memory device are reversed. The reader should note that link-ECC does not protect against single-bit errors within the memory array. Link-ECC was designed for LPDDR5 devices to address bus errors not covered by on-die ECC. In practice, the combination of on-die ECC and link-ECC enables point-to-point ECC coverage in LPDDR5.

Memory Scrubbing

The ECC mechanism is closely tied to direct memory access, meaning that an error is detected precisely when the data is required. Consequently, a single-bit corruption in an unread memory area remains uncorrected. Considering that bit-flip events follow a normal distribution and are independent of previous occurrences, this implies that the probability of UCE (double bit-flip in SECDED) in infrequently accessed memory areas is higher. To minimize the probability of double errors, both the MC and the memory device may employ a memory scrubbing mechanism: reading the entire memory area, correcting any bit flips using ECC, and writing the corrected data back to the original memory location. Patrol scrubbing refers to memory scrubbing performed by the MC on a time basis (typically, the MC is configured to perform patrol scrubbing within 24

hours, keeping the impact on available memory bandwidth negligible). The MC can also be configured for manual scrubbing, where memory access is no longer time-based but controlled manually. In some next-generation memory devices, memory scrubbing is integrated with on-die ECC. In this case, the mechanism is called Error Correcting and Scrubbing (ECS). ECS is a highly significant RAS feature because, as previously mentioned, the role of memory scrubbing helps maintain a very low probability of double-bit flips within the device.

Bit Poisoning

In correlation with the Patrol Scrubbing mechanism, fault management can activate fault isolation techniques. During memory verification, ECC may detect a UCE. In this case, rather than generating a UCE, the memory can be flagged as corrupted using a RAS feature called bit poisoning. Instead of raising an exception and causing a system crash, CM marks the memory array as corrupted. The multiprocessor then monitors the presence of bit poisoning in the memory areas used by applications. The UCE does not immediately generate an exception if no software tries to use the corrupted memory, and firmware raises a UCR exception only if the corrupted memory pertains to a user-space application that the OS can terminate without broader system impact.

Memory Error Flow

Figure 8.4 shows the memory error flow across the different system levels. It illustrates the five cornerstones of fault management: detection, localization, isolation, recovery, and prediction. The RAS features implement the cycle of detection, localization, isolation, and recovery functions directly within the hardware. Error information, together with correction results, is reported to the firmware, which handles CE and UCE. The firmware may apply its own detection policies, for instance, distinguishing UCE from UCR based on the criticality of the affected memory areas for the system. A memory test action, such as a write-back operation, may enrich the error information to distinguish between soft errors and permanent ones. Middleware components, such as the operating system and platform services, perform detection by consolidating error statistics, because an excessively high frequency of CE or UCR over time

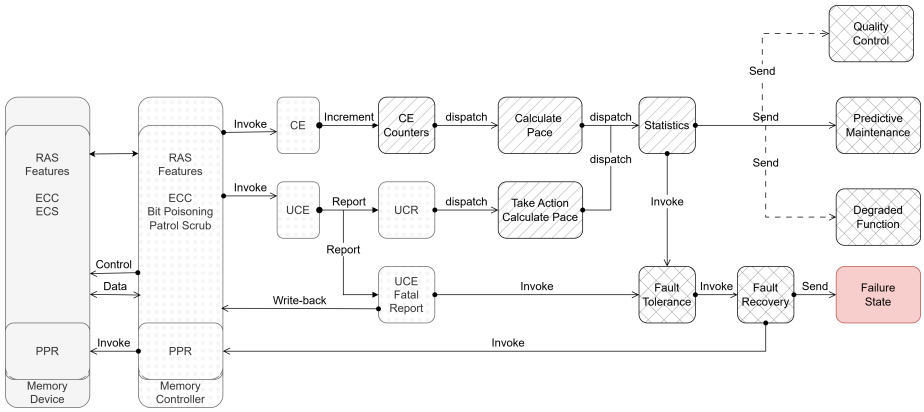


Figure 8.4. The Memory Error Information Flow.

may indicate an aging issue or, more generally, low product quality. Error and error-statistics information are then utilized by the FM, as a system function, to take appropriate recovery actions. For transient errors, the recovery action may involve reconfiguration of the component, such as adjusting memory access settings. For permanent errors, recovery may employ Post-Package Repair (PPR), a RAS feature originally intended to increase production yield. PPR consists of a limited number of redundant physical memory entries that can replace memory arrays affected by permanent errors. Hardware and firmware may also implement low-level actions to enable PPR mechanisms to operate at runtime. Utilizing PPR is an excellent example of an action directed toward lower layers as a result of a recovery operation defined and managed at higher layers. Suppose redundant memory resources are unavailable or exhausted. In that case, the system can interact with the Degraded Function feature to isolate the faulty memory array (through device fault isolation mechanisms) and allow the system to continue operating, albeit with reduced performance (if the isolated memory segment plays a significant role in data traffic management). Predictive maintenance is an action enabled by fault prediction. Through temporal analysis of error statistics, it becomes possible to identify typical error-probability growth patterns caused by aging phenomena and plan maintenance actions before the error probability leads to more severe system disturbances.

IX

Chapter 9

Fault Prediction

Let us reconsider the main functions that characterize FM, as previously illustrated in Figure 4.2. Chapter 8.1 states that fault prediction operates in parallel with the five classical pillars of FM. Its purpose is to directly identify, from the system's behavior, any critical situation, thereby enabling preventive actions to avoid system disturbances altogether. Since FM is the system function responsible for detecting, locating, isolating, and correcting faults, fault prediction represents an integral component of it. Fault prediction can identify an ongoing alteration in the system or detect an evolving condition that could lead to a disturbance if left unaddressed. The key enabler of fault prediction is data-driven modeling.

Data-driven modeling is a data analysis technique that consists of identifying functional meanings within the examined data. Its objective is to achieve a deeper understanding of the data and to support the decision-making process [189]. In the search for this behavioral model, it is necessary to define the optimal topology for extracting meaningful information from system data. The fundamental categories of data-riven modeling include clustering [190], classification [191], outlier analysis [192], and pattern mining [193]. Most pattern mining approaches are unsupervised, since their primary goal is to exploit telecommunication data for fault management purposes.

Nouioua et al. conducted an extensive analysis of fault prediction within the telecommunications sector [51], particularly at the network level. This area is

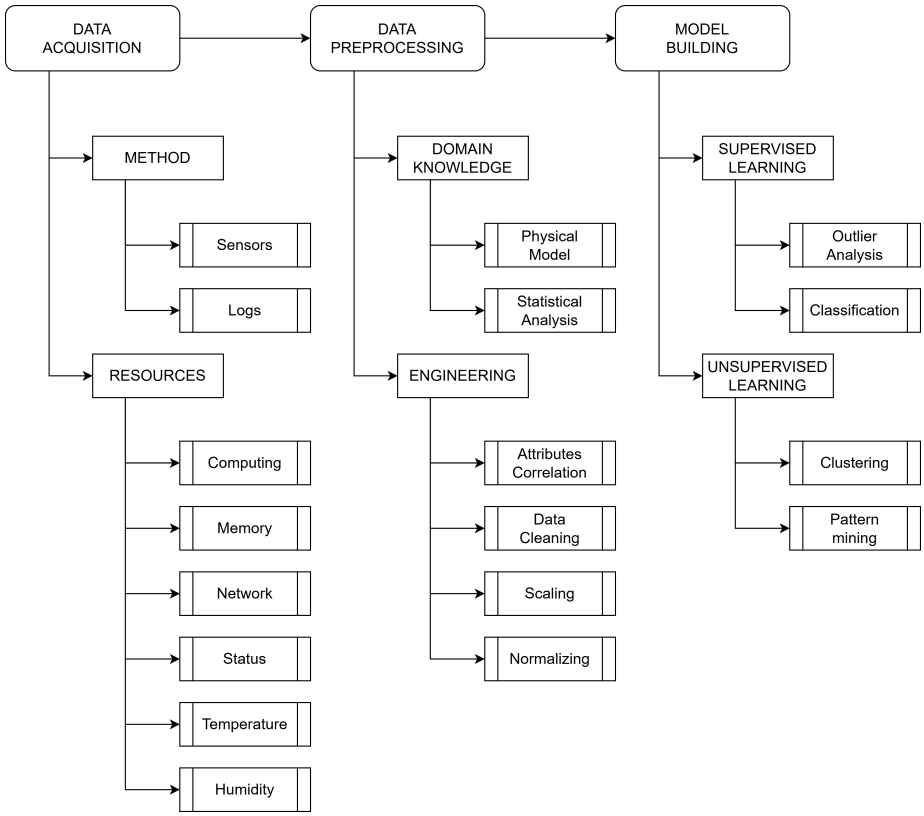


Figure 9.1. The Data-Driven Modeling Process Phases.

undoubtedly of significant research interest and continues to evolve [194, 195, 196]. The use of system log analysis for fault identification [197, 198, 199] deserves special mention. Figure 9.1 shows the three fundamental aspects characterizing any data-driven modeling process, regardless of the specific analytical approach adopted [200]:

- **Data Acquisition**: The first step is data collection. In the domain of RAN embedded systems, data acquisition represents a challenge. Storage resources are limited, and system functions are sensitive to disturbances

such as data reading and writing processes. It is therefore crucial to determine which data to collect, when to collect them, and how to collect them. For FM, three categories of data are particularly relevant: resource utilization, environmental operating statistics, and error logs.

- **Data Preprocessing**: Data quality is of paramount importance. No ML algorithm can achieve high-accuracy results without meaningful data. Consequently, the second step in any investigation must be data analysis, which involves identifying the relevant data to consider, defining validity ranges, and determining the appropriate attribute set for the specific function under examination. It is essential to emphasize that this discriminative process heavily relies on domain expertise. Most researchers say that the application of AI requires three skill sets: programming, mathematics, and domain knowledge [201, 202, 203, 204]. Among these, domain expertise plays a central and crucial role in the success of the entire AI/ML process. System expertise enables the extraction of features through variable combinations and enhances data value through knowledge of the physical model of the embedded system.

- **Model Building**: The final aspect corresponds to the specific type of data mining previously discussed. There are two learning phases associated with different models: supervised learning (including outlier analysis and classification) and unsupervised learning (such as clustering and pattern mining).

The conceptual framework and methodological process described above provide the foundation for implementing fault prediction in embedded environments. However, to prove the actual feasibility and effectiveness of the proposed approach, it is necessary to validate it through concrete applications. The following section, therefore, presents a representative case study addressing distinct aspects of deploying machine learning–based fault prediction in embedded systems. The case shows how the data-driven modeling process, comprising data acquisition, preprocessing, and model building, can work in a specific context, highlighting both the practical challenges and the benefits achieved in real-world scenarios.

9.1 Outlier Analysis Based on System Fingerprint

An embedded system can be defined as an integrated ensemble of hardware and software components designed to perform a specific function or to provide a particular service [205]. One of the most distinctive characteristics of such systems—and arguably the primary factor distinguishing them from general-purpose computing systems—lies in the necessity to ensure the execution of their functions within strict temporal constraints [206]. The microprocessor constitutes the computational core of the embedded system. In complex architectures, it is often implemented as a multi-core SoC, responsible for supervising the execution of the intended services. Depending on the functional specifications, the processor may host either a Real-Time Operating System (RTOS) or a suitably adapted general-purpose operating system designed to guarantee specific temporal execution properties. Within the context of RAN, the BaseBand Unit (BBU) represents a critical embedded system, responsible for performing signal encoding and decoding, modulation and demodulation, as well as other digital processing tasks essential for managing the data streams transmitted to and received from the Remote Radio Heads (RRH) [207]. The BBU constitutes the logical element dedicated to implementing Radio Access Technologies (RAT), such as GSM, UMTS, LTE, and 5G, which define the modalities of access to the wireless network [208, 209, 210, 211]. Generally, the BBU enables remote software updates; however, such operations are not frequent. Consequently, for a given software version, the BBU tends to operate according to largely invariant functional principles, responding consistently to external stimuli in compliance with predefined design rules. This behavioral stability enables the system to exhibit deterministic dynamics (in hard real-time systems) or quasi-deterministic dynamics (in soft real-time systems). As a result, an embedded system of this nature demonstrates an execution profile that, under identical operational and input conditions, displays a statistically stable distribution of hardware resource utilization over time.
Let

$$\Psi(R, n, T;\, t)$$

where

- $R(t) \in \mathbb{R}^m$: System Resource Vector at time t;

- $n(t) \in \mathbb{N}_0$: number of connection in runtime;
- $T(t) \in \mathbb{R}$: Environmental Temperature .

be the statistical joint distribution of the components of R, varying over time as a function of the number of users and the operating temperature. Our hypothesis is that, if the system's behavior can be characterized by a statistical function describing resource usage, then there also exists a function

$$\epsilon(R, n, T;\, t)$$

which represents the statistical deviation of the joint distribution of the resource set utilized under an error condition that constitutes an outlier. The analysis of this function allows the identification of error conditions that would otherwise remain unreported.

To validate this assumption, we analyze a memory device containing a permanent single-bit fault. We employ the patrol-scrub mechanism to induce spontaneous correctable (CE) errors. We investigate whether the presence of this memory fault can be detected solely by monitoring system-level resources. For simplicity, experiments are conducted on an embedded system maintained under constant environmental conditions and driven by randomly generated heavy traffic conditions produced by a traffic load generation program.

9.1.1 Data Acquisition

The first step in the analysis is to determine which sensors and system logs are available within the embedded platform under investigation. Since the objective is to characterize the system's usage statistics, we rely on the logs of resource utilization provided by the Linux environment. Data are collected through a Bash script that automatically performs multiple log readings every second. The quantity of interest is not the absolute value of each resource, but rather the variation in its usage relative to the previous second of observation. In this way, each set of attributes characterizes the system's behavior during the most recent second of execution, thereby enabling a temporal representation of resource utilization—precisely the type of information required for statistical characterization.

Because memory errors are not artificially injected, if we denote CE = 0 as an error-free event and CE = 1 as an error event, the proportion of entries in the dataset corresponding to CE = 1 is expected to be around 0.5%. This imbalance must be carefully considered during the data engineering phase to prevent bias in the resulting model.

9.1.2 Data Prepocessing

Not all information contained in Linux resource logs is relevant for constructing a system fingerprint in relation to memory errors. The first step, therefore, consists of an attribute analysis aimed at determining the relative importance of each parameter. The objective is not only to enhance the quality of the database under construction but also to minimize measurement overhead by restricting data collection to the minimal set of essential attributes.

To achieve such a goal, we computed the attribute correlation matrix [212] and, by means of a heatmap-based filtering mechanism [213], removed all attributes whose correlation index with the CE event was either null or close to zero. Figure 9.2 presents the resulting heatmap obtained after this attribute selection process. Table 9.1 reports the final list of selected attributes, along with their corresponding sources in terms of Linux system resources.

Once the dimensionality and typology of the most significant attributes for the characterization of the CE event have been determined, for each attribute $R_x \in R$ we calculate the marginal distribution

$\Psi\big(R_x, n_0, T_0;\, t\big) \equiv \Psi\big(R_x\big)$,

which describes the utilization of the resource under the test load conditions observed in the no-CE case, as a particular instance of the joint statistical distribution $\Psi(R, n, T;\, t)$. Similarly, we calculate the marginal distribution

$\epsilon\big(R_x, n_0, T_0;\, t\big) \equiv \epsilon\big(R_x\big)$,

representing the utilization of the resource under the traffic load test condition observed in the CE case. By comparing the two distributions, it is thus possible to verify the initial hypothesis that the CE event entails a different pattern of system resource utilization.

Let $\mu_{\mathrm{nom}}(t)$ and $\sigma_{\mathrm{nom}}(t)$ denote, respectively, the mean and the standard

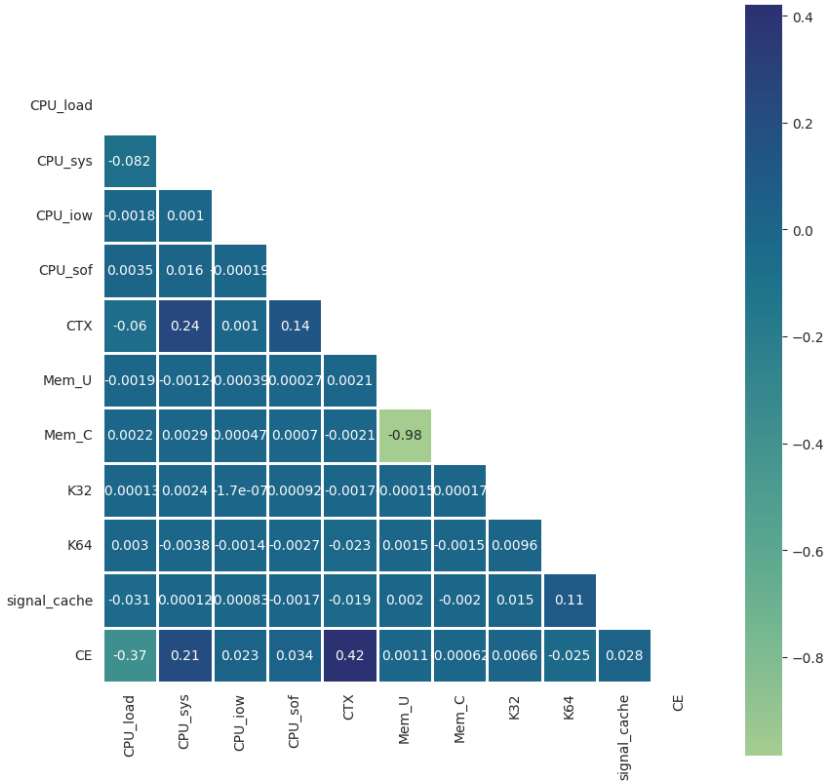


Figure 9.2. Heatmap Attributes Correlation.

deviation of the nominal behavior conditioned on n_0 and T_0. We define

$$\delta(r_x) := \frac{r_x - \mu_{\text{nom}}(t)}{\sigma_{\text{nom}}(t)}.$$

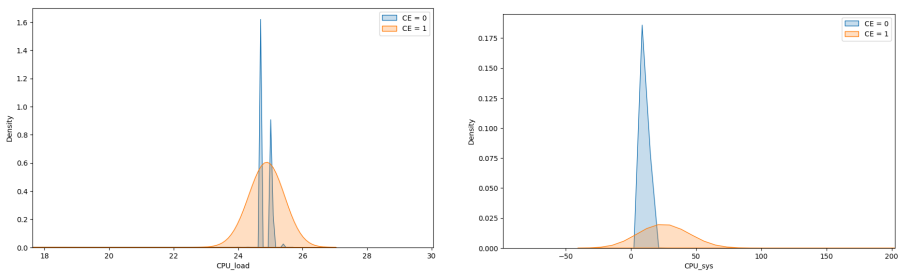
which provides a standardized measure of the deviation of the observed resource utilization from the nominal regime.

Figure 9.3 shows the distribution comparison for two attributes in the dataset: CPU average load and the kernel execution time.

Table 9.2 summarizes the statistical characteristics of the distributions under conditions with and without CE for the features most influential to the ML

Name	Source	Meaning
CPU		
cpu_load	Top command	CPU Load, average
CPU_sys	/proc/stat file	per core `jiffies` time in kernel execution
CPU_iow	/proc/stat file	`jiffies` time waiting for I/O completion
CPU_sof	/proc/stat file	the amount of `jiffies` servicing softirq
CTX	/proc/stat file	total number of context switches across all CPUs
MEMORY		
mem_u	free Command	Memory used in MB
mem_c	free Command	Memory available per process memory in MB
KERNEL OBJECTS		
kmalloc_32	/proc/slabinfo	Memory buffer for kernel 32bytes
kmalloc_64	/proc/slabinfo	Memory buffer for kernel 64bytes
signal_cache	/proc/slabinfo	Memory used for kernel signalling

Table 9.1. List of Attributes for Outlier Analysis Detection (Jiffy is the Linux Kernel System Clock Time Unit. 10ms in Our System).



(a) CPU load Distribution Comparison.

(b) Kernel Execution Time Comparison.

Figure 9.3. Distribution Comparison Between No-CE and CE for Average CPU Load and Kernel Execution Time.

model.

The correlation matrix of the attributes represents a highly valuable tool for validating any statistical treatment applied to the database under examination. Specifically, owing to the automatic collection of attribute values every second, the dataset used as the starting point for this outlier analysis comprises 738000 samples. Among these, as largely expected, only 4200 are significant with

respect to the CE condition. To prevent erroneous decisions arising from the bias of the ML model, it is necessary to balance the dataset. The correctness of the applied processing is verified by assessing the distance between the correlation values of the original matrix and those of the transformed matrix. We have demonstrated that over-sampling is more suitable for minimizing the variation in the correlation index [9]. Subsequently, the dataset was normalized using the z-score method [214, 215], as this approach has been shown to improve ML model accuracy [216, 217]. Finally, 5% of anomalous values were removed for all attributes.

9.1.3 Model Building

In the literature, clustering algorithms are commonly employed for outlier analysis [218, 219, 220]. As noted in previous sessions, clustering is instrumental for detecting deviations, yet it is not efficient when the goal is to distinguish the underlying cause of an anomaly. Supervised machine learning models are better at identifying the functional relationship between statistical variation and the corresponding error source. The evaluation of supervised ML algorithms is conducted using 80% of the dataset for training and 20% for testing. The comparison relies on standard ML performance metrics. In particular:

- **Accuracy** Fraction of correctly predicted observations.

	CPU load		CPU sys		CTX		Signal cache	
	no-CE	CE	no-CE	CE	no-CE	CE	no-CE	CE
mean	24.76	22.72	11.39	52.20	7282.5	15235	-0.012	4.383
std	1.28	5.07	5.95	150.2	730.3	9578.5	7.9520	19.816
min	10.20	10.10	3.00	5.00	1944.0	1434.0	-323.0	-105.0
25%	24.90	24.60	9.00	16.00	6745.0	7387.0	-1.000	-6.000
50%	25.00	24.90	11.00	25.00	7208.0	10981	0.0000	0.0000
75%	25.10	25.00	13.00	36.00	7656.0	23384	0.0000	10.000
max	25.70	25.50	1264	1664	99323	92376	150.00	187.00

Table 9.2. ML Most Significant Attributes Statistics Measurement.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$$

- **AUC** Area Under the Curve. Quantifies the model's ability to discriminate between classes [221].
- **Recall** Proportion of actual positives correctly identified.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positive + False Negative}}$$

- **Precision** Proportion of predicted positives that are indeed correct.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positive + False Positive}}$$

- **F1** Harmonic mean of Precision and Recall, providing a balanced metric.

$$\text{F1} = 2 \cdot \frac{\text{Precisione} \cdot \text{Recall}}{\text{Precision + Recall}}$$

- **Kappa** (Cohen's Kappa) Degree of agreement between predictions and ground truth, adjusted for chance. This metric accounts for the probability of random agreement and performs well in the presence of class imbalance, where accuracy alone may be misleading [222].

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT(sec)
rf	Random Forest	0.9958	0.9987	0.9943	0.9973	0.9958	0.9916	0.9917	5.6880
et	Exrea Trees	0.9956	0.9987	0.9936	0.9976	0.9956	0.9912	0.9912	4.6090
catboost	CatBoost	0.9920	0.9983	0.9912	0.9928	0.9920	0.9840	0.9840	16.8620
knn	K Neighbors	0.9912	0.9953	0.9921	0.9903	0.9912	0.9824	0.9824	5.0490
dt	Decision Tree	0.9891	0.9891	0.9878	0.9903	0.9890	0.9781	0.9872	2.5540
lightgbm	Light Gradient Boosting Machine	0.9760	0.9973	0.9626	0.9890	0.9756	0.9519	0.9522	3.0330
gbc	Gradient Boosting	0.9534	0.9823	0.9253	0.9803	0.9520	0.9068	0.9082	6.1580
ada	Ada Boost	0.9464	0.9792	0.9208	0.9705	0.9450	0.8927	0.8939	03.7590
svm	SVM - Linear Kernel	0.8965	0.0000	0.8332	0.9540	0.8895	0.7929	0.7994	2,7690
lr	Logistic Regression	0.8954	0.9417	0.8476	0.9372	0.8902	0.7908	0.7945	3,3670

Table 9.3. ML Model Performance Measurement.

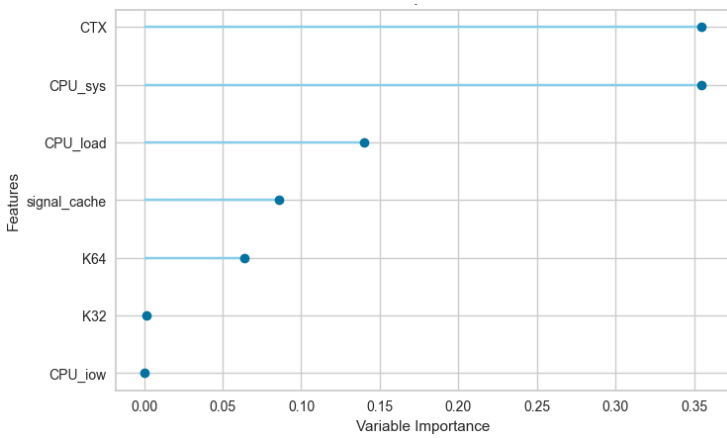


Figure 9.4. Feature Importance Plot.

- **MCC** (Matthews Correlation Coefficient) Balanced measure for binary and multiclass classification. This metric is also robust in scenarios with imbalanced classes [223].

- **TT(sec)** Training time in seconds. Lower values indicate faster model training.

Once the best-performing ML model has been identified, it is applied to a new dataset consisting of samples not used in either training or testing. Denoting this dataset as *unknown*, it is worth noting that, beyond the high performance values reported in Table 9.3, the model's accuracy on the *unknown* dataset tends to increase with the size of the training dataset.

Figure 9.4 highlights the attributes that provide the greatest contribution to the predictive performance of the Random Forest classifier. Based on these results, it is possible to derive a physical interpretation of the outlier analysis. In particular, the number of context switches and the computational load in kernel space emerge as the predominant variables in the classification of CE events. This outcome aligns with the expected system behavior: when a CE occurs, the memory controller issues a system interrupt to the microcode of the memory subsystem (firmware) to update the corresponding error counters. The handling

of this interrupt, together with the transition from user space to firmware within the execution flow, leads to an increase in kernel-space computational activity. Furthermore, the interrupt temporarily suspends the execution of applications in the user space, resulting in a higher number of context switches. This mechanism accounts for the observed increase in both the mean value and the variance of their statistical distribution.

X

Chapter 10

Fault Management Hardware Requirements

Chapter 7 highlighted how the complexity of the FM cost system can be framed in terms of a set of requirements that operate as structuring constraints within the decision-making process, thereby enabling convergence toward optimal solutions. Chapter 8 further illustrated the decisive role of hardware as the first layer involved in FM mechanisms, not only during the detection phase but throughout the execution of the discipline's five pillars, with the overarching goal of minimizing recovery time. This chapter undertakes a systematic analysis of how hardware design conditions the effective implementation of the proposed FM model. The aim is to identify the hardware features and technical characteristics that emerge as indispensable design requirements. The following sections of this chapter present comprehensive analyses and derived conclusions regarding the impacts of FM on the hardware design of the subsystems within a reference embedded system. Specifically, each analytical section concludes with a summary table enumerating the design requirements pertinent to the examined board or subsystem. This summary table includes: **Slogan**: A concise phrase encapsulating the essence of the required design constraint or guideline. **Description**: A detailed narrative specifying the expected design requirement, outlining its purpose and implementation context. **Domain**: The FM functional area (as defined in Chapter 8.1) that benefits from compliance with the requirement.

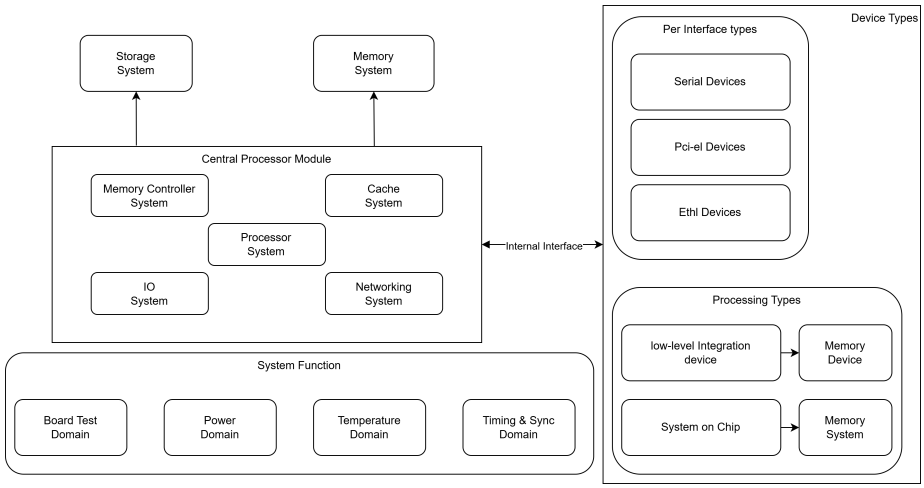


Figure 10.1. The DU Hardware Reference.

10.1 Hardware Reference

RAN embedded systems constitute the reference domain. Assessing the impact of FM on hardware design requires, as a preliminary step, identifying the reference architecture. Figure 10.1 depicts the structure of the DU, which is considered the baseline hardware platform for the system. It comprises the following subsystems:

- **Central Processor Module (CPM):** The control subsystem of the board, responsible for coordinating all other functional blocks. It is typically implemented as an SoC and incorporates:
  - one or more multicore processing units;
  - a complete memory hierarchy, from first-level cache to system memory controllers;
  - an interconnection system for peripheral access;
  - one or more network interfaces.

- **Memory System:** The working memory subsystem, consisting of one or more volatile memory devices.
- **Storage System:** The permanent storage subsystem, typically implemented using one or more mass-storage devices.
- **Board Test Domain:** A subsystem dedicated to verification and operational control functionalities for the processor.
- **Power Domain:** A subsystem responsible for managing, regulating, and distributing power within the DU.
- **Temperature Domain:** A subsystem for thermal monitoring and control, based on one or more sensors located at critical points of the board.
- **Time & Sync Domain:** A subsystem dedicated to generating and distributing timing references within the DU.
- **Serial Interface:** Device interfaces based on serial buses such as I2C, I3C, USB, or UART.
- **PCI-e Interface:** Device interfaces based on PCI-Express technology.
- **Ethernet Interface:** Interfaces based on Ethernet technology, either internal or external to the board.
- **Radio Interface:** External radio interfaces, such as CPRI, eCPRI, or Wi-Fi.
- **Low-level Integration Subsystem Device:** Low-integration devices (processors, ASICs, or FPGAs) equipped with dedicated local memory, without advanced hierarchical levels or caching mechanisms.
- **Secondary SoC Device:** High-integration SoC devices comprising multiple subsystems dedicated to the deployment of specific services, generally featuring a dedicated memory system.

It is worth noting that, while FM directly supervises internal interfaces, its involvement with external interfaces is limited to ensuring the correct operation of the line termination, without intervening in functional domains beyond the system's physical boundary.

10.2 Functional Requirements

Let us return to the three dimensions through which we define the RAS requirements in Chapter 4.2.4. These dimensions serve as guiding principles for establishing the technical and functional characteristics required of the DU components: *Self-Describing*, *Fault Information*, and *Contextualization & Recovery*. These three dimensions of RAS directly translate into the components' functional features.

- **Hardware fault detection:** every component must recognize its own state, distinguishing between normal operation and fault conditions. The device must support the dynamic propagation of hardware error information to upper system layers, such as firmware and the operating system. Dynamic propagation requires that notification pathways are fully configurable and capable of operating under both polling-based and signaling-based schemes. In particular, the architecture must expose error information through readable logs while allowing upper layers to configure dedicated control registers that manage the operational modes of the error-reporting subsystem. Upper layers can, if needed, explicitly configure actions such as clearing or masking unread error events, or set information logs to work as read&clean. The architecture shall support both interrupt-driven and event-driven propagation. Interrupt-driven copes with low-latency reporting and is therefore suitable for error classes whose prompt handling is essential for service continuity. However, interrupt-based mechanisms may incur non-negligible processing overhead and can exacerbate interrupt storms when error rates increase [224]. By contrast, event-driven (polling-based) approaches offer greater control over when and how upper layers consume error information. Therefore, the hardware must expose configuration interfaces that allow upper layers to

dynamically select the preferred notification mode for each supervised fault, enabling a reporting system that meets performance, safety, and reliability objectives. Recent research compares the efficiency of adaptive error-handling architectures with static propagation policies [225]. To support such adaptive behavior, the device must provide appropriate mask, status, and interrupt registers through which upper layers can tailor propagation rules for each error event. An important consequence of this requirement is that each component must include mechanisms to verify its correct operation. The *self-detection* concept leads to the *self-testing*. There are three types of tests that a component must support: *Power-on*, *Continuous*, and *Initiated* test.

  - *Power-on Test:* a structural and functional test to be executed automatically (if the FM configures the component to do so) at every power-on. The Power-on test aims to ensure the highest possible fault coverage, as there is no need to prevent potential disturbances to the component's functionality during the Power-on phase.
  - *Continuous Test:* real-time functional testing and verification. The continuous test must in no way affect the component's performance. Executed at runtime by the component, the continuous test must identify any alteration that may compromise service quality and level.
  - *Initiated Test:* a diagnostic test of the component or of a module thereof. FM can explicitly request an initiated test as a recovery mechanism for a previously reported fault condition. The fundamental difference from the Power-on Test is that FM may exclusively request a test for the specific component's module previously reported as faulty.
  - *Error injection:* it must be possible to verify that the component is indeed capable of detecting fault conditions. The verification of fault detection is possible only if the hardware provides an error injection mechanism upstream of the detection mechanism [226, 227].

- **Hardware fault localization:** every component must be capable of reporting the fault condition with a high level of granularity in the associated

fault information. Granularity must be a function of the component's modularity and must correspond to the smallest part of the component that is in a faulty state.

- **Hardware fault isolation:** every component must be designed as a set of modules that can be isolated in the event of a fault, thereby preventing error propagation. This requirement is strictly related to the granularity of the fault information detected during fault localization. The relevance of modularity emerges clearly when assessing the cost of redundancy. As discussed in the analysis of system memory in Chapter 8.2.1, memory modularity at the array level enables using PPR as a redundancy mechanism, rather than requiring an entire memory device as a spare unit. Fault isolation and modular component design thus enable removing a module in a fault condition, substituting it with a spare module, or continuing operation under degraded conditions when feasible.

- **Hardware Fault Recovery:** it must be evident at every level that the central objective of FM is the remediation of fault conditions. Recovery is the endpoint of every FM flow, as illustrated in Figure 8.1. Hardware is no exception and must incorporate recovery mechanisms capable of correcting a fault condition. Mechanisms include actions performed autonomously by the component once a fault condition has been detected (e.g., SECDED for memory devices), actions executed under the control of higher layers (e.g., the PPR mechanism managed by firmware), and system-level coordinated actions for contextualizing operating conditions (e.g., removing an entire memory array to maintain operation under degraded function [228, 229], or reconfiguring read/write training parameters in the memory interface to adapt dynamically to different environmental conditions [230]).

- **Hardware Identification:** the quality of fault information requires that it must always be possible to identify the component exhibiting the fault condition uniquely. Safe identification is feasible only if each component is associated with a unique identifier. Unique identification also enables verification of DU integrity, since it is always possible to confirm that the components in use are indeed the expected ones. The Identificator (ID) is a hardware feature shared with DFT, security, and DU observability [231].

- **Hardware Fault Prediction:** it is necessary to emphasize that the component must be designed to provide functional and usage statistics. As discussed in Chapter 9, the effectiveness of any fault prediction mechanism depends on the high quality of the data collected. It is therefore essential that, during hardware design, resource-usage counters, state-change counters, and fault counters be made available for use in fault prediction [232, 233].

Functional Requirement Summary

Slogan	Description	Domain
Self-State Awareness; Fault Detection	Component must detect deviations from working operation and expose error information upward through configurable propagation pathways (interrupt-driven and event-driven).	Self-Describing; Fault Information
Dynamic Error Propagation Configuration	FM upper layers must be able to select notification mode (interrupt/polling), and configure control registers, mask/clear unread events, and program read-and-clean registers.	Fault Information; Contextualization; Recovery
Power-On Test	Component must execute a structural and functional test at power-up if configured by the FM to execute them.	Self-Describing
Continuous Test	Component must run non-intrusive real-time self-tests without affecting performance.	Self-Describing
Initiated Test	FM must be able to trigger targeted module-level diagnostic tests after a reported fault.	Contextualization; Recovery
Error Injection Support	Hardware must provide mechanisms to inject faults upstream of detection logic to validate detection coverage.	Self-Describing; Fault Information
Fault Localization Granularity	Component must report fault conditions at the smallest addressable HW component's macro.	Fault Information
Fault Isolation	Component must support module-level isolation to prevent propagation and enable containment.	Contextualization; Recovery

Continues in the next page

Slogan	Description	Domain
Hardware Fault Recovery	Component must implement autonomous and coordinated recovery actions (e.g., SECDED, PPR, degraded-mode reconfiguration).	Contextualization; Recovery
Hardware Identification	Each component must expose a unique identifier to ensure unambiguous identification and integrity validation.	Self-Describing
Fault Prediction Data Provisioning	Component must generate and expose usage, state-change, and fault counters for predictive modeling.	Fault Information

Table 10.1. HW Design FM-driven Functional Requirements.

10.3 Board Requirements

This section defines the FM-driven requirements for the board design, with focus on the test domain, thermal sensing and supervision, timing and synchronization, and power-distribution integrity.

10.3.1 Board Test Domain

The board must support a hierarchical, fully traceable power-on test flow in which each executing entity contributes to the verification chain and stores its own state before control is transferred to the next element. The mechanism is orchestrated by the Power Management (PM) subsystem, which records the Master Test Controller (MTC) state.

It is worth mentioning that the MTC is a function, the test control function. If it is an actual dedicated device or a macro embedded in another subsystem, it is part of the hardware design. The MTC, in turn, stores the CPM status before granting execution. All other subsystems on the board will participate in the flow and provide their status.

It is valuable that their state can be preserved across power cycles. To enable this hierarchical sequencing, each device in the supervision chain must be equipped with non-volatile storage that retains state across power interruptions. General hardware supervision functions run at runtime on Layer-1 (physical

layer) and on Layer-2 (transport layer) interconnects and components within the unit boundary.

They encompass voltage supervision, memory, and internal-link monitoring, status indicators, clock-signal quality detectors, and hardware error-detection codes. Logged events must be consistently forwarded to the processor for recording and processing, including fault recovery and device restarts.

Temperature supervision is intentionally excluded from this group, as thermal excursions are considered environmental faults, not hardware faults, and must suppress all secondary indications until they return within valid ranges. Collectively, these mechanisms ensure that fault conditions are consistently prioritized and that power-on execution always follows a testable, secure, and traceable chain of control.

10.3.2 Temperature and Temperature Sensor

Temperature is the dominant environmental factor affecting hardware behavior. Thermal excursions can alter timing characteristics, affect power dissipation, and disrupt voltage distribution. For this reason, temperature problems are treated as primary faults and suppress all secondary indications. The temperature is sampled periodically (e.g., every 10 seconds), and no hardware alarm may be triggered until a new temperature evaluation is complete. As mentioned already, a thermal excursion is not a hardware fault. Still, its supervision is used to assess the consistency of hardware fault indications across devices and to autonomously shut down the board if a thermal excursion could permanently compromise the devices' status. Temperature sensor supervision is required to ensure that thermal measurements themselves are trustworthy. Faulty sensors are detected by evaluating measurement plausibility and comparing each measurement to the sensor's median. The maximum allowed delta among sensors must be determined through thermal simulation and stored as a board reference parameter. These requirements apply to board-level sensors, which must be significantly more reliable for system-level decision-making than device-integrated sensors and must therefore be treated as critical components for fault localization. Still, they also provide data for environmental monitoring.

10.3.3 Clock and Synchronization

Clock-domain integrity is essential for synchronization, frequency stability, and reliable operation of radio and Ethernet subsystems. The synchronization architecture consists of incoming external references connected to the Clock & Sync subsystem and redistributed to all devices (Transport Network - TN). The resulting phase error controls two digital-controlled oscillators (DCOs), corresponding to the radio base station (RBS) and Ethernet (ETH) domains, and is fed back to the clock & Sync subsystem. Each DCO includes a high-precision Oven Controller Crystal Oscillator (OCXO) and dual Digital Phase-Locked Loop (DPLL) within the clock & Sync subsystem. Clock domains (TN, ETH, RBS) are therefore in synchronization with these DCOs. Hardware supervision is limited to frequency-based references. Supervision points include incoming references, distributed clocks where observable, and internal indicators. OCXO temperature readings must be available at runtime and are required for statistical analysis and alarm-threshold detection (though it must be possible to distinguish between missing supervision point readings and critical values). Requirements for time and synchronization functions include:

- The working temperature of the reference crystal oscillator must be accessible at runtime.
- The OCXO must expose a readable identifier.
- The clock-multiplier status must be interrupt-supervised.
- All clock domains must be supervised for missing or unstable references.
- Missing or unstable clock references must generate interrupts, as recovery must occur rapidly.

Clock-signal quality detectors must be capable of generating faults that trigger logging and device or board restart. Timer-error indications also require interrupt-based signalling due to the need for fast corrective action. Current synchronization subsystems rely on a single OCXO or MEMS resonator. While MEMS resonators reduce size and improve temperature-dependent stability,

single-point-of-failure concerns and limitations in fault localization persist. Multiple external synchronization sources can mitigate these issues, and redundancy among them significantly improves diagnostic capability and availability.

10.3.4 Power Distribution

Power integrity is the foundational prerequisite for trustworthy fault supervision. Power-related errors are treated as primary faults whenever the temperature is within limits. A faulty power domain invalidates all subsequent diagnostics because cascading effects can propagate across the system. Emergency shutdown mechanisms must be available to protect the unit from thermal or overcurrent conditions. In typical scenarios, excessive temperature triggers a shutdown, even when the immediate fault is overcurrent.

- The design must incorporate a dedicated Power Module that supervises the primary (incoming) and secondary (internal distributed) power-distribution sections and supports its own internal functionality. The PM must verify stability, safety thresholds, transient behavior, and overall integrity of the input supply, even though its operation inherently depends on that supply. Passive or low-power sensing circuits must be included to detect critical conditions during degraded states.
- The PM must support autonomous self-test mechanisms for both digital and analog domains.
- The primary section includes input filtering and stabilization. Although full redundancy is not feasible under current technological constraints, modularity and granularity must be maximized to enable selective testing and isolation of critical components.
- The secondary section distributes power to board-level subsystems via digital power converters equipped with integrated telemetry. Each supply must support independent activation or deactivation, localized diagnostic sampling, and secure reporting of device-level power states. This enables the PM to provide a complete, authenticated description of the system's power status, including self-test results and verified fault conditions.

- Voltage supervision must implement both warning and fault thresholds with time-based filtering. Warnings are logged without further action, whereas fault conditions trigger domain-specific recovery procedures, such as disabling secondary rails, cycling the domain, or using fully integrated voltage regulators when available.

Board Requirement Summary

Slogan	Description	Domain
Hierarchical Power-On Test Chain	The board must implement a fully traceable, hierarchical power-on verification flow in which each executing entity stores its execution state before transferring control to the next stage.	Self-Describing
Runtime Hardware Supervision	Continuous supervision must be provided for voltage, memory, internal links, status indicators, clock-signal quality, and error-detection codes.	Fault Detection; Fault Information
Thermal-Fault Suppression Logic	Temperature excursions are treated as a primary environmental critical condition that suppresses all secondary hardware-fault indications until thermal conditions stabilize.	Contextualization
High-Integrity Temperature Sensing	Temperature sensors must be supervised through plausibility checks and median-based comparison across sensors. Board-level sensors are critical and must exhibit significantly higher reliability than device-integrated sensors.	Contextualization;
Trustworthy Thermal-Driven Shutdown	The board must autonomously shut down in case a thermal excursion risks permanent hardware degradation.	Fault Recovery
Clock-Domain Supervision	All clock domains (TN, ETH, RBS) must be monitored for missing or unstable references. Clock-multiplier status must be interrupt-supervised, and missing or unstable references must generate interrupts to enable rapid recovery.	Fault Detection

Continues in the next page

Slogan	Description	Domain
OCXO Monitoring and Identification	The working temperature of the reference OCXO must be available at runtime, and the OCXO must expose a readable identifier. Supervisory mechanisms must distinguish between missing readings and critical values.	Self-Describing
Clock-Signal Quality Fault Reporting	Timer-error conditions require interrupt-based signaling to support fast corrective measures.	Fault Recovery
Synch Source Redundancy	Redundant external synchronization sources must be supported to mitigate single-point-of-failure risks associated with OCXO resonators and to enhance diagnostic capacity and availability.	Fault Localization
Power-Domain Integrity Assurance	Power integrity must be treated as a primary condition for valid diagnostics. Faulty power domains invalidate all downstream supervision.	Contextualization
Autonomous Power-Module Supervision	The PM must supervise the primary and secondary power distribution sections, verifying stability, thresholds, transients, and overall supply integrity.	Fault Information
PM Self-Test Capability	The Power Module must support autonomous self-tests of both digital and analog domains to validate integrity during initialization and runtime.	Self-Describing
Modular Power-Input Architecture	The primary power section must include filtering and stabilization. Although full redundancy is not feasible, modularity and granularity must enable selective testing and isolation of critical elements.	Fault Recovery
Telemetry-Enabled Secondary Power Rails	Secondary power rails must be distributed by digital power converters with telemetry support. Each rail must permit independent control, localized diagnostic sampling, and secure reporting of device-level.	Fault Detection; Fault Information

Continues in the next page

Slogan	Description	Domain
Voltage Supervision	Voltage supervision must implement warning and fault thresholds with temporal filtering. Warnings are logged, while fault thresholds initiate domain-specific recovery actions, such as selective rail shutdown or cycling.	Fault Recovery

Table 10.2. HW Design FM-driven Board-level Requirements.

10.4 Component Requirements

This section defines FM-driven requirements for board subsystems, referred to as reference hardware in section 10.1 and reported in Figure 10.1.

10.4.1 Central Processor Module

The CPM functions as the core of the embedded system and is the central element of the FM at the system level. It operates as the primary control and coordination component, responsible for deploying the FM across all board subsystems. This central role encompasses all functions and services, making the required reliability level particularly critical. An FM-driven design for the CPM is best defined by referring to the discussion in Section 10.2, which clarifies the requirements for a complex object such as the CPM to meet RAS standards. These requirements specifically address the reliability of three fundamental functionalities: computation, memory, and communication. Therefore, it is unacceptable for the CPM to continue operating when a silent fault [234] affects internal memory devices, cores, or connection endpoints such as Ethernet, PCI-E, and serial interfaces. RAS compliance requires that the hardware detect errors and report them dynamically to higher system layers, and isolate error conditions to prevent their propagation. Recovery mechanisms must enable continued system operation even in the presence of permanent hardware faults.

Error-Information Propagation

The CPM must implement a comprehensive, flexible, and integrated architecture for managing and propagating error information within existing operating sys-

tem ecosystems. The Machine Check Architecture (MCA) [235, 236] serves as a well-known example. All error notifications must be delivered to the operating system in accordance with recognized standards, covering errors originating both within the CPM and from connected subsystems. The error-propagation architecture must automatically include the error source in each notification, with each error source assigned to a dedicated system register as appropriate. Error reporting must provide the highest possible level of detail, including both the source and precise localization information when available. Error information must be linked to a logging system that persists across CPM restarts and conforms to common Linux standards. The logging system must be sufficiently robust to retain information even in the event of multiple cascading or nested errors. The CPM must provide mechanisms for activation and deactivation, enabling higher system layers, such as firmware and software, to dynamically select which events trigger interrupts. Filtering mechanisms must be supported to trigger interrupts when the number of errors within a specified time window exceeds a defined threshold. For instance, if three Corrected Errors (CEs) are detected within 5 milliseconds on Core 2, this should trigger only a cluster-level interrupt. Error propagation must follow a hierarchical structure, requiring interrupt controllers at both the SoC and per-core levels to ensure a configurable interrupt chain. This configuration allows for interrupt handling at either the component or single-core level. The CPM must also enable higher system layers to access polling events, necessitating that the CPM and its peripherals be accessible through state registers. Corrected error propagation must not cause system disturbances. Supervision should rely on CE counters, which must be accessible via polling or interrupts, and the system must be able to configure which cores handle CE when interrupts are required.

Computing Function

Each multicore cluster within the CPM must report status and detect errors for each core using a dedicated register as part of the error-propagation architecture. The CPM must incorporate plug-in and plug-out mechanisms for every core to support rapid fault isolation and maintain operation with a reduced number of cores. The degraded function condition is responsible for managing the updated system-level configuration. The CPM must incorporate Error Detection and Cor-

rection (EDAC) mechanisms for its cores [237]. Structural and logical tests on cores must be supported [238, 239, 240], with these tests automatically triggering core-correction actions, including internal-array reconfiguration. Core-array correction provides minimal physical redundancy, which may be complemented by core-level redundancy mechanisms. Each cluster should include spare cores to facilitate verification, production, runtime, and recovery, serving as fault recovery methods if core-array correction is insufficient. If core-correction is unsuccessful, the CPM must employ a resilient-boot system [241]. During the boot process, any core identified as faulty through logical testing must be disabled, and the state of each core—active or disabled—must be reported to the degraded function 8. To enhance fault coverage, the CPM should implement mechanisms for cross-checking execution across cores, such as Lockstep [242, 243]. Voltage and timing distribution systems must be continuously monitored for each core, as these are considered primary error sources. This monitoring enables the system to distinguish between true core faults and faults related to timing or power delivery. The CPM utilizes system-software-type microcode [244, 245] for its primary configuration. It must be capable of detecting microcode corruption and distinguishing between physical faults and faults occurring during microcode execution, with appropriate reporting mechanisms. The architecture must support direct microcode updates in deployed products, including remote and secure update capabilities. Hardware support for core virtualization is also required.

Memory Function

All CPM memories, ranging from L0 to L3 cache [246], require protection against corruption through appropriate detection and correction mechanisms. The CPM must report detected and corrected errors (CE), uncorrected errors affecting primary operating-system functions (UCE), and uncorrected errors not affecting primary operating-system functions (UCR). This reporting enables continued system operation while upper layers manage the recovery process. For L2 or higher-level memories, at least SECDED-type ECC must be implemented. For L1 memories, which are frequently refreshed, at least parity correction is required. The CPM must include isolation mechanisms to mitigate UCE-type memory corruption, such as data poisoning. Additionally, the CPM must filter

CE events for cache lines that have already been reported to reduce redundancy in error reporting. The CPM must provide error-injection mechanisms for every cache level, memory address, and access type, serving as validation tools for fault supervision and memory testing. A memory controller subsystem is responsible for managing system memory access and must support Side-band, in-line, and link ECC 8.2.1, depending on the device type within the memory subsystem. The memory controller must provide CE and UCR error counters and, according to upper-layer settings, report external-memory errors detected by ECC through the MCA. Additionally, the memory controller must support patrol-scrubbing algorithms and the PPR mechanism for system memory. Resilient-boot must enable the system to reconfigure and remove unrecoverable memory addresses and banks following a fault, with the degraded function managing the reduced available memory. The CPM must support DDR memory training to address potential timing delays in system memory. Memory structure checks using M-BIST must be supported, and error injection through the memory controller must validate supervision mechanisms. Hardware support for system memory virtualization is also required in the CPM.

Connection Function

Consistent with the provisions on error propagation, all termination-related errors in the CPM must be accessible through the MCA via registers that implicitly indicate the error source. PCI-E terminations must support Advanced Error Reporting (AER) and Message Signaling Interrupt (MSI-X) [247]. Only UCE-type errors must generate system exceptions, whereas CE handling must have no performance impact. Ethernet terminations must include BER, FEC, and CRC error counters. The CPM's internal die-to-die communication must be protected by FEC, CRC, and retry mechanisms for error detection and correction. It must be possible to test the structural integrity of connections through BIST on JTAG and loop mechanisms for serial terminations, enabling the distinction of serial-termination faults in the CPM. The CPM must incorporate the hardware techniques necessary to support the virtualization of Ethernet terminations.

CPM Requirement Summary

Slogan	Description	Domain
Central FM Execution Integrity	The CPM shall guarantee that computation, memory, and communication operate within RAS principles.	Fault Detection, Fault Reporting, Fault Isolation
Silent-Fault Intolerance	The CPM shall not continue operation under silent faults affecting internal memory, cores, or communication endpoints, requiring immediate detection and reporting.	Fault Coverage
Dynamic Error Detection and Reporting	The CPM shall provide standardized error detection and dynamic reporting of errors discovered both internally and in connected subsystems.	Fault Detection, Fault Reporting
Error Isolation Guarantee	The CPM shall isolate hardware error conditions to prevent error propagation across the system.	Fault Isolation
Fault-Resilient Continuity	The CPM shall incorporate mechanisms that ensure continued system operation under permanent hardware faults through appropriate recovery strategies.	Fault Recovery
Integrated Error-Propagation Framework	The CPM shall implement a comprehensive and flexible error-propagation architecture consistent with established OS mechanisms, such as MCA.	Fault Information, Self-Describing
Standard-Compliant OS Notification	All CPM error notifications shall follow recognized OS standards and include source and localization information when available.	Fault Reporting, Fault Information
Persistent Robust Logging	Error information shall be stored in a persistent logging infrastructure surviving CPM restarts and supporting complex nested error scenarios.	Self-Describing
Dynamic Interrupt Configuration	The CPM shall allow higher layers to dynamically enable or disable interrupt-triggering events and configure filtering mechanisms for time-window-based thresholds.	Contextualization
Hierarchical Interrupt Architecture	Error propagation shall follow a hierarchical interrupt structure using SoC-level and per-core controllers to assure configurable interrupt paths.	Contextualization

Continues in the next page

Slogan	Description	Domain
Polling Accessibility for Error State	The CPM and peripherals shall expose state registers enabling polling of error conditions by higher layers.	Fault Reporting, Self-Describing
CE Counter Accessibility	CE counters shall be accessible by polling or interrupts.	Fault Reporting
Per-Core Error Status Reporting	Each multicore cluster shall report per-core status and error information via dedicated registers.	Fault Reporting, Self-Describing
Core Plug-In/Plug-Out Capability	The CPM shall support plug-in and plug-out mechanisms for each core to enable rapid isolation and continued operation with reduced resources.	Fault Isolation, Fault Recovery, Degraded Function
Automatic Core Correction Tests	Structural and logical tests shall automatically trigger core-correction actions, including array reconfiguration.	Fault Recovery, Self-Test
Spare-Core Redundancy	Each cluster shall include spare cores for verification, production, runtime operations, and recovery when core-array correction is insufficient.	Fault Recovery
Resilient-Boot for Faulty Cores	The CPM shall employ resilient-boot procedures that disable faulty cores during initialization and report the updated core-state configuration.	Fault Isolation, Degraded Function
Execution Cross-Checking	The CPM shall implement cross-checking mechanisms such as Lockstep to improve fault coverage.	Fault Coverage
Voltage and Timing Monitoring	Voltage and timing distribution for each core shall be continuously monitored to differentiate core faults from timing or power-related issues.	Contextualization
Microcode Integrity Protection	The CPM shall detect microcode corruption and distinguish physical faults from microcode execution faults, with appropriate reporting.	Fault Detection, Contextualization
Secure Remote Microcode Update	The CPM shall support direct, remote, and secure microcode updates in deployed systems.	Contextualization
Hardware Support for Core Virtualization	The CPM shall include hardware capabilities enabling virtualization of cores.	Contextualization

Continues in the next page

Slogan	Description	Domain
Comprehensive Memory Protection	All CPM memory levels (L0–L3) shall implement adequate detection and correction mechanisms.	Fault Detection, Fault Recovery
Memory Error Classification	The CPM shall report CE, UCE, and UCR events to enable system-level management and continued operation.	Fault Reporting, Fault Information
Minimum ECC Requirements	L2 and higher memories shall implement at least SECDED ECC, while L1 memories shall include at least parity correction.	Fault Detection
UCE Isolation through Data Poisoning	The CPM shall include mechanisms such as data poisoning to contain UCE-type corruption.	Fault Isolation
CE Redundancy Filtering	The CPM shall filter CE events for previously reported cache lines to minimize redundant reporting.	Fault Reporting
Memory Error Injection Framework	The CPM shall allow error injection at every cache level, address, and access type.	Fault Detection
Memory Controller ECC Support	The memory controller shall support Sideband, in-line, and link ECC (according to the connected memory device type) and shall report external-memory errors via MCA.	Fault Detection, Fault Reporting
Memory Error Counters	The memory controller shall expose CE and UCR counters to higher layers.	Fault Prediction Data Provisioning
Patrol-Scrubbing and PPR Support	The CPM shall support patrol-scrubbing algorithms and PPR mechanisms for system memory.	Fault Detection, Fault Recovery
Resilient-Boot for Memory Faults	Resilient-boot shall reconfigure memory to exclude unrecoverable addresses or banks, with degraded-mode tracking.	Fault Isolation, Degraded Function
DDR Training Support	The CPM shall support DDR memory training to mitigate timing-related errors.	Fault Prediction
M-BIST Memory Structure Checks	The CPM shall support M-BIST for structural memory validation.	Fault Detection, Self-Test
Memory Error Injection via Controller	The memory controller shall support error injection for supervision-validation purposes.	Fault Detection, Self-test

Continues in the next page

Slogan	Description	Domain
Memory Virtualization Support	The CPM shall provide hardware support for system-memory virtualization.	Contextualization
Termination Error Visibility through MCA	All termination-related errors shall be accessible via MCA registers that implicitly indicate the source.	Fault Reporting, Fault Information
PCI-E AER and MSI-X Compliance	PCI-E terminations shall support AER and MSI-X, ensuring UCE generates system exceptions while CE remains performance-neutral.	Fault Reporting
Ethernet Error Counters	Ethernet terminations shall provide BER, FEC, and CRC error counters.	Fault Information
Intra-die Communication Protection	Internal die-to-die communication shall include FEC, CRC, and retry mechanisms for robust error detection and correction.	Fault Detection, Self-test
Connection BIST and Loopback Testing	The CPM shall provide BIST for JTAG and loopback testing for serial terminations to isolate termination-related faults.	Fault Detection, Self-test
Ethernet Virtualization Support	The CPM shall include hardware support enabling virtualization of Ethernet terminations.	Contextualization

Table 10.3. HW Design FM-driven CPM Requirements.

10.4.2 Memory System

In chapter 8.2.1, we analyzed the possible causes of memory corruption in detail. We also examined hardware techniques to mitigate and correct them. Figure 8.4 shows the entire System memory error-handling flow. This figure serves as the motivation for defining the hardware design requirements of memory devices. Figure 10.2 is a schematic representation of the origin of data corruption. It summarizes the causes of bit-flips listed in chapter 8.2.1. A bit flip can be caused by data corruption in the device's memory array or by an error on the memory bus. Thus, the memory system design must allow for supervision and correction of bit-flips in both the memory array and the link. This requirement can be described as follows: the memory system must provide hardware detection, signaling, and correction techniques for memory arrays and the system memory

bus.

The mechanism of ECS, mandated by the DDR5 technology standard, is crucial. It allows you to detect and correct a bit flip directly on the memory device. This does not affect the operating system's execution flow directly. ECS supplies a no-impact alternative to patrol scrubbing technology. Patrol scrubbing still has minimal cost because it requires managing the SMI: in the absence of errors, there is one SMI for each patrol scrubbing cycle, which is theoretically one every 24 hours. However, ECS requires a concurrent ECC mechanism for the link supervision (MC patrol scrubbing is, instead, a complete solution).

The memory controller of the central processor is always responsible for supervising and correcting bit-flips on the system memory bus. This is possible thanks to ECC techniques cascaded with the physical bus termination. The solution to adopt depends mainly on the system's characteristics. Sideband-ECC minimizes latency impacts on information transfer. It is strongly recommended for hard or soft real-time systems. For LPDDR5 devices, the bus width does not support sideband-ECC. In this case, use link-ECC. As shown in figure 10.3, both the memory device and the memory controller of the central processor require this mechanism. It is important to note that link-ECC technology protects exclusively against bus errors and, therefore, is not interchangeable with on-die ECS. They constitute a response to the previously described requirements if implemented simultaneously. On-die ECS is a fundamental feature that ensures corrective action for a single-bit flip. Without the scrubbing algorithm (for

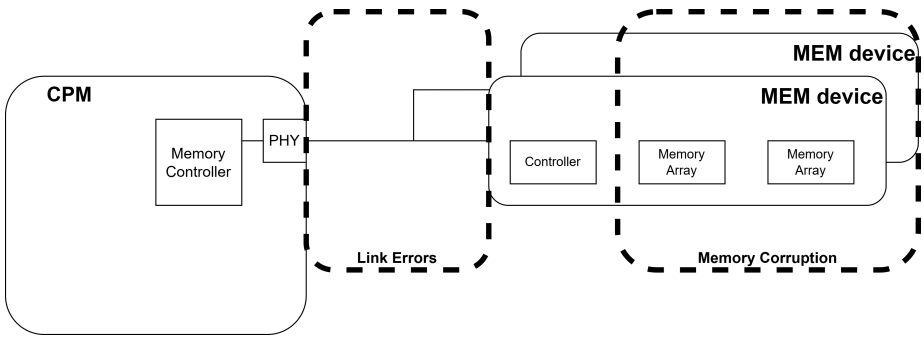


Figure 10.2. Schematic Representation of the Origin of Data Corruption.

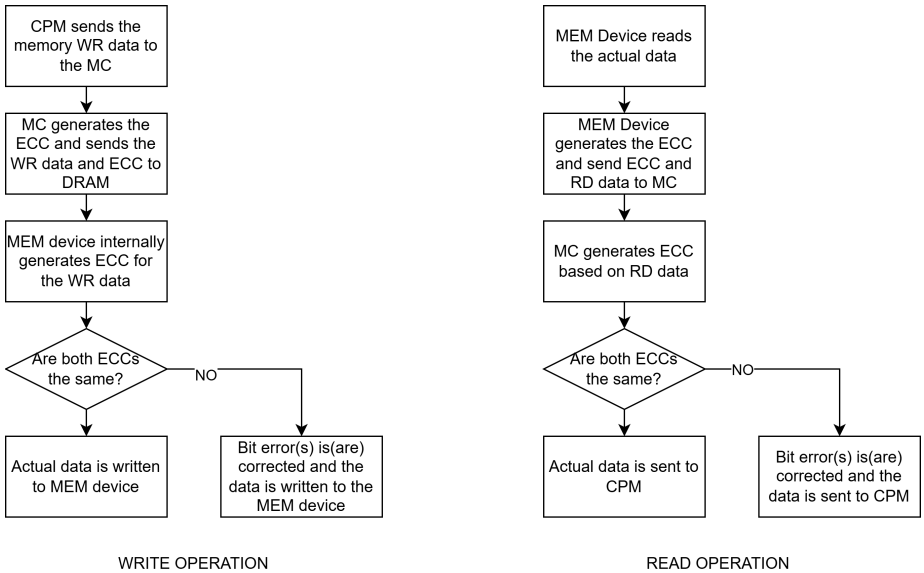


Figure 10.3. WR and RD Operation Flows with Link-ECC.

example, using only on-die ECC), a single-bit flip is not actually corrected in the memory device, increasing the likelihood of a double-bit flip error, which is an uncorrectable error.

Regardless of the adopted technical solution, the memory device must provide error information at the highest possible granularity and with maximum flexibility. The memory device must have physical redundancy in terms of PPR and be modularly designed, with a granularity equal to the available PPR resolution. It is strongly recommended that the device be able to report CE and UCE to MC on demand and interrupt-based, if requested. CE and UCE counters in the memory device are very valuable for statistics and fault prediction purposes.

Memory System Requirement Summary

Slogan	Description	Domain
End-to-End Bit-Flip Protection	The memory system shall provide hardware mechanisms for the detection, signaling, and correction of bit-flips occurring both in the memory array and on the memory bus.	Detection, Reporting, Correction
Explicit Fault Origin Differentiation	The system memory shall distinguish, at the hardware level, between bit-flips originating in the memory array and bit-flips originating on the memory bus.	localization, Isolation, Error information
Autonomous Array Scrubbing	When DDR5 ECS is implemented, the memory device shall provide autonomous memory array scrubbing capabilities.	Self-test, Detection, Correction
Controller-Based Link Supervision	The memory controller shall be responsible for detecting and correcting bit-flips occurring on the system memory bus through ECC mechanisms.	Detection, Correction, Isolation
ECC Selection Driven by System Constraints	The memory system shall support a memory bus ECC solution selected according to system characteristics, including latency sensitivity and real-time constraints.	Contextualization
Low-Latency ECC for Real-Time Systems	For systems with hard or soft real-time requirements, the memory system shall support sideband-ECC to minimize latency impact on memory data transfers.	Contextualization
Mandatory Link-ECC for LPDDR5	For LPDDR5-based systems, the memory system shall implement link-ECC for memory bus supervision.	Detection, Correction, Isolation
High-Granularity Error Reporting	The memory device shall provide error information to the memory controller with the highest feasible spatial and functional granularity.	Error information, localization, Isolation
PPR-Based Physical Redundancy	The memory device shall support physical redundancy through PPR and shall be modularly designed with a granularity matching the available PPR resolution.	Correction, Isolation, Degraded function

Continues in the next page

Slogan	Description	Domain
Flexible CE and UCE Reporting	The memory device shall be capable of reporting CE and UCE to the memory controller, both on demand and via interrupt-based mechanisms, when requested.	Reporting, Error information, Contextualization
CE and UCE Error Counters	The memory device shall maintain internal counters for CE and UCE events to support statistical analysis and fault prediction.	Prediction, Error information

Table 10.4. HW Design FM-driven Memory System Requirements.

10.4.3 Storage System

The consideration of fault-management concepts for non-volatile storage devices is a structural design constraint that affects architectural choices and metrics exposure, as well as system-level fault management and recovery strategies. In the reference domain, it is essential to distinguish between fault conditions that compromise service availability and degradation conditions that cause information loss without directly impacting service stability. This distinction is also essential for recovery management: not all fault conditions must be addressed the same way or with the same urgency. Storage refers to non-volatile memory, i.e., hard disks and flash storage. In the RAN domain, the challenge is not only to find a balance between efficiency (from a RAS perspective) and implementation costs, but also to clearly define the extent to which recovery can be managed locally by the device and when it must be orchestrated by the system.

Disk

Solid State Disks (SSD)s are the preferred technology solution over Hard Drive Disks (HDD)s because they eliminate the main causes of mechanical failure and offer superior I/O performance. However, SSDs introduce failure modes related to NAND memory physics that cannot be fully recovered through local intervention.

In modern 3D flash, charge loss in the cells leads to a rapid increase in read errors immediately after programming (early retention loss) [248], a phenomenon more pronounced in high-density architectures [249]. Furthermore,

the multilevel nature of 3D NAND architectures introduces spatial variability in the error rate, depending on the physical localization of the data. These characteristics are not occasional anomalies but intrinsic limitations of the technology, which require a fault management approach oriented toward fault limitation and containment rather than fault elimination [250]. At the device level, fault recovery is primarily preventative. The use of Low-Density Parity-Check (LDPC) codes enables bit-level error recovery within the code's correction capacity [251]; similarly, wear-leveling and bad block management mechanisms isolate and replace degraded blocks before they become unusable [252]. However, when degradation exceeds these mechanisms, for example, due to endurance exhaustion or an increase in Raw Bit Error Rate (RBER) beyond ECC thresholds, local recovery is no longer possible. For this reason, fault management cannot be limited to internal mitigation; it must explicitly enable system-level recovery. The telemetry, logging, and self-diagnostics features defined by NVMe and SMART constitute the primary interface through which the device communicates its state of degradation [253]. Indicators such as Percentage Used, Available Spare, and Media and Data Integrity Errors are not mere monitoring parameters, but signals that trigger recovery strategies, such as pre-emptive data migration, reallocation of application functions, and scheduling device replacement [254]. The NVMe SSD's health must therefore be verified both at boot time and periodically during runtime. Exceeding degradation thresholds (e.g., less than 15% remaining life) must be treated as a pre-fault condition, logged as a persistent warning, and used to initiate corrective actions at the system level. Some SSDs implement mechanisms that place the device in read-only mode when critical thresholds are reached (e.g., 5% remaining life) [255, 256]. This behavior does not represent recovery, but rather a degraded function intended to protect data. It is therefore essential that proximity to this state is signaled in advance via persistent logs so that the system can complete recovery operations before losing write capability.

Flash Storage

For NOR flash used for boot and firmware, the fault recovery paradigm is substantially different. Cryptographic validation of boot images serves as a memory-integrity supervisory mechanism, detecting both malicious modifications and spontaneous corruption due to retention failures or bit flips [257].

If validation fails, the system enters fail-stop mode, interrupting the boot process [258]. In this context, in-place recovery of the flash content is not provided. Recovery does not rely on error correction, but on an architectural redundancy strategy [259]. The introduction of redundant boot flashes and automatic switching mechanisms enables the system to recover functionality by switching to a valid alternative firmware image. This approach explicitly recognizes that some faults are not recoverable at the data level and that recovery must occur by selecting a healthy alternative resource [258].

Storage Requirement Summary

Slogan	Description	Domain
Local recovery before system impact	SSD devices shall attempt local fault recovery through error correction mechanisms, bad block remapping, and wear leveling before escalating the fault to higher system layers.	Fault Recovery, Fault Isolation
No silent exhaustion of local recovery	Exhaustion of local recovery mechanisms (e.g., error correction margin or spare block pool) shall be explicitly reported to the system before service availability is impacted.	Fault Reporting, Fault Recovery
Boot-time and run-time health verification	SSD devices shall support self-test health status verification both at boot time and periodically during run time, allowing early detection of pre-fault conditions.	Fault Detection, Self-Test
Standardized endurance and health reporting	eNVM SSD devices shall expose standardized endurance, health, and data integrity statistics through SMART and NVMe log pages, enabling system-level fault prediction, maintenance planning, and failure analysis.	Fault Reporting, Fault Information, Fault Prediction
Pre-degraded recovery window	The system shall be informed of proximity to read-only mode with sufficient advance notice to complete recovery actions before write capability is lost.	Fault Prediction, Fault Recovery

Continues in the next page

Slogan	Description	Domain
Fail-stop boot recovery model	For boot and firmware flash, fault recovery shall not rely on in-place data correction but on architectural redundancy and automatic switching to an alternative valid image.	Fault Recovery, Fault Isolation
Fail-stop detection for boot flash	Cryptographic validation of boot and firmware images shall be used as a deterministic fail-stop mechanism to detect both malicious modification and spontaneous data corruption in non-volatile boot flash.	Fault Detection, Fault Isolation, Fault Contextualization
Deterministic recovery path selection	Upon detection of a non-recoverable boot flash fault, the system shall deterministically select an alternative recovery path without attempting repeated execution from the faulty image.	Fault Recovery, Fault Isolation
Recovery traceability	All recovery actions, including transitions to degraded modes and activation of redundant resources, shall be recorded in persistent fault logs.	Fault Information, Fault Reporting

Table 10.5. HW Design FM-driven Storage Requirements.

10.4.4 Low-Processing Device

This section analyzes the contextualization of RAS requirements for a specific class of components: low-processing devices. The definition of a low-processing device can be ambiguous. This ambiguity requires clarification. This work uses low-processing-device terminology for Device Processors (DPs). A DP is a specialized computing unit, usually dedicated to specific functions. It executes bare-metal software with strict timing and determinism requirements. DPs have much simpler architectures than Central Processing Modules (CPMs), with a simpler memory hierarchy and fewer cache levels. This design reduces cost and energy use, but it also limits functionality [260]. For example, DPs lack advanced fault management mechanisms. Stringent performance requirements, limited resources, and a simple architecture make it difficult to implement complex fault management in DPs. Another key characteristic is that DPs have a subordinate role within the CPM. These two factors may lead to underestimating the value of failure management in DPs. This view is misleading. FM in DPs must not be minimized; it should be properly contextualized within the

system architecture. In a RAS-compliant system, each subsystem must assess its health and report to the higher-level control entity [261]. A DP must have at least one self-test health mechanism. This can be executed at power-on, on CPM request, or as a recovery strategy for fatal faults. The self-test is not a replacement for runtime fault detection. Instead, it complements and integrates with FM across the DP lifecycle. The DP shall detect and report any fault that can lead to persistent functional misbehavior; silent data corruption is not acceptable for the considered class of applications. All detected errors must be logged and stored in persistent memory. This adds resource overhead but allows event reconstruction even after recovery or on CPM request. Redundancy of DP processing units and associated memories is another trade-off. Adoption depends on the criticality of the DP's function and the impact of its malfunction. Because DPs are subordinate to CPMs, local redundancy is often not efficient or cost-effective. Sometimes, functional redundancy on another subsystem, such as the CPM, could be preferable [262]. How the DP reports errors to the CPM depends on the fault type and severity. The DP must support both interrupt-based and polling modes. Its status and configuration registers must be accessible by the CPM. The CPM retains ultimate responsibility for system decisions. DP memories are small and used frequently. Adopting complex EDAC schemes is not always justified. Simple error detection is often enough. These allow the DP to recover at a broader level and reduce hardware complexity. Each DP must have a unique identifier that is readable by the CPM. DPs also need temperature and power-stability sensors. Anomalies from these sensors must be sent to the CPM as separate error events. This separation avoids diagnostic confusion with functional or logical status. The DP microcode must be loadable and updatable by the CPM. This enables remote recovery for functional faults and repairs corrupted microcode. Checking microcode integrity remains the DP's job. The DP must verify its microcode using cryptographic hashing (if available for security purposes) or CRC. It must also manage the stop mechanism, request downloads at startup, and perform integrity verification periodically during runtime.

Low-Processing Devices Requirement Summary

Slogan	Description	Domain
Health Awareness	Each Device Processor shall autonomously assess its own health status and provide this assessment to the higher-level control entity, in accordance with RAS-compliant system principles.	Fault detection, reporting, contextualization
Self-Test Capability	Each Device Processor shall implement at least one self-test health mechanism, executable at power-on, upon CPM request, or as part of recovery from fatal faults.	Self-test
Runtime Detection	Device Processors shall continuously monitor for faults during operation.	Fault detection
Persistent Fault Evidence	All detected faults and errors shall be logged and stored in persistent memory to enable post-mortem analysis, event reconstruction, and CPM-initiated diagnostics, even after recovery or reset.	Fault Reporting
Redundancy	Redundancy of Device Processor computing units and local memories can be physical or functional, depending on the criticality of the low-processing function for the system and the implementation cost.	Fault Recovery, Contextualization
Fault Reporting Mode	The Device Processor shall support both interrupt-driven and polling-based error reporting mechanisms toward the CPM.	Fault Reporting
Memory Protection	For a low-processing device with limited memory, a simple error-detection mechanism is sufficient. Correction and recovery action are under the responsibility of the DP-level firmware FM agent.	Fault Detection, Fault Recovery
Identity	Each Device Processor shall expose a unique identifier readable by the CPM.	Fault information
Environmental awareness	Each Device Processor shall integrate temperature and power-stability sensors. Anomalies from these sensors shall be reported as distinct error events to avoid conflation with functional or logical faults.	Fault information, Contextualization
Recoverable Microcode	Device Processor microcode shall be loadable and updatable by the CPM.	Fault Recovery

Continues in the next page

Slogan	Description	Domain
Microcode Integrity Verification	The Device Processor shall verify microcode integrity using cryptographic hashing when available, or CRC otherwise. Integrity checks shall be performed at startup and periodically during runtime, and the DP shall autonomously manage stop, download request, and verification procedures.	Fault detection, recovery, Self-test

Table 10.6. HW Design FM-driven Low-Processing Device Requirements.

10.4.5 SoC Device

The complexity of the services to be provided drives a high level of integration in the design of electronic components. Processors, memories, analog blocks, network terminations, and many other functional blocks are designed on a single silicon die and must communicate with each other, often with significant amounts of high-speed data. The complexity of these components, which we call SoCs, is a direct consequence of the high performance and design quality required to build them. This trend continues: new products increasingly have higher performance requirements, requiring more functional blocks to be integrated, increasing complexity, and requiring a greater commitment to design efficiency. Our analysis of resilience, specifically the role of fault management, in SoC design falls within this context and is critical to the entire system [263]. A first consequence is the need for the entire design to be highly modular. Each functional block, whether a DSP, core, memory array, or communication termination, must be designed with plug-and-play capabilities usable both in production and at runtime. This ensures high fault detection granularity and fault isolation mechanisms that allow the system to recover from a degraded state [264]. A second obvious consequence is the need to consider SoC test and verification mechanisms. This means directly addressing the RAS requirement, which requires each subsystem to monitor its own health status [265]. In other words, one of the SoC's functional blocks must be the test macro, a self-test mechanism for performing structural and functional tests of the SoC. The component self-test algorithm must execute dynamically: autonomously at power-on, directly commanded by the CPM to which the SoC is connected, or as a fault handling recovery action. Test execution and test result availability

must not depend on the state of the CPM or the CPM's communication channel, meaning the SoC manages the self-test locally and stores the test results in non-volatile memory, along with any runtime error indications from fault detection. The test architecture, as a functional block of the SoC, increases the device's complexity and cost. However, the effort and design costs are widely justified by the SoC's importance to the system's service [266]. For example, if the SoC provides L1-L2 functions for a 5G interface, it has a fundamental role in the service availability. These SoCs are equipped with a memory system comparable in complexity to the CPM one, to which we directly apply the same FM capabilities. For external memory, DDR5 memory with on-die ECC (ECS) must be used to maintain an extremely low probability of double-bit corruption. The ECC must be at least SECDED. The memory bus must also be supervised to prevent single-bit corruption events, which requires an ECC mechanism (sideband or link, depending on the DDR5 type used). In addition to the data redundancy provided by ECC mechanisms, memories must include spare physical redundancy (PPR-like), which is helpful in production and for recovery in the event of a permanent fault in memory arrays. CE and UCE counters must be accessible from the CPM to provide statistics on memory health, enabling fault-prediction strategies. The granularity of memory fault information must not exceed that of the PPR-like spare resources, allowing the simplest and most convenient recovery from permanent faults. For internal memories, undetected corruption must not occur. Corruption must always be detected. The use of single-bit correction mechanisms (SECDED) is strongly recommended, while implementing failure-isolation mechanisms (e.g., memory pollution) is mandatory to invalidate a corrupted memory entry and prevent the error from propagating to higher levels of the system. A single bit flip is, in most cases, a soft corruption that can be fixed by simply rewriting the memory. It is therefore essential for the SoC to distinguish between soft and hard errors and to adopt recovery actions that are proportionate and appropriate to the type of error detected. Granularity aligned with `atomic` modularity is also essential for processing. SoCs are multicore structures, whether CPUs or DSPs, and the number of processing units continues to grow. Each processing unit must be able to be isolated in the event of a permanent fault, and the SoC must have a minimum number of spare units to replace the faulty unit.

This means that the SoC can perform structural and functional tests on the individual processing unit, and this capability must be used as a recovery and verification mechanism. Error information must be propagated between the SoC's functional blocks via hardware signaling so it can be stored in a permanent error log. The flow of error information must be independent of the state of the CPM. The CPM must be able to configure the SoC to cascade internal error information or directly access the error log, but under no circumstances must it compromise the availability of error information in the SoC error log. The SoC error log in permanent memory must always survive a component restart. Each functional block must be dynamically configurable; that is, the SoC must be able to mask a functional block to isolate an error condition and control the SoC's overall power consumption. Redundant units will allow the SoC to locally mask the error. If the redundant resources are also consumed, recovery management is scaled to a higher system level to implement the degraded function condition. Communication between functional blocks should also be configurable, allowing for dynamic management of these blocks. For each error detected by an individual functional block, a fault detection policy verification process must be available. This is only possible if the hardware supports error injection mechanisms. Each functional block must include configuration, status, and error masking registers. The CPM must be able to manage all the SoC's control and configuration registers.

Eventually, SoC architectures incorporate a unique device identifier, such as an Electronic Chip ID or a Physically Unclonable Function (PUF) UID [267], on-die environmental sensors for voltage/temperature monitoring, and support for secure microcode/firmware updates over authenticated channels. These features, while adding minimal area overhead, are essential for enabling chip-level traceability, health monitoring, and remote serviceability in mission-critical deployments.

SoC Devices Requirement Summary

Slogan	Description	Domain
Modularity Enables Resilience	The SoC shall be designed as a set of highly modular functional blocks with plug-and-play capabilities, enabling fine-grained fault detection and independent fault isolation at both production and runtime.	Fault Detection, Isolation
Self-Aware Subsystems	Each SoC functional block shall continuously monitor its own health status and report fault conditions autonomously, without reliance on external controllers.	Fault Detection
Autonomous Self-Test Execution	The SoC shall include an on-die self-test macro capable of executing structural and functional tests autonomously at power-on, on CPM command, or as part of a fault recovery action.	Self-Test
Self-Test Independence from CPM	Self-test execution and test result availability shall not depend on the CPM's operational state or communication availability.	Fault Detection
Persistent Fault Evidence	The SoC shall store self-test results and runtime fault indications in non-volatile memory that survives component reset and power cycling.	Fault Information
ECC-Protected External Memory	External DDR5 memories interfaced with the SoC shall implement at least SECDED on-die ECC and scrubbing.	Fault Detection
ECC-Protected Memory Bus	SoC CM shall implement bus-level at least SECDED ECC supervision.	Fault Detection, Recovery
Physical Memory Redundancy	Memory subsystems shall implement spare physical redundancy (PPR-like) and provide an interface and a method to recover from permanent memory array faults detected during production or operation.	Fault Recovery, Redundancy
Memory Health Observability	Correctable and uncorrectable error counters (CE/UCE) shall be accessible by the CPM to support memory health assessment and fault prediction strategies.	Fault Prediction

Continues in the next page

Slogan	Description	Domain
Granular Memory Fault Localization	The granularity of reported memory fault information shall match the granularity of available spare resources, enabling deterministic and minimal-impact recovery actions.	Fault localization, Redundancy
No Silent Data Corruption	Internal memories shall guarantee that data corruption is always detected.	Fault Detection, Fault Coverage
Mandatory Memory Isolation	Upon detection of memory corruption, the SoC shall invalidate the affected memory entry to prevent error propagation to higher system layers.	Fault Isolation
Proportional Memory Recovery	The SoC shall distinguish between soft and hard memory errors and support/apply recovery actions proportional to the fault type, avoiding unnecessary escalation.	Fault Recovery
Isolatable Processing Units	Each processing unit (CPU/DSP core) shall be independently isolatable in case of a permanent fault.	Fault Isolation
Processing Redundancy	The SoC shall include spare processing resources that can replace faulty resources after isolation and verification.	Fault Recovery, Redundancy
Hardware Error Propagation	Fault and error information shall be propagated between functional blocks using hardware signaling mechanisms independent of CPM state.	Fault Information
Error Log Availability	The CPM may access or configure error information flow and the SoC permanent error log.	Fault Information, Contextualization
Dynamic Block Masking	The SoC shall support dynamic masking of functional blocks to limit fault impact and manage power consumption.	Degraded Function, Contextualization
Configurable Interconnect	Communication paths between functional blocks shall be dynamically configurable to support fault containment and reconfiguration.	Fault Isolation, Degraded Function
Hardware Error Injection	Each functional block shall support hardware error injection mechanisms to verify fault detection policies during validation and operation.	Fault Detection

Continues in the next page

Slogan	Description	Domain
Device Identity and Sensors	The SoC shall integrate a unique device identifier and on-die environmental sensors to enable traceability, health monitoring, and contextual fault analysis.	Fault prediction, Contextualization
Secure Update Capability	The SoC shall support authenticated microcode or firmware updates to enable safe remote recovery and long-term serviceability.	Security, Fault Recovery

Table 10.7. HW Design FM-driven SoC Device Requirements.

10.5 Embedded System Connectivity

In highly integrated embedded systems, hardware monitoring of connectivity links is essential for system resilience. This monitoring goes beyond simply detecting if a link is up or down. Instead, it involves continuously assessing the channel's physical state. The aim is to distinguish between transient errors, gradual deterioration, and complete failures. This aligns with the trend in high-speed transceivers, where operating margins are managed adaptively and statistically [268].

A main architectural principle is separating monitoring domains for external and internal interfaces. Structural supervision of external links (Ethernet/Transport Network, radio) transcends board-level scope, necessitating networking management-plane coordination to access physical link states known only systemically [269]. For one embedded system in the RAN domain, FM focuses on the local physical termination rather than the remote side. Here, high BER and CDR lock loss are key indicators. To distinguish a link fault from a hardware fault in the termination, the termination must support loopback testing. This lets you check and locate faults. Internal Ethernet interfaces, usually between the CPM and board subsystems, are supervised at the board level. These peer-to-peer connections lack a control hierarchy like PCI Express. Link supervision must run on both ends with matching observability and control tools.

For 50 Gb/s Ethernet interfaces following IEEE 802.3-2022 Clause 134 [270],

there is a main change from earlier versions. RS code adoption means the channel is not always error-free. The RS-FEC decoder corrects burst errors within its limits and reports uncorrectable events only when those limits are exceeded. Still, occasional uncorrectable FEC errors alone are not a reliable sign of hardware failure and should not trigger automatic fault escalation or be logged directly. This is a conscious trade-off between sensitivity and the avoidance of false positives. To supervise such links well, monitor trends in BER, FEC statistics, CRC errors, CDR status, and timing over time. Monitoring end-to-end and round-trip latency using methods such as the Two-Way Active Measurement Protocol (TWAMP) [271] provides useful data for AI/ML-based fault prediction. However, this alone cannot fully describe the channel's physical state. An effective operational model involves classifying the link state into three phases:

- **Healthy**, in which the link operates within nominal margins;
- **Degraded**, in which persistent deviations or negative trends are observed;
- **Down**, in which the link is inoperative.

The change from Healthy to Degraded marks the Link Degradation Start Point. At this point, controls shift from broad checks for single errors or small degradation to detailed checks for burst errors and repeating issues. The switch from Degraded to Down triggers link reconfiguration, which may include a targeted reset. Note that a link-down event only counts as a true high-speed link failure if it follows clear and sustained degradation. If a link goes down with no historical trend, it is ambiguous and may be caused by software or systemic issues.

Error-injection tools are also part of this supervision setup. They validate whether detection and logging work correctly. These tools are not used during system operation, but they are vital to design reliability.

Extending the discussion to other architectures, in the PCI Express domain, error management is based on a hierarchical structure formally defined in the PCIe Base Specification. Advanced Error Reporting (AER) technology enables fine-grained error reporting at the physical, link, and transaction levels. Enabling AER capabilities represents a mature example of integration between hardware

fault detection mechanisms and software recovery policies.

For devices that do not use PCIe, the MCA (Machine Check Architecture, a hardware mechanism for reporting errors) provides error reporting via dedicated registers. Only fatal, uncorrectable events are stored in the hardware error logs to avoid accumulating reports that have little diagnostic value. External radio interfaces follow the same philosophy but focus on local termination: BER and CDR unlock are the only key fault indications. Internal radio links, often using Ethernet or CPRI between L1/L2 subsystems, require the same supervision as internal Ethernet links: statistics, time trends, and contextual logs.

Structural system supervision uses JTAG boundary-scan, which operates in board, external, internal, and system modes. It can detect structural faults such as stuck-at, open, or short circuits, even in multi-board setups. This is a critical addition to runtime diagnostics based on link monitoring and self-tests.

RS-FEC supervision, PCIe-AER reporting, MCA management, and JTAG boundary-scan work together to form a layered diagnostics system. This approach relies on linking related events over time rather than just reacting to isolated signals. The trade-off balances physical rigor, design complexity, and reliability, mirroring how high-speed embedded designs are moving toward predictive, resilient models.

Connectivity Requirement Summary

Slogan	Description	Domain
Physical-State Awareness	Connectivity hardware shall expose observability of the physical channel state beyond link up/down, including BER, FEC counters, CDR lock status, and timing margins, to enable assessment of transient, degrading, and failed conditions.	Fault detection, information, contextualization

Continues in the next page

Slogan	Description	Domain
Local-Termination Focus	For external Ethernet and radio links, hardware monitoring shall focus exclusively on the local physical termination, avoiding assumptions about remote endpoints.	Fault isolation, localization
Loopback Capability	External physical terminations shall support hardware loopback modes to discriminate between channel faults and termination hardware faults during diagnostics.	Fault isolation, self-test, localization
Bidirectional Supervision	Internal Ethernet or radio links without a defined hierarchy shall provide symmetric monitoring, control, and statistics at both endpoints to enable consistent peer-to-peer supervision.	Fault detection, isolation, recovery
FEC-Aware Monitoring	For RS-FEC–enabled Ethernet links, hardware shall expose detailed FEC statistics and trends; single uncorrectable error events shall not be treated as direct indicators of hardware failure.	Fault detection, information, contextualization
Trend-Based Degradation Detection	Connectivity hardware shall support time-correlated statistics (BER, FEC, CRC, CDR, latency) to enable detection of persistent deviations and negative trends preceding failure.	Fault prediction, degraded function
Explicit Degraded State Support	Hardware shall support classification of link states into Healthy, Degraded, and Down, with clear thresholds enabling controlled transitions and targeted recovery actions.	Degraded function, recovery
Controlled Recovery Hooks	Upon transition from Degraded to Down, hardware shall support targeted recovery actions (e.g., link retrain or reset) rather than unconditional full-system resets.	Recovery
Error Injection Support	Connectivity subsystems shall support error-injection mechanisms for validation of detection, logging, and escalation paths during design and verification phases.	Self-test, fault detection
Hierarchical Error Reporting	Where applicable (e.g., PCIe), hardware shall implement standardized hierarchical fault reporting mechanisms (such as AER) enabling fine-grained classification across protocol layers.	Fault detection, information, recovery

Continues in the next page

Slogan	Description	Domain
Fatal-Only Hardware Logging	Connectivity Hardware shall support link status and error type signaling configuration to mask fault information on demand.	Information, fault isolation
Structural Fault Coverage	Connectivity hardware shall support structural supervision mechanisms such as JTAG boundary-scan to detect opens, shorts, and stuck-at faults beyond runtime link monitoring.	Fault detection, localization, self-test

Table 10.8. HW Design FM-driven Connectivity Requirements.

XI

Chapter 11

Conclusions

This thesis investigated system resilience in embedded systems for mobile networks and identified functions that confer resilience. It focused on hardware fault management, which emerged as the main system function of resilience. As a natural consequence, the thesis analyzed the impacts of fault management on hardware design, fault probabilistic modeling, cost-aware fault management, and system-level coordination of fault management. This approach allowed us to frame fault management holistically, with service availability in RAN environments as a key factor. Following this holistic service-oriented approach, this thesis aims to build a strong link between academic research and real-world industrial needs.

11.1 Summary of Research Aims and Achievements

The primary research goal of this dissertation was to develop an efficient, resilience-oriented fault management framework for RAN embedded systems, with particular emphasis on the role of hardware design. This goal was pursued through five complementary research challenges:

(i) Analysis of the system resilience,

(ii) Analysis of fault management as a system function,

(iii) Investigation of the contribution of hardware design to fault management efficiency,

(iv) Develop a new mathematical model for failure probability,

(v) Validate the proposed framework through realistic industrial use cases.

All research objectives have been met. The thesis provides a coherent conceptual foundation linking resilience, dependability, and fault management and translates it into quantitative models, architectural guidelines, and concrete hardware requirements.

A key outcome of the work is the demonstration that resilience in embedded RAN systems cannot be achieved solely through network-level mechanisms, but must be designed and enabled at the node and component levels, starting with board and device architecture.

11.2 Conceptual Contributions to Resilience and Fault Management

One of the central contributions of this research is the formal interpretation of fault management as the primary operational function through which resilience is realized. By distinguishing resilience from reliability and explicitly linking resilience to recovery dynamics rather than to failure avoidance alone, the thesis clarifies a conceptual gap in the existing literature. The thesis uses a hierarchical functional framework for system resilience: design, operational, adaptive, and cognitive. The functional framework enables visualization of how different system functions contribute to overall behavior during disturbances. The fault management is identified as a key factor. This is particularly due to its influence on fault detection latency (MTTD), recovery time (MTTR), and the ability to operate under degraded conditions. Reinterpreting the resilience triangle model represents a step forward in understanding a system's resilience. Through an extended version of Bruneau's traditional formulation, the thesis explains the roles of detection time, partial recovery, and the management of degraded functionality. The new model is more functional in representing RAN embedded

systems. Furthermore, the revised model enables a direct mapping between resilience parameters and concrete fault-management mechanisms.

11.3 Quantitative Modeling of Failure Probability

The thesis introduces a new quantitative method for failure probability calculation based on hypergraph theory. This method addresses the practical limitations of classical Fault Tree Analysis with complex, highly interconnected systems. By expressing functional dependencies as vertex-weighted hypergraphs and incidence matrices, the proposed model allows scalable, methodical computation of failure probability. The thesis shows how to apply the method to different cases. The intrinsic flexibility of the proposed model allows it to be used even in cases of different system characterizations, such as latency, capacity, and administrative costs for networks. This mathematical method constitutes a general-purpose tool for dependability analysis.

11.4 Fault Management Cost and System-Level Implications

An additional significant contribution of the thesis is the structured analysis of fault management from an economic and systemic perspective. By modeling fault management costs as a complex socio-technical system, the thesis highlights the nonlinear relationship between early design investments and long-term operational and reputational outcomes. The analytical relationship between fault coverage and return rate provides a concrete example of how insufficient fault management capabilities can lead to disproportionate lifecycle costs. The analysis also shows that fault coverage has decreasing returns above certain thresholds. The previous considerations emphasize the need for balanced and cost-conscious design decisions. The application of systems theory and game-theoretic reasoning helps us understand the importance of having shared design requirements in organizations as a coordination mechanism between different business realities.

11.5 Fault Prediction and Data-Driven Methods

The integration of machine learning into fault management is treated as a complementary, rather than a substitutive, capability. The thesis shows that fault prediction can operate effectively in parallel with classical fault management pillars, particularly for detecting latent or progressive degradation phenomena. The presented case study on memory error detection demonstrates that statistical variations in the system fingerprint can reveal hardware failure conditions. The results confirm that data-driven methods can improve fault coverage. The analysis also highlighted the importance of meaningful, high-quality telemetry, further strengthening the critical role of fault-management-oriented hardware design. But data quality analysis highlights how system knowledge is a critical skill for this type of analysis.

11.6 Hardware-Centered Design Guidelines

A major outcome of this research is the comprehensive set of fault-management-driven hardware requirements defined at board, component, and subsystem levels. These requirements translate abstract resilience and fault management concepts into concrete design constraints for central processors, memory systems, storage devices, low-integration components, system-on-chip devices, and connectivity interfaces. Across all analyzed subsystems, the thesis consistently demonstrates that resilience requires self-describing components, high-granularity fault information, modular isolation capabilities, autonomous and coordinated recovery mechanisms, and explicit support for fault prediction. Silent faults pose a critical threat to system dependability, and their elimination emerges as a non-negotiable design principle. Many resilience-enhancing mechanisms—such as error correction, self-test, redundancy, logging, and telemetry—are already technologically feasible. However, they are often not systematically integrated into embedded systems. The proposed requirements serve both as technical guidance and as a bridge between research and industrial practice.

11.7 Industrial and Scientific Impact

The thesis demonstrates the value of close and continuous interaction between academic research and industrial development. All proposed models and frameworks have been conceived with applicability in real RAN products. Several of them have been validated through concrete use cases. This dual orientation ensures that the scientific contributions are not only theoretically sound but also practically relevant. From a scientific perspective, the work makes a valuable contribution to service-oriented resilience modeling, fault management architecture, and reliability analysis. From an industrial perspective, it provides workable guidelines to improve service availability and reduce maintenance costs.

11.8 Limitations and Future Work

Although the thesis provides a comprehensive treatment of hardware fault management in embedded RAN systems, several points remain open for future research. Certainly, the contribution of other functions in the hierarchical resilience framework deserves attention. The strong relationship between fault management and them clearly shows that the full realization of the fault management model cannot ignore the interaction with other resilience functions. The quantitative failure probability model supports a design decision-making process, which must be carefully explored. By characterizing systems as polymatrodes, their convergent nature can be exploited to identify effective decision-making algorithms. As RAN architectures evolve toward increasingly virtualized and disaggregated implementations, future work could extend the presented framework to distributed, cloud-native environments.

11.9 Concluding Remarks

This thesis has shown that resilience in embedded systems for mobile networks is not an emergent property or a post-deployment optimization. It is a design objective that must be addressed systematically from the earliest stages of hardware and system architecture. Fault management has been demonstrated to

be the critical function through which resilience is implemented and measured. This work contributes a unified framework for understanding, designing, and evaluating resilient embedded systems in mobile networks. We believe this contribution will support both further academic research and the development of more reliable and serviceable network infrastructures.

Bibliography

[1] Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Jakob Danielsson, Alf Larsson, and Thomas Nolte. A reliability-oriented faults taxonomy and a recovery-oriented methodological approach for systems resilience. *COMPSAC 2022, The Proceeding IEEE 46th Annual Computers, Software, and Applications Conference*, pages 48–55, 6 2022.

[2] Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Jakob Danielsson, Alf Larsson, and Thomas Nolte. Fault management framework and multilayer recovery methodology for resilient system. In *ICSRS 2022, the 6th International Conference on System Reliability and Safety*, pages 32–39. IEEE, 11 2022.

[3] Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, Jakob Danielsson, Alf Larsson, and Thomas Nolte. Fault management impacts on the networking systems hardware design. In *IECON 2023, The 49th Annual Conference of the IEEE Industrial Electronics Society*, pages 1–8, 2023.

[4] Carlo Vitucci, Daniel Sundmark, Jakob Danielsson, Marcus Jägemar, Alf Larsson, and Thomas Nolte. Run time memory error recovery process in networking system. In *ICSRS 2023, The 7th International Conference on System Reliability and Safety*, pages 590–597, 2023.

[5] Selma Rahman, Mattias Olausson, Carlo Vitucci, and Ioannis Avgouleas. Ambient temperature prediction for embedded systems using machine learning. In *Engineering of Computer-Based Systems*, pages 12–25. Springer Nature Switzerland, 2024.

[6] Carlo Vitucci, Thomas Westerbäck, Daniel Sundmark, Håkan Forsberg, Thomas Nolte, and Marcus Jägemar. A deductive fault analysis method based on hypergraphs. In IEEE, editor, *SAFEPROCESS 2024, The 12th IFAC Symposium on Fault Detection, Supervision and Safety for Technical Processes*. IEEE, 6 2024.

[7] Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, and Thomas Nolte. A fault management oriented resilience model for networking systems. In IEEE, editor, *ICSRS 2024, The 8th International Conference on System Reliability and Safety*, pages 1–8. IEEE, 11 2024.

[8] Selma Rahman, Mattias Olausson, Carlo Vitucci, and Ioannis Avgouleas. Machine learning-based ambient temperature prediction in radio access network environments. *International Journal on Software Tools for Technology Transfer*, 27:81–92, 2024.

[9] Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, and Thomas Nolte. Low-level anomaly detection in embedded systems using machine learning. In *2025 International Conference on Computer Technology Applications (ICCTA)*, pages 115–121. IEEE, 5 2025.

[10] Carlo Vitucci, Daniel Sundmark, Marcus Jägemar, and Thomas Nolte. Fault management cost system analysis in radio access network embedded systems. In *2025 9th International Conference on System Reliability and Safety (ICSRS)*, pages 477–484, 2025.

[11] Giovanni Fiaschi, Carlo Vitucci, Thomas Westerbäck, Daniel Sundmark, and Thomas Nolte. A formal model for path set attribute calculation in network systems. In *2025 International Symposium on Networks, Computers and Communications (ISNCC)*, pages 1–8, 2025.

[12] Carlo Vitucci. The role of fault management in the embedded system design, 2024.

[13] Carlo Vitucci, J. Lelli, A. Parri, and M. Marinoni. A linux-based virtualized solution providing computing quality of service to sdn-nfv telecommunication applications. *undefined*, 2014.

[14] Carlo Vitucci and Alf Larsson. Hat, hardware assisted trace: Performance oriented trace & debug system. In AFIS, editor, *ICSSEA 2015, 26th International Conference on software & systems engineering and their applications*, pages 1–7, 5 2015.

[15] Carlo Vitucci and Alf Larsson. Flexible 5g edge server for multi industry service network. *International Journal on Advances in Networks and Services*, 10:55–69, 12 2017.

[16] Carlo Vitucci and Alf Larsson. Seed, a server platform for the edge of the network. In IARIA, editor, *ICN 2017 : The Sixteenth International Conference on Networks (includes SOFTNETWORKING 2017)*, pages 118–123. ThinkMind, 4 2017.

[17] Tommaso Cucinotta, Mauro Marinoni, Alessandra Melani, Andrea Parri, and Carlo Vitucci. Temporal isolation among lte/5g network functions by real-time scheduling. *CLOSER 2017 - Proceedings of the 7th International Conference on Cloud Computing and Services Science*, pages 340–347, 2017.

[18] Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Alessio Balsini, and Carlo Vitucci. Virtual network functions as real-time containers in private clouds. *IEEE International Conference on Cloud Computing, CLOUD*, 2018-July:916–919, 9 2018.

[19] Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, and Carlo Vitucci. The importance of being os-aware: In performance aspects of cloud computing research. *CLOSER 2018 - Proceedings of the 8th International Conference on Cloud Computing and Services Science*, 2018-January:626–633, 2018.

[20] Mauro Marinoni, Tommaso Cucinotta, Luca Abeni, and Carlo Vitucci. Allocation and control of computing resources for real-time virtual network functions. In IARIA, editor, *ICN 2018 : The Seventeenth International Conference on Networks*, pages 52–57. ThinkMind, 4 2018.

[21] Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Alessio Balsini, and Carlo Vitucci. Reducing temporal interference in private clouds through real-time containers. *Proceedings - 2019 IEEE International Conference on Edge Computing, EDGE 2019 - Part of the 2019 IEEE World Congress on Services*, pages 124–131, 7 2019.

[22] Carlo Vitucci, Luca Abeni, Tommaso Cucinotta, and Mauro Marinoni. The strategic role of inter-container communications in ran deployment scenarios. In IARIA, editor, *ICN 2019: The Eighteenth International Conference of Networks*, pages 31–36. IARIA, 3 2019.

[23] Gabriele Ara, Luca Abeni, Tommaso Cucinotta, and Carlo Vitucci. On the use of kernel bypass mechanisms for high-performance inter-container communications. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 11887 LNCS:1–12, 2019.

[24] Gabriele Ara, Tommaso Cucinotta, Luca Abeni, and Carlo Vitucci. Comparative evaluation of kernel bypass mechanisms for high-performance inter-container communications. *CLOSER 2020 - Proceedings of the 10th International Conference on Cloud Computing and Services Science*, pages 44–55, 2020.

[25] Carlo Vitucci, Tommaso Cucinotta, Riccardo Mancini, and Luca Abeni. Implementation and deployment of a server at the edge using openstack components. In IARIA, editor, *ICN 2020 : The Nineteenth International Conference on Networks*, pages 22–29. ThinkMind, 2 2020.

[26] Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Riccardo Mancini, and Carlo Vitucci. Strong temporal isolation among containers in openstack for nfv services. *IEEE Transactions on Cloud Computing*, pages 1–1, 9 2021.

[27] Gabriele Ara, Leonardo Lai, Tommaso Cucinotta, Luca Abeni, and Carlo Vitucci. A framework for comparative evaluation of high-performance virtualized networking mechanisms. *Communications in Computer and Information Science*, 1399 CCIS:59–83, 2021.

[28] Claudio Porfiri and Carlo Vitucci. A transmitting entity and method performed thereby for transmitting one or more data packets to a receiving entity, 2015.

[29] Claudio Porfiri and Carlo Vitucci. Method and network node for handling sctp packets, 2016.

[30] Claudio Porfiri and Carlo Vitucci. Methods and arrangements for determining indication of maximum datagram size supported without fragmentation in an ip network, 2018.

[31] Claudio Porfiri and Carlo Vitucci. Methods and apparatus for multihoming sctp communication between a network entity and a remote host, 2021.

[32] Angelo Centonza, Gian Luca Masini, Claudio Porfiri, and Carlo Vitucci. Encapsulating an sctp packet in a udp packet, 2022.

[33] Wahab A. Iddrisu and Ibrahim A. Gedel. Modeling downtime severity of telecommunication networks using discrete time markov chains. *International Journal of Advanced Technology and Engineering Exploration*, 10, 2023.

[34] Alexandru Martian, Razvan-Florentin Trifan, Teodora-Cristina Stoian, Marius-Constantin Vochin, and Frank Y. Li. Towards open ran in beyond 5g networks: Evolution, architectures, deployments, spectrum, prototypes, and performance assessment. *Computer Networks*, 259:111087, 2025.

[35] Matthew C. Valenti, Salvatore Talarico, and Peter Rost. The role of computational outage in dense cloud-based centralized radio access networks. In *Proceedings - IEEE Global Communications Conference, GLOBECOM*, 2014.

[36] Luigi De Simone, Mario Di Mauro, Roberto Natella, and Fabio Postiglione. Performance and availability challenges in designing resilient 5g architectures. *IEEE Transactions on Network and Service Management*, 21:5291–5303, 2024.

[37] David Escudero-Mancebo, Nieves Fernández-Villalobos, Óscar Martín-Llorente, and Alejandra Martínez-Monés. Research methods in engineering design: a synthesis of recent studies using a systematic literature review. *Research in Engineering Design*, 2023.

[38] Viswanath Venkatesh, Susan A. Brown, and Hillol Bala. Bridging the qualitative-quantitative divide: Guidelines for conducting mixed methods research in information systems. *MIS Quarterly: Management Information Systems*, 37, 2013.

[39] Alan Bryman. Integrating quantitative and qualitative research: How is it done? *Qualitative Research*, 6, 2006.

[40] André Queirós, Daniel Faria, and Fernando Almeida. Strengths and limitations of qualitative and quantitative research methods. *European Journal of Education Studies*, 3, 2017.

[41] Angela Carrera-Rivera, William Ochoa, Felix Larrinaga, and Ganix Lasa. How-to conduct a systematic literature review: A quick guide for computer science research. *MethodsX*, 9, 2022.

[42] Behrooz Parhami. Defect, fault, error, ..., or failure? *IEEE Transactions on Reliability*, 46:450–451, 1997.

[43] Guru Prasad Bhandari and Ratneshwer Gupta. Extended fault taxonomy of soa-based systems. *Journal of Computing and Information Technology*, 25:237–257, 1 2018.

[44] Alessandro Birolini. *Reliability Engineering*. Springer Berlin Heidelberg, 2014.

[45] Dov Dori, Hillary Sillitto, Regina M. Griego, Dorothy McKinney, Eileen P. Arnold, Patrick Godfrey, James Martin, Scott Jackson, and Daniel Krob. System definition, system worldviews, and systemness characteristics. *IEEE Systems Journal*, 14:1538–1548, 6 2020.

[46] Steve Scargall. Reliability, availability, and serviceability (ras). *Programming Persistent Memory*, pages 333–346, 2020.

[47] Dhanya R Mathews, Mudit Verma, J Lakshmi, and Pooja Aggarwal. Towards more effective and explainable fault management using cross-layer service topology. In *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*, pages 94–96. IEEE, 2022.

[48] Peter Brooks and Bjoørn Hestnes. User measures of quality of experience: Why being objective and quantitative is important. *IEEE Network*, 24:8–13, 3 2010.

[49] Sepideh Safari, Mohsen Ansari, Heba Khdr, Pourya Gohari-Nazari, Sina Yari-Karin, Amir Yeganeh-Khaksar, Shaahin Hessabi, Alireza Ejlali, and Jörg Henkel. A survey of fault-tolerance techniques for embedded systems from the perspective of power, energy, and thermal issues. *IEEE Access*, 10:12229–12251, 2022.

[50] Elham Moridi, Majid Haghparast, Mehdi Hosseinzadeh, and Somaye Jafarali Jassbi. Fault management frameworks in wireless sensor networks: A survey, 4 2020.

[51] Mourad Nouioua, Philippe Fournier-Viger, Ganghuan He, Farid Nouioua, and Zhou Min. *A Survey of Machine Learning for Network Fault Management*. Springer International Publishing, 2021.

[52] Małgorzata Steinder and Adarshpal S. Sethi. A survey of fault localization techniques in computer networks. *Science of Computer Programming*, 53:165–194, 2004.

[53] Mukunda K. Das and Krishnan Rangarajan. Performance monitoring and failure prediction of industrial equipments using artificial intelligence and machine learning methods: A survey. *Proceedings of the 4th International Conference on Computing Methodologies and Communication, ICCMC 2020*, pages 595–602, 3 2020.

[54] B.W. Johnson. *Design and Analysis of Fault-tolerant Digital Systems*. Addison-Wesley series in electrical and computer engineering. Addison-Wesley Publishing Company, 1989.

[55] Majed Valad Beigi, Sudhanva Gurumurthi, and Vilas Sridharan. Reliability, availability, and serviceability challenges for heterogeneous system design. In *IEEE International Reliability Physics Symposium Proceedings*, volume 2022-March, 2022.

[56] Nematollah Bidokhti. How to close the gap between hardware and software using fmea. *2007 Proceedings - Annual Reliability and Maintainability Symposium, RAMS*, pages 167–172, 2007.

[57] Manuel Barranco, Sinisa Derasevic, and Julian Proenza. An architecture for highly reliable fault-tolerant adaptive distributed embedded systems. *Computer*, 53:38–46, 3 2020.

[58] Harry Hulvershorn, Paul Soong, and Saman Adham. Linking diagnostic software to hardware self test in telecom systems. *IEEE International Test Conference (TC)*, pages 986–993, 1995.

[59] Nan Ya, Xingwei Wang, Shuang Zhang, and Min Huang. Multipath fault-tolerance routing mechanism in data center network. *Proceedings - 2018 17th International Symposium on Distributed Computing and Applications for Business Engineering and Science, DCABES 2018*, pages 246–249, 12 2018.

[60] Taizhi Liu, Chang Chih Chen, Woongrae Kim, and Linda Milor. Comprehensive reliability and aging analysis on srams within microprocessor systems. *Microelectronics Reliability*, 55:1290–1296, 8 2015.

[61] Vilas Sridharan, Nathan De Bardeleben, Sean Blanchard, Kurt B. Ferreira, Jon Stearley, John Shalf, and Sudhanva Gurumurthi. Memory errors in modern systems the good, the bad, and the ugly. *ACM SIGPLAN Notices*, 50:297–310, 4 2015.

[62] Shubu Mukherjee. Error coding techniques. *Architecture Design for Soft Errors*, pages 161–206, 1 2008.

[63] Vadhiraj Sankaranarayanan. Ecc in ddr memories. *©2022 Synopsys, Inc*, 2022.

[64] Andi Kleen. hwpoison — the linux kernel documentation. Technical report, Linux, 2009.

[65] R. Kothe, C. Galke, S. Schultke, H. Fröschke, S. Gaede, and H. T. Vierhaus. Hardware/software based hierarchical self test for socs. *2006 IEEE Design and Diagnostics of Electronic Circuits and systems*, 2006:157–158, 2006.

[66] Saeed Shamshiri, Hadi Esmaeilzadeh, and Zainalabedin Navabi. Instruction level test methodology for cpu core software-based self-testing. *Proceedings - IEEE International High-Level Design Validation and Test Workshop, HLDVT*, pages 25–28, 2004.

[67] Anna Lanzaro, Antonio Pecchia, Marcello Cinque, Domenico Cotroneo, Ricardo Barbosa, and Nuno Silva. A preliminary fault injection framework for evaluating multicore systems. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 7613 LNCS:106–116, 2012.

[68] Jie Yao, Hong Jiang, Qiang Cao, Lei Tian, and Changsheng Xie. Elastic-raid: A new architecture for improved availability of parity-based raids by elastic mirroring. *IEEE Transactions on Parallel and Distributed Systems*, 27:1044–1056, 4 2016.

[69] Juan L. Bejarano-Luque, Matias Toril, Mariano Fernandez-Navarro, Carolina Gijon, and Salvador Luna-Ramirez. A deep-learning model for estimating the impact of social events on traffic demand on a cell basis. *IEEE Access*, 9:71673–71686, 2021.

[70] Masanori Miyazawa and Munefumi Tsurusawa. Fault localization mechanism using integrated resource information model in next generation network. *Proceedings of the 2010 IEEE/IFIP Network Operations and Management Symposium, NOMS 2010*, pages 734–747, 2010.

[71] Konstantinos Samdanis and Tarik Taleb. The road beyond 5g: A vision and insight of the key technologies. *IEEE Network*, 34(2):135–141, 2020.

[72] Nazli Siasi, Mohammed A Jasim, Aysegül Yayimli, and Nasir Ghani. Service function chain survivability provisioning in fog networks. *IEEE Transactions on Network and Service Management*, 19:1117–1128, 2022.

[73] Che-Chun Teng, Mei-Chun Chen, Min-Han Hung, and Hong-Jie Chen. End-to-end service assurance in 5g crosshaul networks. In *2020 21st Asia-Pacific Network Operations and Management Symposium (APNOMS)*, pages 306–309. IEEE, 2020.

[74] Nessrine Trabelsi, Lamia Chaari Fourati, and Chung Shue Chen. Interference management in 5g and beyond networks: A comprehensive survey, 2024.

[75] Maraj Uddin Ahmed Siddiqui, Faizan Qamar, Faisal Ahmed, Quang Ngoc Nguyen, and Rosilah Hassan. Interference management in 5g and beyond network: Requirements, challenges and future directions. *IEEE Access*, 9, 2021.

[76] Kasem Khalil, Omar Eldash, Ashok Kumar, and Magdy Bayoumi. Machine learning-based approach for hardware faults prediction. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 67:3880–3892, 2020.

[77] Mukunda K. Das and Krishnan Rangarajan. Performance monitoring and failure prediction of industrial equipments using artificial intelligence and machine learning methods: A survey. *Proceedings of the 4th International Conference on Computing Methodologies and Communication, ICCMC 2020*, pages 595–602, 3 2020.

[78] Hussaini Adamu, Bashir Mohammed, Ali Bukar Maina, Andrea Cullen, Hassan Ugail, and Irfan Awan. An approach to failure prediction in a cloud based environment. *2017 IEEE 5th International Conference on Future Internet of Things and Cloud (FiCloud)*, pages 191–197, 2017.

[79] Isaac Boixaderas, Darko Zivanovic, Sergi Moré, Javier Bartolome, David Vicente, Marc Casas, Paul M Carpenter, Petar Radojković, and Eduard Ayguadé. Cost-aware prediction of uncorrected dram errors in the field.

SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–15, 2020.

[80] Alexandru Vulpe, Mihai Idu, Denisa Gheorghe, Alexandru Martian, and Octavian Fratu. Ml-based analytics framework for beyond 5g mobile communication systems. *2020 28th Telecommunications Forum (TELFOR)*, pages 1–4, 2020.

[81] Wuyang Zhang, Russell Ford, Joonyoung Cho, Charlie Jianzhong Zhang, Yanyong Zhang, and Dipankar Raychaudhuri. Self-organizing cellular radio access network with deep learning. *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 429–434, 2019.

[82] Abdelaziz Guerrouat and Harald Richter. A formal approach for analysis and testing of reliable embedded systems. In *Electronic Notes in Theoretical Computer Science*, volume 141, 2005.

[83] Paula Herber, Timm Liebrenz, and Julius Adelt. Combining forces: How to formally verify informally defined embedded systems. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, volume 13047 LNCS, 2021.

[84] Stephen Edwards, Luciano Lavagno, Edward A. Lee, and Alberto Sangiovanni-Vincentelli. Design of embedded systems: formal models, validation, and synthesis. *Proceedings of the IEEE*, 85, 1997.

[85] Bassem Ouni, Cécile Belleudy, and Eric Senn. Accurate energy characterization of os services in embedded systems. *Eurasip Journal on Embedded Systems*, 2012, 2012.

[86] Luis Alejandro Cortés, Petru Eles, and Zebo Peng. Modeling and formal verification of embedded systems based on a petri net representation. *Journal of Systems Architecture*, 49, 2003.

[87] Syed Muhammad Ali Shah, Daniel Sundmark, Birgitta Lindström, and Sten F. Andler. Robustness testing of embedded software systems: An industrial interview study. *IEEE Access*, 4, 2016.

[88] Abhishek Dubey, Steve Nordstrom, Turker Keskinpala, Sandeep Neema, Ted Bapty, and Gabor Karsai. Towards a verifiable real-time, autonomic, fault mitigation framework for large scale real-time systems. *Innovations in Systems and Software Engineering*, 3, 2007.

[89] Ivica Crnkovic. Component-based approach for embedded systems. In *Ninth International Workshop on Component-Oriented Programming*, 2004.

[90] Dongfang Zhang, Hui Ji, Zhennan Li, and Hui Ge. Design of building environment detection system for architectures based on internet of things. *Computational Intelligence and Neuroscience*, 2022, 2022.

[91] F. Bleicher, D. Biermann, W. G. Drossel, H. C. Moehring, and Y. Altintas. Sensor and actuator integrated tooling systems. *CIRP Annals*, 72, 2023.

[92] Maria Domenica Di Benedetto and Giordano Pola. Networked embedded control systems: From modelling to implementation. In *Electronic Proceedings in Theoretical Computer Science, EPTCS*, volume 124, 2013.

[93] Pao Ann Hsiung and Chuen Hau Gau. Formal synthesis of real-time embedded software by time-memory scheduling of colored time petri nets. In *Electronic Notes in Theoretical Computer Science*, volume 65, 2002.

[94] Luigi Palopoli, Daniele Fontanelli, Luca Abeni, and Bernardo Villalba Frias. An analytical solution for probabilistic guarantees of reservation based soft real-time systems. *IEEE Transactions on Parallel and Distributed Systems*, 27, 2016.

[95] Insop Song, Sehjeong Kim, and Fakhreddine Karray. A real-time scheduler design for a class of embedded systems. *IEEE/ASME Transactions on Mechatronics*, 13, 2008.

[96] Daniele Fontanelli, Luca Greco, and Luigi Palopoli. Soft real-time scheduling for embedded control systems. In *Automatica*, volume 49, 2013.

[97] 3GPP. Vocabulary for 3gpp specifications. Technical Report TR 21.905 V18.0.0 (2024-04-03), 3GPP, 2024.

[98] 3GPP. Nr; nr and ng-ran overall description; stage-2. Technical Report TR 38.300 V18.6.0 (2025-06-24), 3GPP, 2025.

[99] E. Marilly, A. Aghasaryan, S. Betgé-Brezetz, O. Martinot, and G. Delègue. Alarm correlation for complex telecommunication networks using neural networks and signal processing. *2002 IEEE Workshop on IP Operations and Management, IPOM 2002*, pages 3–7, 2002.

[100] Negin Lashkari, Hamid Fekri Azgomi, Javad Poshtan, and Majid Poshtan. Asynchronous motors fault detection using ann and fuzzy logic methods. *ECCE 2016 - IEEE Energy Conversion Congress and Exposition, Proceedings*, 2016.

[101] Negin Lashkari, Hamid Fekri Azgomi, Javad Poshtan, and Majid Poshtan. Robust stator fault detection under load variation in induction motors using ai techniques. *Proceedings - 2015 IEEE International Electric Machines and Drives Conference, IEMDC 2015*, pages 1446–1451, 2 2016.

[102] Konstantinos Michail, Kyriakos M. Deliparaschos, Spyros G. Tzafestas, and Argyrios C. Zolotas. Ai-based actuator/sensor fault detection with low computational cost for industrial applications. *IEEE Transactions on Control Systems Technology*, 24:293–301, 1 2016.

[103] Hasbi Ash Shiddieqy, Farkhad Ihsan Hariadi, and Trio Adiono. Power line transmission fault modeling and dataset generation for ai based automatic detection. *Proceeding - 2019 International Symposium on Electronics and Smart Devices, ISESD 2019*, 10 2019.

[104] R. Jothi. A comparative study of unsupervised learning algorithms for software fault prediction. *Proceedings of the 2nd International Conference on Intelligent Computing and Control Systems, ICICCS 2018*, pages 741–745, 3 2019.

[105] Tianxiang Zheng, Zizhi Wang, Cong Tan, and Renjie Wang. Research on fault prediction and diagnosis method of pcb circuit. *Proceedings - 2020 International Conference on Artificial Intelligence and Computer Engineering, ICAICE 2020*, pages 387–390, 10 2020.

[106] Fatih Karpat, Ahmet Emir Dirik, Oguz Dogan, Onur Can Kalay, Burak Korcuklu, and Celalettin Yuce. A novel ai-based method for spur gear early fault diagnosis in railway gearboxes. *Proceedings - 2020 Innovations in Intelligent Systems and Applications Conference, ASYU 2020*, 10 2020.

[107] Lu Junyong, Tang Yinyin, Zeng Delin, Yan Feifei, and Zheng Yufeng. Fault prediction of electromagnetic launch system based on knowledge prediction time series. *IEEE Transactions on Industry Applications*, 57:1830–1839, 3 2021.

[108] Nematollah Bidokhti. The impact of reliability requirements on development life cycle. *Proceedings - Annual Reliability and Maintainability Symposium*, pages 307–311, 2008.

[109] Ammar Aljer and Philippe Devienne. Formal fault tolerant architecture. *3rd Int. Conf. on Communication Theory, Reliability, and Quality of Service, CTRQ 2010, Includes MOPAS 2010: 1st Int. Conf. on Models and Ontology-Based Design of Protocols, Architecture and Services*, pages 73–78, 2010.

[110] Subhrendu Chattopadhyay, Niladri Sett, Sukumar Nandi, and Sandip Chakraborty. Flipper: Fault-tolerant distributed network management and control. *Proceedings of the IM 2017 - 2017 IFIP/IEEE International Symposium on Integrated Network and Service Management*, pages 421–427, 7 2017.

[111] Ayush Dusia and Adarshpal S. Sethi. Recent advances in fault localization in computer networks. *IEEE Communications Surveys and Tutorials*, 18:3030–3051, 10 2016.

[112] Sihem Cherrared, Sofiane Imadali, Eric Fabre, Gregor Gossler, and Imen Grida Ben Yahia. A survey of fault management in network virtualization environments: Challenges and solutions. *IEEE Transactions on Network and Service Management*, 16:1537–1551, 12 2019.

[113] Fetahi Wuhib, Chunyan Fu, and Mbarka Soualhia. A look at automated fault management with machine learning - ericsson. Technical report, Ericsson AB, 2019.

[114] Andreas Menychtas and Kleopatra G. Konstanteli. *Fault Detection and Recovery Mechanisms and Techniques for Service Oriented Infrastructures*. IGI Global, 10 2012.

[115] Stefan Bruning, Stephan Weissleder, and Miroslaw Malek. A fault taxonomy for service-oriented architecture. pages 367–368. IEEE, 11 2007.

[116] SAE. Guidelines and methods for conducting the safety assessment process on civil airborne systems and equipment. 1996.

[117] ISO/FDIS 26262. Road vehicles - functional safety. pages part 1–10, 2011.

[118] IEC. Nuclear power plants - instrumentation and control system important to safety - software aspects for computer-based systems performing category a functions. 2009.

[119] IEC. Functional safety of electrical/electronic/programmable electronic safety-related systems. *IEC 61508, Edition 2.0*, 2010.

[120] Elena Dubrova. *Fault-tolerant design*. Springer New York, NY, 2013.

[121] Eric Bauer. *Design for Reliability: Information and Computer-Based Systems*. Wiley-IEEE Press, ieee book edition, 2010.

[122] Katherine Schroeder, Hung Trinh, and Victoria Yan Pillitteri. Measurement guide for information security volume 1 :. Technical report, National Institute of Standards and Technology, 12 2024.

[123] Érika Cota, Alexandre De Morais Amory, and Marcelo Soares Lubaszewski. *Reliability, availability and serviceability of networks-on-chip*. Spinger New York, NY, 2012.

[124] My M. Sellberg, Allyson Quinlan, Rika Preiser, Katja Malmborg, and Garry D. Peterson. Engaging with complexity in resilience practice. *Ecology and Society*, 26, 2021.

[125] Terje Aven. The call for a shift from risk to resilience: What does it mean? *Risk Analysis*, 39, 2019.

[126] Working Paper Series and John D Sterman. System dynamics: Systems thinking and modeling for a complex world. *European journal of computer science*, 21, 2003.

[127] Morten Wied, Josef Oehmen, and Torgeir Welo. Conceptualizing resilience in engineering systems: An analysis of the literature. *Systems Engineering*, 23, 2020.

[128] Elmar Kutsch, Mark Hall, and Neil Turner. *Project resilience: The art of noticing, interpreting, preparing, containing and recovering*. Ashgate Publishing Limited, 2016.

[129] Zi Li, Xiaoyang Li, and Rui Kang. System resilience modeling and assessment based on minimal paths. In *Proceedings of 2016 Prognostics and System Health Management Conference, PHM-Chengdu 2016*, 2017.

[130] Bryan C. Watson, Marc J. Weissburg, and Bert Bras. Sosrm: A new metric to standarize system-of system resilience evaluation. 2020.

[131] Fuchun Ren, Tingdi Zhao, Jian Jiao, and Yuqing Hu. Resilience optimization for complex engineered systems based on the multi-dimensional resilience concept. *IEEE Access*, 5:19352–19362, 9 2017.

[132] Yongqi Wang, Penghui Lin, Limao Zhang, Hongbo Yu, and Tiong Lee Kong Robert. Resilience-oriented design for complex mep systems in bim. *Advanced Engineering Informatics*, 55, 2023.

[133] Seyedmohsen Hosseini, Kash Barker, and Jose E. Ramirez-Marquez. A review of definitions and measures of system resilience. *Reliability Engineering and System Safety*, 145, 2016.

[134] Adina Aldea, Egle Vaicekauskaite, Maya Daneva, and Jean Paul Sebastian Piest. Assessing resilience in enterprise architecture: A systematic review. 2020.

[135] Xing Pan, Yanjing Yang, Guozhong Zhang, and Bozi Zhang. Resilience-based optimization of recovery strategies for network systems. In *2017 Second International Conference on Reliability Systems Engineering (ICRSE)*, pages 1–6, 2017.

[136] Paul M. Beach, Logan O. Mailloux, Brent T. Langhals, and Robert F. Mills. Analysis of systems security engineering design principles for the development of secure and resilient systems. *IEEE Access*, 7, 2019.

[137] M. Bruneau, Stephanie E. Chang, Ronald T. Eguchi, George C. Lee, T D O'rourke, Andrei M. Reinhorn, Masanobu Shinozuka, Kathleen J. Tierney, William A. Wallace, and Detlof von Winterfeldt. A framework to quantitatively assess and enhance the seismic resilience of communities. *Earthquake Spectra*, 19:733 – 752, 2003.

[138] Shangjia Dong, Xinyu Gao, Ali Mostafavi, Jianxi Gao, and Utkarsh Gangwal. Characterizing resilience of flood-disrupted dynamic transportation network through the lens of link reliability and stability. *Reliability Engineering and System Safety*, 232, 2023.

[139] Taro Kanno, Toshio Fujii, Ryosuke Watari, and Kazuo Furuta. *Modeling And Simulation Of A Service System In A Disaster To Assess Its Resilience*, pages 128–134. Presses des Mines, 10 2016.

[140] Nanxi Wang and Kum Fai Yuen. Resilience assessment of waterway transportation systems: Combining system performance and recovery cost. *Reliability Engineering and System Safety*, 226, 2022.

[141] Jean Claude Laprie. Dependable computing and fault tolerance: Concepts and terminology. In *Digest of Papers - FTCS (Fault-Tolerant Computing Symposium)*, 1985.

[142] David Sherwin, A. Hoyland, and M. Rausand. System reliability theory-models and statistical methods. *The Statistician*, 44, 1995.

[143] Wayne B. Nelson. Statistical methods for reliability data, second edition,. *Technometrics*, 63, 2021.

[144] Haitao Guo and Xianhui Yang. A simple reliability block diagram method for safety integrity verification. *Reliability Engineering and System Safety*, 92, 2007.

[145] Clifton a. Ericson. Fault tree analysis – a history. *The 17th International System Safety Conference*, 1999.

[146] Randal E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Transactions on Computers*, C-35, 1986.

[147] J. deMercado, N. Spyratos, and B. A. Bowen. A method for calculation of network reliability. *IEEE Transactions on Reliability*, R-25, 1976.

[148] William J. Stewart. *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press, 2021.

[149] Michael K. Molloy. Performance analysis using stochastic petri nets. *IEEE Transactions on Computers*, C-31, 1982.

[150] Salvatore Distefano and Antonio Puliafito. Dependability evaluation with dynamic reliability block diagrams and dynamic fault trees. *IEEE Transactions on Dependable and Secure Computing*, 6, 2009.

[151] Ilias Iliadis. Reliability and availability engineering: Modeling, analysis, and applications. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 182, 2019.

[152] Yuan Shun Dai, Yi Pan, and Xukai Zou. A hierarchical modeling and analysis for grid service reliability. *IEEE Transactions on Computers*, 56, 2007.

[153] Gloria Gori, Marco Papini, and Alessandro Fantechi. Efficient reliability block diagram evaluation through improved algorithms and parallel computing. *Applied Sciences (Switzerland)*, 15, 2025.

[154] Aliaksei Sandryhaila and José M.F. Moura. Discrete signal processing on graphs: Frequency analysis. *IEEE Transactions on Signal Processing*, 62, 2014.

[155] C. Berge. *Hypergraphs: Combinatorics of Finite Sets*. North-Holland Mathematical Library. Elsevier Science, 1984.

[156] Walter Hoffman and Richard Pavley. A method for the solution of the nth best path problem. *Journal of the ACM (JACM)*, 6, 1959.

[157] D. Eppstein. Dynamic euclidean minimum spanning trees and extrema of binary functions. *Discrete & Computational Geometry*, 13, 1995.

[158] Huiping Liu, Cheqing Jin, Bin Yang, and Aoying Zhou. Finding top-k shortest paths with diversity. *IEEE Transactions on Knowledge and Data Engineering*, 30, 2018.

[159] Allan Borodin, Aadhar Jain, Hyun Chul Lee, and Yuli Ye. Max-sum diversification, monotone submodular functions, and dynamic updates. *ACM Transactions on Algorithms*, 13, 2017.

[160] Giovanni Fiaschi, Francesco Davide Calabrese, and Daniele Ceccarelli. Methods and apparatus for transmitting data over multiple paths in a network, 4 2020. Patent WIPO PCT/EP2018/078025.

[161] Bill Haskins, Jonette Stecklein, Brandon Dick, Gregory Moroney, Randy Lovell, and James Dabney. 8.4.2 error cost escalation through the project life cycle. *INCOSE International Symposium*, 14, 2004.

[162] Barry W. Boehm. Software engineering economics. *IEEE Transactions on Software Engineering*, SE-10, 1984.

[163] Joseph M. Southgate, Katrina Groth, Peter Sandborn, and Shapour Azarm. Cost-benefit analysis using modular dynamic fault tree analysis and monte carlo simulations for condition-based maintenance of unmanned systems, 2024.

[164] Jakob Axelsson, Damir Bilic, Daniel Brahneborg, Joakin Fröberg, Henrik Gustavsson, Robbert Jongeling, and Daniel Sundmark. Alcea : The architecture life-cycle effect analysis method. *IEEE Open Journal of Systems Engineering*, 2, 2024.

[165] Dov Dori, Hillary Sillitto, Regina M. Griego, Dorothy McKinney, Eileen P. Arnold, Patrick Godfrey, James Martin, Scott Jackson, and Daniel Krob. System definition, system worldviews, and systemness characteristics. *IEEE Systems Journal*, 14(2):1538–1548, 2020.

[166] Peter Checkland and John Poulter. *Soft systems methodology*. Springer, 2020.

[167] Michael C. Jackson. Critical systems practice 4: Check—evaluating and reflecting on a multimethodological intervention. *Systems Research and Behavioral Science*, 40, 2023.

[168] Mark W. Maier. Architecting principles for systems-of-systems. *Systems Engineering*, 1, 1998.

[169] David J. Snowden and Mary E. Boone. A leader's framework for decision making. *Harvard Business Review*, 85, 2007.

[170] Garreth Hardin. The tragedy of the commons garrett hardin. *Science*, 162, 1968.

[171] Cuiting Chen, Hans Gerhard Gross, and Andy Zaidman. Analysis of service diagnosis improvement through increased monitoring granularity. *Software Quality Journal*, 25, 2017.

[172] Ding Li and Xin Cheng. A first-out alarm detection method via association rule mining and correlation analysis. *Entropy*, 26, 2024.

[173] Zineb Simeu-Abazi, Arnaud Lefebvre, and Jean Pierre Derain. A methodology of alarm filtering using dynamic fault tree. *Reliability Engineering and System Safety*, 96, 2011.

[174] Michael Glaß, Martin Lukasiewycz, Christian Haubelt, and Jürgen Teich. Incorporating graceful degradation into embedded system design. In *Proceedings -Design, Automation and Test in Europe, DATE*, 2009.

[175] J Von Neumann. Probabilistic logics and the synthesis of reliable organisms from unreliable components. *Automata Studies*, 34, 1956.

[176] A. Aviziens. Fault-tolerant systems. *IEEE Transactions on Computers*, C-25(12):1304–1312, 1976.

[177] R. E. Lyons and W. Vanderkulk. The use of triple-modular redundancy to improve computer reliability. *IBM Journal of Research and Development*, 6, 2010.

[178] Kurt Rohloff, Richard Schantz, and Yarom Gabay. High-level dynamic resource management for distributed, real-time embedded systems. In *Summer Computer Simulation Conference 2007, SCSC'07, Part of the 2007 Summer Simulation Multiconference, SummerSim'07*, volume 2, 2007.

[179] Michael Pecht, Manu Dube, Marjorie Ann Natishan, Reg Williams, Julian Banner, and Ian Knowles. Evaluation of built-in test. *IEEE Transactions on Aerospace and Electronic Systems*, 37, 2001.

[180] SAE Aerospace. Arp4754: Guidelines for development of civil aircraft and systems. *SAE International Journal of Aerospace*, 2011.

[181] J. F. Ziegler and W. A. Lanford. The effect of sea level cosmic rays on electronic devices. *Journal of Applied Physics*, 52, 1981.

[182] Donghyuk Lee, Yoongu Kim, Gennady Pekhimenko, Samira Khan, Vivek Seshadri, Kevin Chang, and Onur Mutlu. Adaptive-latency dram: Optimizing dram timing for the common-case. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture, HPCA 2015*, 2015.

[183] Anil Kumar Pandey. Signal and power integrity analysis of ddr4 address bus of onboard memory module. 2019.

[184] Kyung Min Koo, Woo Young Chung, Sang Yi Lee, Gyu Han Yoon, and Woo Young Choi. Modeling of statistical variation effects on dram sense amplifier offset voltage. *Micromachines*, 12, 2021.

[185] Onur Mutlu and Jeremie S. Kim. Rowhammer: A retrospective. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39, 2020.

[186] J.L. Walker. The nordstrom-robinson code is algebraic-geometric. *IEEE Transactions on Information Theory*, 43(5):1588–1593, 1997.

[187] Wen-Ku Su, Pei-Yu Shih, Tsung-Ching Lin, and Trieu-Kien Truong. Decoding the extended binary golay code by an iterative invert-one-bit scheme. In *2008 11th IEEE International Conference on Communication Technology*, pages 429–431, 2008.

[188] JEDEC Technology. DDR5 SDRAM. *JESD79-5B, JEDEC, Global Standards for Microelctronics Industry*, 8 2022.

[189] Philippe Fournier-Viger, Jerry Chun, Wei Lin, Rage Uday Kiran, Yun Sing Koh, and Rincy Thomas. A survey of sequential pattern mining. *Data Science and Pattern Recognition*, 1, 2017.

[190] Dongkuan Xu and Yingjie Tian. A comprehensive survey of clustering algorithms. *Annals of Data Science*, 2, 2015.

[191] Mohamed Saouabi and Abdellah Ezzati. Data mining classification algorithms. *International Journal of Mathematics and Computer Science*, 15, 2020.

[192] Ch Sanjeev Kumar Dash, Ajit Kumar Behera, Satchidananda Dehuri, and Ashish Ghosh. An outliers detection and elimination framework in classification task of data mining. *Decision Analytics Journal*, 6, 2023.

[193] Philippe Fournier-Viger, Jiaxuan Li, Jerry Chun Wei Lin, Tin Truong Chi, and R. Uday Kiran. Mining cost-effective patterns in event logs. *Knowledge-Based Systems*, 191, 2020.

[194] Killian Murphy, Antoine Lavignotte, and Catherine Lepers. Fault prediction for heterogeneous telecommunication networks using machine learning: A survey. *IEEE Transactions on Network and Service Management*, 21, 2024.

[195] Antonio Massaro, Dimitre Kostadinov, Alonso Silva, Alexander Obeid Guzman, and Armen Aghasaryan. Predicting network hardware faults through layered treatment of alarms logs. *Entropy*, 25, 2023.

[196] Mulugeta Weldezgina Asres, Million Abayneh Mengistu, Pino Castrogiovanni, Lorenzo Bottaccioli, Enrico Macii, Edoardo Patti, and Andrea Acquaviva. Supporting telecommunication alarm management system with trouble ticket prediction. *IEEE Transactions on Industrial Informatics*, 17, 2021.

[197] Gary M Weiss. Predicting telecommunication equipment failures from sequences of network alarms. *Handbook of Knowledge Discovery and Data Mining*, 2001.

[198] Ahmad S. Kazmi. Application of statistical sampling to predict faults from real time alarm data. In *Proceedings of the 14th IEEE International Multitopic Conference 2011, INMIC 2011*, 2011.

[199] Željko Deljac, Mirko Randić, and Gordan Krčelić. A multivariate approach to predicting quantity of failures in broadband networks based on

a recurrent neural network. *Journal of Network and Systems Management*, 24, 2016.

[200] Aurora Esteban, Amelia Zafra, and Sebastián Ventura. Data mining in predictive maintenance systems: A taxonomy and systematic review. *WIREs Data Mining and Knowledge Discovery*, 12(5):e1471, 2022.

[201] Amit Kumar, Archana Mantri, and Rubina Dutta. Development of an augmented reality-based scaffold to improve the learning experience of engineering students in embedded system course. *Computer Applications in Engineering Education*, 29, 2021.

[202] Zhihao Cui, Oi lam Ng, and Morris Siu Yung Jong. Integration of computational thinking with mathematical problem-based learning: Insights on affordances for learning. *Educational Technology and Society*, 26, 2023.

[203] Yazeed Yasin Ghadi, Tehseen Mazhar, Tamara Al Shloul, Tariq Shahzad, Umair Ahmad Salaria, Arfan Ahmed, and Habib Hamam. Machine learning solutions for the security of wireless sensor networks: A review. *IEEE Access*, 12, 2024.

[204] Jasneet Kaur, M. Arif Khan, Mohsin Iftikhar, Muhammad Imran, and Qazi Emad Ul Haq. Machine learning techniques for 5g and beyond. *IEEE Access*, 9, 2021.

[205] Steven F. Barrett. Embedded systems design. *Synthesis Lectures on Digital Circuits and Systems*, Part F1960, 2024.

[206] Philip Axer, Rolf Ernst, Heiko Falk, Alain Girault, Daniel Grund, Nan Guan, Bengt Jonsson, Peter Marwedel, Jan Reineke, Christine Rochange, Maurice Sebastian, Reinhard Von Hanxleden, Reinhard Wilhelm, and Wang Yi. Building timing predictable embedded systems. *ACM Transactions on Embedded Computing Systems*, 13, 2014.

[207] Sassan Ahmadi. *5G NR: Architecture, technology, implementation, and operation of 3GPP new radio standards*. Academic Press, 2019.

[208] Technical Specification. Ts 136 104 - v8.4.0 - lte; evolved universal terrestrial radio access (e-utra); base station (bs) radio transmission and reception (3gpp ts 36.104 version 8.4.0 release 8). *European Telecommunications*, 0, 2009.

[209] 3GPP. Lte; evolved universal terrestrial radio access (e-utra) and evolved universal terrestrial radio access network (e-utran); overall description; stage 2 (3gpp ts 36.300 version 8.12.0 release 8). *ETSI TS 136 300*, 2010.

[210] 3GPP TS 38.300. 5g; nr; overall description; stage-2. *Etsi Ts 138 300 - V15.3.1*, 15.3.1, 2018.

[211] ETSI. 5g; system architecture for the 5g system (3gpp ts 23.501 version 15.2.0 release 15). *European Telecommunications Standards Institute. Tech. Rep. V15.2.0*, 15, 2018.

[212] M Friendly. Statistical computing and graphics; corrgrams exploratory displays for correlation matrices. *American Statistician*, 2010.

[213] Leland Wilkinson and Michael Friendly. History corner the history of the cluster heat map, 2009.

[214] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. An introduction statistical machine learning with applications in python. *Springer Texts in Statistics*, 2023.

[215] Fariha Sohil, Muhammad Umair Sohali, and Javid Shabbir. An introduction to statistical learning with applications in r. *Statistical Theory and Related Fields*, 6, 2022.

[216] R. Senthil Kumar, Anidha Arulanandham, Suresh Arumugam, G. Dinesh, R. Thirukkumaran, and R. Subashmoorthy. Analysis of classification and clustering techniques for ambient aqi using machine learning algorithms. In *Proceedings - 4th International Conference on Smart Systems and Inventive Technology, ICSSIT 2022*, 2022.

[217] Kelsy Cabello-Solorzano, Isabela Ortigosa de Araujo, Marco Peña, Luís Correia, and Antonio J. Tallón-Ballesteros. The impact of data normalization on the accuracy of machine learning algorithms: A comparative analysis. In *Lecture Notes in Networks and Systems*, volume 750 LNNS, 2023.

[218] Sridhar Ramaswamy, Rajeev Rastogi, and Kyuseok Shim. Efficient algorithms for mining outliers from large data sets. *SIGMOD Record (ACM Special Interest Group on Management of Data)*, 29, 2000.

[219] Fabrizio Angiulli and Clara Pizzuti. Fast outlier detection in high dimensional spaces. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, volume 2431 LNAI, 2002.

[220] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey, 2009.

[221] Tom Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 27, 2006.

[222] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20, 1960.

[223] Pierre Baldi, Søren Brunak, Yves Chauvin, Claus A.F. Andersen, and Henrik Nielsen. Assessing the accuracy of prediction algorithms for classification: An overview, 2000.

[224] Jisoo Yang, Dave B. Minturn, and Frank Hady. When poll is better than interrupt. In *Proceedings of FAST 2012: 10th USENIX Conference on File and Storage Technologies*, 2012.

[225] Nicasio Canino, Stefano Di Matteo, Daniele Rossi, and Sergio Saponara. Hw-sw interface design and implementation for error logging and reporting for ras improvement. *IEEE Access*, 12:60081–60094, 2024.

[226] Alfredo Benso and Paolo Prinetto. *Fault Injection Techniques and Tools for Embedded Systems Reliability Evaluation*. Springer New York, NY, 2004.

[227] Mei Chen Hsueh, Timothy K. Tsai, and Ravishankar K. Iyer. Fault injection techniques and tools, 1997.

[228] Dong Wan Kim and Mattan Erez. Relaxfault memory repair. In *Proceedings - 2016 43rd International Symposium on Computer Architecture, ISCA 2016*, 2016.

[229] Dong Tang, Peter Carruthers, Zuheir Totari, and Michael W. Shapiro. Assessment of the effect of memory page retirement on system ras against hardware faults. *Proceedings of the International Conference on Dependable Systems and Networks*, 2006:365–370, 2006.

[230] JEDEC Solid State Technology Association et al. Ddr5 sdram standard. *JEDEC Standard*, (79-5):465, 2020.

[231] Alireza Shamsoshoara, Ashwija Korenda, Fatemeh Afghah, and Sherali Zeadally. A survey on physical unclonable function (puf)-based security solutions for internet of things, 2020.

[232] Cemal Yilmaz. Using hardware performance counters for fault localization. In *Proceedings - 2nd International Conference on Advances in System Testing and Validation Lifecycle, VALID 2010*, 2010.

[233] Leandro Lanzieri, Gianluca Martino, Goerschwin Fey, Holger Schlarb, Thomas C. Schmidt, and Matthias Wählisch. A review of techniques for ageing detection and monitoring on embedded systems. *ACM Comput. Surv.*, 57(1), October 2024.

[234] Manu Shamsa and David Lerner. Defect mechanisms responsible for silent data errors. In *IEEE International Reliability Physics Symposium Proceedings*, 2024.

[235] Ravishankar K. Iyer, Nithin M. Nakka, Zbigniew T. Kalbarczyk, and Subhasish Mitra. Recent advances and new avenues in hardware-level reliability support. *IEEE Micro*, 25, 2005.

[236] Nikhil Pandit, Zbigniew Kalbarczyk, and Ravishankar K. Iyer. Effectiveness of machine checks for error diagnostics. In *Proceedings of the International Conference on Dependable Systems and Networks*, 2009.

[237] Roger C. Goerl, Paulo R.C. Villa, Letícia B. Poehls, Eduardo A. Bezerra, and Fabian L. Vargas. An efficient edac approach for handling multiple bit upsets in memory array. *Microelectronics Reliability*, 88-90, 2018.

[238] Jaehoon Lee, Hyeonuk Son, Seohyun Kang, Dahyun Kang, Dongkwan Han, Jongsin Yun, Artur Pogiel, Etienne Racine, Krzysztof Jurga, Lori Schramm, and Martin Keim. Diagnosis of intermittent faults and corresponding algorithm development beyond 5nm technologies. In *Proceedings - International Test Conference*, 2024.

[239] Charles Stroud, John Sunwoo, Srinivas Garimella, and Jonathan Harris. Built-in self-test for system-on-chip: A case study. In *Proceedings - International Test Conference*, 2004.

[240] Tsung Yu Wu, Po Yuan Chen, Cheng Wen Wu, and Ding Ming Kwai. Improving testing and diagnosis efficiency for regular memory arrays. In *Proceedings of 2010 International Symposium on VLSI Design, Automation and Test, VLSI-DAT 2010*, 2010.

[241] Sergey Ostrikov. Resilient boot. In *Proceedings of the 2021 IEEE International Conference on Cyber Security and Resilience, CSR 2021*, 2021.

[242] Andrea Floridia and Ernesto Sanchez. On-line self-test mechanism for dual-core lockstep system-on-chips. *Microelectronics Reliability*, 112, 2020.

[243] Mong Tee Sim and Yanyan Zhuang. A dual lockstep processor system-on-a-chip for fast error recovery in safety-critical applications. In *IECON Proceedings (Industrial Electronics Conference)*, volume 2020-October, 2020.

[244] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. Operating system concepts seventh edition. *Fundamentos de Bases de Datos*, 2005.

[245] Subrata Saha and Subhodip Mukherjee. *Basic Computation and Programming with C*. Cambridge University Press, 2016.

[246] Steven A. Przybylski. Cache and memory hierarchy design : a performance-directed approach. *Elsevier Science*, 2014.

[247] Jasmin Ajanovic. *PCI Express*, pages 1487–1498. Springer US, Boston, MA, 2011.

[248] Yixin Luo, Saugata Ghose, Yu Cai, Erich F. Haratsch, and Onur Mutlu. Improving 3d nand flash memory lifetime by tolerating early retention loss and process variation. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 2, 2018.

[249] Omin Kwon, Kyungjun Oh, Jae Yong Lee, Myungsuk Kim, and Jihong Kim. Star: Improving lifetime and performance of high-capacity modern ssds using state-aware randomizer. *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9, 2025.

[250] Michela Borghesi, Cristian Zambelli, Rino Micheloni, and Stefano Bonnini. Modeling 3d nand flash with nonparametric inference on regression coefficients for reliable solid-state storage. *Future Internet*, 15(10), 2023.

[251] Kai Zhao, Wenzhe Zhao, Hongbin Sun, Tong Zhang, Xiaodong Zhang, and Nanning Zheng. Ldpc-in-ssd: Making advanced error correction codes work effectively in solid state drives. In *Proceedings of the 11th USENIX Conference on File and Storage Technologies, FAST 2013*, 2013.

[252] Borja Peleato, Haleh Tabrizi, Rajiv Agarwal, and Jeffrey Ferreira. Ber-based wear leveling and bad block management for nand flash. In *IEEE International Conference on Communications*, volume 2015-September, 2015.

[253] Young Tack Jin, Sungjoon Ahn, and Sungjin Lee. Performance analysis of nvme ssd-based all-flash array systems. In *Proceedings - 2018 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2018*, 2018.

[254] Yunfei Gu, Chentao Wu, and Xubin He. Exploit both smart attributes and nand flash wear characteristics to effectively forecast ssd-based storage

failures in clusters. In *Proceedings of the 2024 USENIX Annual Technical Conference, ATC 2024*, 2024.

[255] Jisung Park, Myungsuk Kim, Myoungjun Chun, Lois Orosa, Jihong Kim, and Onur Mutlu. Reducing solid-state drive read latency by optimizing read-retry. In *International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS*, 2021.

[256] Micron Technology, Inc. Ssds and SMART data. `https://www.crucial.com/support/articles-faq-ssd/ssds-and-smart-data`, 2024. Accessed: 2026-01-08.

[257] Yunjae Lee and Seehwan Yoo. Secure boot + measured boot: Guaranteeing the integrity of the linux booting process. *KIISE Transactions on Computing Practices*, 23, 2017.

[258] Andrew Regenscheid. Platform firmware resiliency guidelines. *NIST Special Publication*, 193, 2018.

[259] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Error characterization, mitigation, and recovery in flash-memory-based solid-state drives. *Proceedings of the IEEE*, 105, 2017.

[260] Georg Wassen and Stefan Lankes. Within a general-purpose operating system. In *OpenAccess Series in Informatics*, volume 47, 2015.

[261] Ryan Aalund and Vincent Philip Paglioni. Enhancing reliability in embedded systems hardware: A literature survey. *IEEE Access*, 13, 2025.

[262] Charles P. Shelton and Philip Koopman. Improving system dependability with functional alternatives. In *Proceedings of the International Conference on Dependable Systems and Networks*, 2004.

[263] Marti Alonso, David Andreu, Ramon Canal, Stefano Di Carlo, Odysseas Chatzopoulos, Cristiano Chenet, Juanjo Costa, Andreu Girones, Dimitris Gizopoulos, George Papadimitriou, Enric Morancho, Beatriz Otero, and Alessandro Savino. Special session: Security and ras in the computing

continuum. In *Proceedings - IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems, DFT*, 2024.

[264] Philip M. Wells, Koushik Chakraborty, and Gurindar S. Sohi. Adapting to intermittent faults in multicore systems. *Operating Systems Review (ACM)*, 2008.

[265] Jens Anders, Pablo Andreu, Bernd Becker, Steffen Becker, Riccardo Cantoro, Nikolaos I. Deligiannis, Nourhan Elhamawy, Tobias Faller, Carles Hernandez, Nele Mentens, Mahnaz Namazi Rizi, Ilia Polian, Abolfazl Sajadi, Mathias Sauer, Denis Schwachhofer, Matteo Sonza Reorda, Todor Stefanov, Ilya Tuzov, Stefan Wagner, and Nusa Zidaric. A survey of recent developments in testability, safety and security of risc-v processors. In *Proceedings of the European Test Workshop*, volume 2023-May, 2023.

[266] Balwinder Singh, Arun Khosla, and Sukhleen B. Narang. Cost modeling for soc modules testing. *International Journal of Information Engineering and Electronic Business*, 5, 2013.

[267] Abidin Esidir, Sami Pekdemir, Mustafa Kalay, and Mustafa Serdar Onses. Ultradurable embedded physically unclonable functions. *ACS Applied Materials and Interfaces*, 16, 2024.

[268] Fast signal quality monitoring for coherent communications enabled by cnn-based evm estimation. *Journal of Optical Communications and Networking*, 13, 2021.

[269] Deep Medhi and Karthik Ramasamy. Chapter 1 - networking and network routing: An introduction. In Deep Medhi and Karthik Ramasamy, editors, *Network Routing (Second Edition)*, The Morgan Kaufmann Series in Networking, pages 2–29. Morgan Kaufmann, Boston, second edition edition, 2018.

[270] Ieee standard for ethernet. *IEEE Std 802.3-2022 (Revision of IEEE Std 802.3-2018)*, pages 1–7025, 2022.

[271] Jozef Babiarz, Roman M. Krzanowski, Kaynam Hedayat, Kiho Yum, and Al Morton. A Two-Way Active Measurement Protocol (TWAMP). RFC 5357, October 2008.

ISBN 978-91-7485-757-3
ISSN 1651-4238

Address:
P.O. Box 883, SE-721 23 Västerås. Sweden
P.O. Box 325, SE-631 05 Eskilstuna. Sweden
E-mail: info@mdu.se **Web:** www.mdu.se

Mälardalen University